

0-Introduction

May 20, 2026

1 Advanced Python and Machine Learning Course

With this course, we provide an overview of important Machine Learning concepts and introduce more advanced **Python 3** programming topics. In the following notebooks, we will cover the areas of **regression**, **classification**, and **knowledge discovery** as well as some of the most relevant Python libraries for Machine Learning. Of course, we won't be able to provide an exhaustive presentation of all topics and their mathematical depth. The goal of this course is to give you a practical introduction with some real-world examples. There might be some additional topics, objects or commands you later find useful but have not seen in this course. However, we believe that you really become “fluent” in a programming language and the different Machine Learning tools by trying to implement your own projects and learn from difficulties you encounter along the way.

1.0.1 Small overview of working with Jupyter Lab

In this course, we make use of Jupyter Notebooks, which allow us to combine executable code with text in one document, increasing code readability and documentation.

In Jupyter Notebooks, you have different kinds of **cells** you can make use of: **Code**, **Markdown**, and **Raw**. In order to select a cell for execution, you can just click on the relevant part of the notebook and a blue bar on the left will highlight the currently selected cell, like in the image below.



Try to run the following code cell with `/ Run Selected Cells` in the panel above or *Shift+Enter*!

```
[ ]: print("It works!")
```

You can stop the execution of a cell with `/ Interrupt Kernel` in the panel above!

1.0.2 Markdown mode

Try to add a cell below, change to **Markdown** mode and write a note!

1. Add a cell with + in the panel above or *Esc+B*
2. Change to **Markdown** in the panel above or *Esc+M*
3. As before, run the cell with / **Run Selected Cells** in the panel above or *Shift+Enter*

You can make notes inside the notebooks if you like! In order to edit a Markdown cell double-click somewhere in the cell!

In Markdown mode you can use the following special characters:

2 Title

2.1 Subtitle

- bullet point 1
- bullet point 2

italic font **bold**

highlighted code

*** (that's a separator line)

You can also use L^AT_EX if you want, like $\sum_{k=0}^{\infty} q^k = \frac{1}{1-q}$ (double-click here to see how it is used).

2.1.1 Code mode

Creating a **Code** cell works similarly. You can make use of it later on!

1. Add a cell with + in the panel above or *Esc+B*
 2. Change to **Code** in the panel above or *Esc+Y*
 3. Again, run the cell with / **Run Selected Cells** in the panel above or *Shift+Enter*
-

2.1.2 Some Remarks

- Delete a cell with right-click on the cell and the “Delete Cells” option
 - **Be careful which cells you delete** because it might not be possible to retrieve them!
 - In case, you accidentally deleted something try *Esc+Z* to retrieve a cell or *Ctrl+Z* to retrieve changes within a cell
 - Save your notebooks with the *save icon* in the panel above or with *Ctrl+S*! **Try to save after every bigger change!** Depending on your machine, Jupyter Lab might crash from time to time.
 - Try not to open too many windows in Jupyter Lab. This might lead to a crash sometimes.
 - If the execution of a code cell does not stop for some reason, i.e. the symbol on the left of your code cell looks like **[*]**: for a long time, stop the cell execution with / **Interrupt Kernel** in the panel above!
-

2.1.3 Final Test

Let's see whether your setup is ready for the course.

If you have your own local installation of Python please make sure that you have all libraries installed, in particular TensorFlow. If not, you can follow the instructions provided in the [YouTube video](#) and the documentation on “**Setting up an environment with Anaconda Navigator**” and “**Add a package to an environment with Anaconda Navigator**” I sent with my first e-mail attached as a PDF!

Please execute the cell at the bottom to check whether all libraries are available that we will use later on.

```
[ ]: import numpy
import pandas
import matplotlib
import seaborn
import scipy
import sklearn
import numba
import deepsmiles
import rdkit
import xarray
import tensorflow
from tensorflow import keras

print("Everything is working!")
```

Class_1-Functions_and_Classes

May 20, 2026

1 1. Functions and Classes

We start by reviewing some basics about functions and classes and see how to

- define our own functions and
- work with classes and subclasses.

Keywords: `def`, `class`, `return`, `help`, `self`

1.1 Functions

Most of the time, there are certain operations which are performed several times in a program. For example, the application of a particular analysis or calculation with different input data, plotting of figures after a new measurement and much more. Examples are **built-in functions** like `print` or `type` and other functions like `numpy.mean()`. The basic steps of how a function works are:

1. The function is called
2. The function executes some action
3. The function returns some value

Let us consider our own example of a function. The function is defined by a **function signature** and a **function body**. The signature starts with `def` and gives the name of the function, the arguments it expects and ends with a colon `:`. The function body contains the code which is executed when the function is called. Usually, a `return` statement indicates what will be returned if the function is called. However, it is not necessary to return anything and so `return` can be omitted, too.

In the following, we study a function which converts degrees Celsius to degrees Fahrenheit and vice versa.

```
[1]: def convert_temp(degrees_celsius = None, degrees_fahrenheit = None):
    """
    This function converts degrees Celsius to degrees Fahrenheit and
    vice versa.

    degree_celsius: Input value in degrees Celsius to be converted to
                    degrees Fahrenheit.
    degree_fahrenheit: Input value in degrees Fahrenheit to be converted
                      to degrees Celsius.
```

```

    return: Temperature in the converted units.
    '''

    if degrees_celsius is not None:
        degrees_fahrenheit = degrees_celsius * 9/5 + 32
        print("{} in °C are {} °F".format(degrees_celsius, degrees_fahrenheit))
        return degrees_fahrenheit
        print("This will never be printed")

    else:
        degrees_celsius = (degrees_fahrenheit - 32) * 5/9
        print("{} in °F are {} °C".format(degrees_fahrenheit, degrees_celsius))
        return degrees_celsius

```

```

[2]: deg_F = convert_temp(degrees_celsius = 30)
      print(deg_F)

```

```

30 in °C are 86.0 °F
86.0

```

```

[3]: deg_C = convert_temp(degrees_fahrenheit = 23)
      print(deg_C)

```

```

23 in °F are -5.0 °C
-5.0

```

```

[4]: help(convert_temp)

```

Help on function convert_temp in module __main__:

```

convert_temp(degrees_celsius=None, degrees_fahrenheit=None)
    This function converts degrees Celsius to degrees Fahrenheit and
    vice versa.

    degree_celsius: Input value in degrees Celsius to be converted to
                    degrees Fahrenheit.
    degree_fahrenheit: Input value in degrees Fahrenheit to be converted
                      to degrees Celsius.
    return: Temperature in the converted units.

```

```

[5]: print(degrees_celsius)

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[5], line 1
----> 1 print(degrees_celsius)

```

```
NameError: name 'degrees_celsius' is not defined
```

1.2 Classes and Object-oriented programming

In object-oriented programming (OOP), objects contain information in the form of *attributes* or *properties* and *methods* with which particular operations can be performed. In most OOPs, objects are **instances of classes**. OOP allow modularity and reusability in your code. Let us see what this exactly means in the following.

```
[6]: class person:
      def __init__(self, name, age):
          self.name = name
          self.age = age

      def say_hi(self):
          print("{} says: Hi!".format(self.name))

      def younger_30(self):
          """
          Print True if younger than 30.
          """
          return self.age < 30
```

You can think of a class as a **blueprint** of an object. Suppose you want to manage different people with your program. You will need to add different persons who all have similar properties. The `person` class allows us to create many people who have their individual properties.

An “example” or a “realisation” of a class is usually referred to as an **instance** of the class.

```
[7]: first_person = person('Alice', 30)
```

```
[8]: first_person
```

```
[8]: <__main__.person at 0x10e07f990>
```

```
[9]: first_person.name
```

```
[9]: 'Alice'
```

```
[10]: first_person.age
```

```
[10]: 30
```

```
[17]: first_person.say_hi()
```

```
Alice says: Hi!
```

Note that `say_hi` is a **function** similar to those we defined before. A class function is referred to as a **class method**. More precisely, `say_hi` is an instance method because it requires the instance object `first_person` in order to be callable.

```
[11]: second_person = person('Bob', 29)
```

```
[12]: first_person
```

```
[12]: <__main__.person at 0x10e07f990>
```

```
[13]: second_person
```

```
[13]: <__main__.person at 0x10e81aad0>
```

```
[14]: first_person == second_person
```

```
[14]: False
```

```
[15]: first_person.name
```

```
[15]: 'Alice'
```

```
[16]: second_person.name
```

```
[16]: 'Bob'
```

```
[18]: print("Is {} younger than 30? Answer: {}".format(
        first_person.name,
        first_person.younger_30()
    ))
```

```
Is Alice younger than 30? Answer: False
```

```
[19]: help(person)
```

```
Help on class person in module __main__:
```

```
class person(builtins.object)
|   person(name, age)
|
|   Methods defined here:
|
|   __init__(self, name, age)
|       Initialize self. See help(type(self)) for accurate signature.
|
|   say_hi(self)
|
|   younger_30(self)
|       Print True if younger than 30.
```

```

|
| -----
| Data descriptors defined here:
|
|   __dict__
|       dictionary for instance variables (if defined)
|
|   __weakref__
|       list of weak references to the object (if defined)

```

```
[20]: help(person.younger_30)
```

Help on function younger_30 in module __main__:

```

younger_30(self)
    Print True if younger than 30.

```

1.2.1 Subclasses

It is possible to define subclasses which build upon other classes and **inherit** their structure and methods.

```
[21]: class child(person):
        def __init__(self, name, age, mother):
            super(child, self).__init__(name, age)
            self.mother = mother

        def print_mother(self):
            print(self.mother.name)
```

```
[22]: newborn = child('Charlie', 1, first_person)
```

```
[23]: newborn.say_hi()
```

Charlie says: Hi!

Note that for the class `child` we did not define a method `say_hi`. However, the method was *inherited* from the “parent” class `person`.

```
[24]: newborn.print_mother()
```

Alice

Why is this relevant?

```
[25]: print(type(1))
```



```
<class 'int'>
```

```
[26]: help(int)
```

Help on class int in module builtins:

```
class int(object)
|   int([x]) -> integer
|   int(x, base=10) -> integer
|
|   Convert a number or string to an integer, or return 0 if no arguments
|   are given.  If x is a number, return x.__int__().  For floating point
|   numbers, this truncates towards zero.
|
|   If x is not a number or if base is given, then x must be a string,
|   bytes, or bytearray instance representing an integer literal in the
|   given base.  The literal can be preceded by '+' or '-' and be surrounded
|   by whitespace.  The base defaults to 10.  Valid bases are 0 and 2-36.
|   Base 0 means to interpret the base from the string as an integer literal.
|   >>> int('0b100', base=0)
|   4
|
|   Built-in subclasses:
|       bool
|
|   Methods defined here:
|
|   __abs__(self, /)
|       abs(self)
|
|   __add__(self, value, /)
|       Return self+value.
|
|   __and__(self, value, /)
|       Return self&value.
|
|   __bool__(self, /)
|       True if self else False
|
|   __ceil__(...)
|       Ceiling of an Integral returns itself.
|
|   __divmod__(self, value, /)
|       Return divmod(self, value).
|
|   __eq__(self, value, /)
|       Return self==value.
```

```

|  __float__(self, /)
|      float(self)
|
|  __floor__(...)
|      Flooring an Integral returns itself.
|
|  __floordiv__(self, value, /)
|      Return self//value.
|
|  __format__(self, format_spec, /)
|      Default object formatter.
|
|  __ge__(self, value, /)
|      Return self>=value.
|
|  __getattr__(self, name, /)
|      Return getattr(self, name).
|
|  __getnewargs__(self, /)
|
|  __gt__(self, value, /)
|      Return self>value.
|
|  __hash__(self, /)
|      Return hash(self).
|
|  __index__(self, /)
|      Return self converted to an integer, if self is suitable for use as an
index into a list.
|
|  __int__(self, /)
|      int(self)
|
|  __invert__(self, /)
|      ~self
|
|  __le__(self, value, /)
|      Return self<=value.
|
|  __lshift__(self, value, /)
|      Return self<<value.
|
|  __lt__(self, value, /)
|      Return self<value.
|
|  __mod__(self, value, /)
|      Return self%value.
|

```

```

|  __mul__(self, value, /)
|      Return self*value.
|
|  __ne__(self, value, /)
|      Return self!=value.
|
|  __neg__(self, /)
|      -self
|
|  __or__(self, value, /)
|      Return self|value.
|
|  __pos__(self, /)
|      +self
|
|  __pow__(self, value, mod=None, /)
|      Return pow(self, value, mod).
|
|  __radd__(self, value, /)
|      Return value+self.
|
|  __rand__(self, value, /)
|      Return value&self.
|
|  __rdivmod__(self, value, /)
|      Return divmod(value, self).
|
|  __repr__(self, /)
|      Return repr(self).
|
|  __rfloordiv__(self, value, /)
|      Return value//self.
|
|  __rlshift__(self, value, /)
|      Return value<<self.
|
|  __rmod__(self, value, /)
|      Return value%self.
|
|  __rmul__(self, value, /)
|      Return value*self.
|
|  __ror__(self, value, /)
|      Return value|self.
|
|  __round__(...)
|      Rounding an Integral returns itself.

```

```

|         Rounding with an ndigits argument also returns an integer.
|
|     __rpow__(self, value, mod=None, /)
|         Return pow(value, self, mod).
|
|     __rrshift__(self, value, /)
|         Return value>>self.
|
|     __rshift__(self, value, /)
|         Return self>>value.
|
|     __rsub__(self, value, /)
|         Return value-self.
|
|     __rtruediv__(self, value, /)
|         Return value/self.
|
|     __rxor__(self, value, /)
|         Return value^self.
|
|     __sizeof__(self, /)
|         Returns size in memory, in bytes.
|
|     __sub__(self, value, /)
|         Return self-value.
|
|     __truediv__(self, value, /)
|         Return self/value.
|
|     __trunc__(...)
|         Truncating an Integral returns itself.
|
|     __xor__(self, value, /)
|         Return self^value.
|
|     as_integer_ratio(self, /)
|         Return integer ratio.
|
|         Return a pair of integers, whose ratio is exactly equal to the original
int    and with a positive denominator.
|
|         >>> (10).as_integer_ratio()
|         (10, 1)
|         >>> (-10).as_integer_ratio()
|         (-10, 1)
|         >>> (0).as_integer_ratio()
|         (0, 1)

```

```

|
| bit_count(self, /)
|     Number of ones in the binary representation of the absolute value of
self.
|
|     Also known as the population count.
|
|     >>> bin(13)
|     '0b1101'
|     >>> (13).bit_count()
|     3
|
| bit_length(self, /)
|     Number of bits necessary to represent self in binary.
|
|     >>> bin(37)
|     '0b100101'
|     >>> (37).bit_length()
|     6
|
| conjugate(...)
|     Returns self, the complex conjugate of any int.
|
| to_bytes(self, /, length=1, byteorder='big', *, signed=False)
|     Return an array of bytes representing an integer.
|
|     length
|         Length of bytes object to use.  An OverflowError is raised if the
|         integer is not representable with the given number of bytes.  Default
|         is length 1.
|     byteorder
|         The byte order used to represent the integer.  If byteorder is 'big',
|         the most significant byte is at the beginning of the byte array.  If
|         byteorder is 'little', the most significant byte is at the end of the
|         byte array.  To request the native byte order of the host system, use
|         `sys.byteorder` as the byte order value.  Default is to use 'big'.
|     signed
|         Determines whether two's complement is used to represent the integer.
|         If signed is False and a negative integer is given, an OverflowError
|         is raised.
|
| -----
| Class methods defined here:
|
| from_bytes(bytes, byteorder='big', *, signed=False) from builtins.type
|     Return the integer represented by the given array of bytes.
|
|     bytes

```

```

|         Holds the array of bytes to convert. The argument must either
|         support the buffer protocol or be an iterable object producing bytes.
|         Bytes and bytearray are examples of built-in objects that support the
|         buffer protocol.
|     bytearray
|         The byte order used to represent the integer. If byteorder is 'big',
|         the most significant byte is at the beginning of the byte array. If
|         byteorder is 'little', the most significant byte is at the end of the
|         byte array. To request the native byte order of the host system, use
|         `sys.byteorder' as the byte order value. Default is to use 'big'.
|     signed
|         Indicates whether two's complement is used to represent the integer.
|
|     -----
|     Static methods defined here:
|
|     __new__(*args, **kwargs) from builtins.type
|         Create and return a new object. See help(type) for accurate signature.
|
|     -----
|     Data descriptors defined here:
|
|     denominator
|         the denominator of a rational number in lowest terms
|
|     imag
|         the imaginary part of a complex number
|
|     numerator
|         the numerator of a rational number in lowest terms
|
|     real
|         the real part of a complex number

```

[27]: `help(list)`

Help on class list in module builtins:

```

class list(object)
|   list(iterable=(), /)
|
|   Built-in mutable sequence.
|
|   If no argument is given, the constructor creates a new empty list.
|   The argument must be an iterable if specified.
|
|   Methods defined here:

```

```

|  __add__(self, value, /)
|      Return self+value.
|
|  __contains__(self, key, /)
|      Return key in self.
|
|  __delitem__(self, key, /)
|      Delete self[key].
|
|  __eq__(self, value, /)
|      Return self==value.
|
|  __ge__(self, value, /)
|      Return self>=value.
|
|  __getattr__(self, name, /)
|      Return getattr(self, name).
|
|  __getitem__(...)
|      x.__getitem__(y) <==> x[y]
|
|  __gt__(self, value, /)
|      Return self>value.
|
|  __iadd__(self, value, /)
|      Implement self+=value.
|
|  __imul__(self, value, /)
|      Implement self*=value.
|
|  __init__(self, /, *args, **kwargs)
|      Initialize self.  See help(type(self)) for accurate signature.
|
|  __iter__(self, /)
|      Implement iter(self).
|
|  __le__(self, value, /)
|      Return self<=value.
|
|  __len__(self, /)
|      Return len(self).
|
|  __lt__(self, value, /)
|      Return self<value.
|
|  __mul__(self, value, /)
|      Return self*value.

```

```

|  __ne__(self, value, /)
|      Return self!=value.
|
|  __repr__(self, /)
|      Return repr(self).
|
|  __reversed__(self, /)
|      Return a reverse iterator over the list.
|
|  __rmul__(self, value, /)
|      Return value*self.
|
|  __setitem__(self, key, value, /)
|      Set self[key] to value.
|
|  __sizeof__(self, /)
|      Return the size of the list in memory, in bytes.
|
|  append(self, object, /)
|      Append object to the end of the list.
|
|  clear(self, /)
|      Remove all items from list.
|
|  copy(self, /)
|      Return a shallow copy of the list.
|
|  count(self, value, /)
|      Return number of occurrences of value.
|
|  extend(self, iterable, /)
|      Extend list by appending elements from the iterable.
|
|  index(self, value, start=0, stop=9223372036854775807, /)
|      Return first index of value.
|
|      Raises ValueError if the value is not present.
|
|  insert(self, index, object, /)
|      Insert object before index.
|
|  pop(self, index=-1, /)
|      Remove and return item at index (default last).
|
|      Raises IndexError if list is empty or index is out of range.
|
|  remove(self, value, /)

```



```

|     Remove first occurrence of value.
|
|     Raises ValueError if the value is not present.
|
| reverse(self, /)
|     Reverse *IN PLACE*.
|
| sort(self, /, *, key=None, reverse=False)
|     Sort the list in ascending order and return None.
|
|     The sort is in-place (i.e. the list itself is modified) and stable (i.e.
the
|     order of two equal elements is maintained).
|
|     If a key function is given, apply it once to each list item and sort
them,
|     ascending or descending, according to their function values.
|
|     The reverse flag can be set to sort in descending order.
|
| -----
| Class methods defined here:
|
| __class_getitem__(...) from builtins.type
|     See PEP 585
|
| -----
| Static methods defined here:
|
| __new__(*args, **kwargs) from builtins.type
|     Create and return a new object.  See help(type) for accurate signature.
|
| -----
| Data and other attributes defined here:
|
| __hash__ = None

```

Class_2-Python_Concepts

May 20, 2026

1 2. Python Concepts

We introduce some more advanced Python concepts, which might prove useful in implementing your Machine Learning projects. In this notebook, we focus on the following topics which, in particular, review some concepts related to functions in Python:

- f-strings
- subplots with Matplotlib
- type annotations and type hints
- lambda functions
- generators
- built-in functions map and filter
- *args and **kwargs
- decorators
- speeding up function executions with Numba

Keywords: `f"String"`, `find`, `plt.subplots`, `plt.figure.add_subplot`, `np.ravel`, `lambda x: ...`, `yield`, `map`, `filter`, `*args`, `**kwargs`, `@decorator`, `@jit`, `%timeit`

1.1 Important Python Libraries

There are several useful Python libraries for scientific computing and Machine Learning in Python.

- **NumPy** extends basic Python data structures (like lists) and provides efficient numerical functions for computations with large data arrays.
- **SciPy** builds on NumPy and provides extended functionality for numerical and statistical methods.
- **Pandas** incorporates data frames (similar to programming language R) and allows for more statistical analyses.
- **Matplotlib** is the standard plotting library in Python.
- **Seaborn** builds on Matplotlib and, in particular, extends Pandas functionality to create appealing plots.
- **Scikit-learn** is a powerful machine learning library that provides a wide range of learning algorithms. It builds upon NumPy, SciPy, and Matplotlib.

- TensorFlow is a library for efficient Machine Learning implementation and a standard library for Deep Learning.
- Keras is a high-level Deep Learning library built on top of TensorFlow, which simplifies creating, training, and evaluating neural networks.
- PyTorch is another standard library for Deep Learning, adhering more closely to basic Python principles.

In this course, we will mostly work with NumPy, Matplotlib, scikit-learn, and TensorFlow / Keras. You will be able to load these libraries like this

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from sklearn import linear_model
```

1.2 f-Strings

Recommended string formatting since Python 3.6:

```
[7]: float_var = 3.141592

print(f'String formatting allows including variables like {float_var:.2f}.')
```

String formatting allows including variables like 3.14.

And while we are talking about strings: When you want to find a substring in a string, you can use

```
[8]: example_string = f'String formatting allows including variables like {float_var:
↪.2f}.'
```

```
search_string = 'variable'
```

```
index_of_search = example_string.find(search_string)
```

```
search_string_len = len(search_string)
```

```
print(f"Starting from index {index_of_search}: '{example_string[index_of_search:
↪index_of_search+search_string_len}]'")
```

Starting from index 35: 'variable'

```
[9]: example_string.find('missing')
```

```
[9]: -1
```

1.3 Subplots with Matplotlib

Let us create some dummy images:

```
[10]: import numpy as np
import matplotlib.pyplot as plt

dummy_images = [[0,0,0,0,0],
                 [0,1,1,1,0],
                 [0,0,1,0,0],
                 [0,0,1,0,0],
                 [0,0,0,0,0]],

                [[0,0,0,0,0],
                 [0,0,1,0,0],
                 [0,1,1,1,0],
                 [0,0,1,0,0],
                 [0,0,0,0,0]],

                [[0,0,0,0,0],
                 [0,1,1,0,0],
                 [0,0,1,0,0],
                 [0,0,1,0,0],
                 [0,0,0,0,0]],

                [[0,0,0,0,0],
                 [0,1,1,1,0],
                 [0,0,0,0,0],
                 [0,1,1,1,0],
                 [0,0,0,0,0]]]
```

We usually use `plt.subplots` to create a figure with several sub-figures.

```
[11]: fig, axs = plt.subplots(1,4)

print("Shape of axs object:", axs.shape)

axs[0].imshow(dummy_images[0])
axs[0].set_title('Example 1')

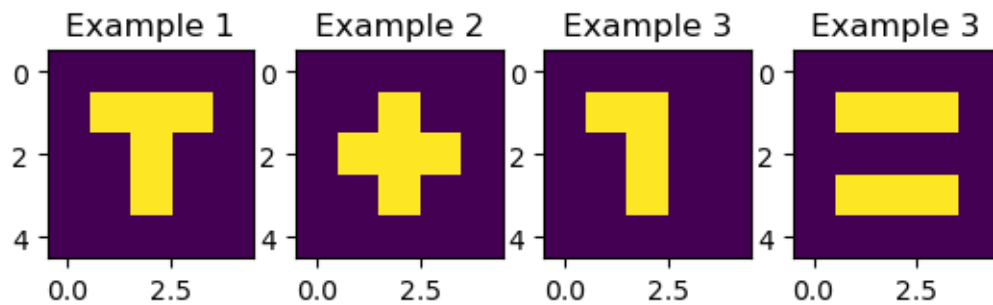
axs[1].imshow(dummy_images[1])
axs[1].set_title('Example 2')

axs[2].imshow(dummy_images[2])
axs[2].set_title('Example 3')

axs[3].imshow(dummy_images[3])
axs[3].set_title('Example 3')
```

Shape of axs object: (4,)

```
[11]: Text(0.5, 1.0, 'Example 3')
```



```
[17]: fig, axs = plt.subplots(2,2)

print("Shape of axs object:", axs.shape)

axs[0,0].imshow(dummy_images[0])
axs[0,0].set_title('Example 1')

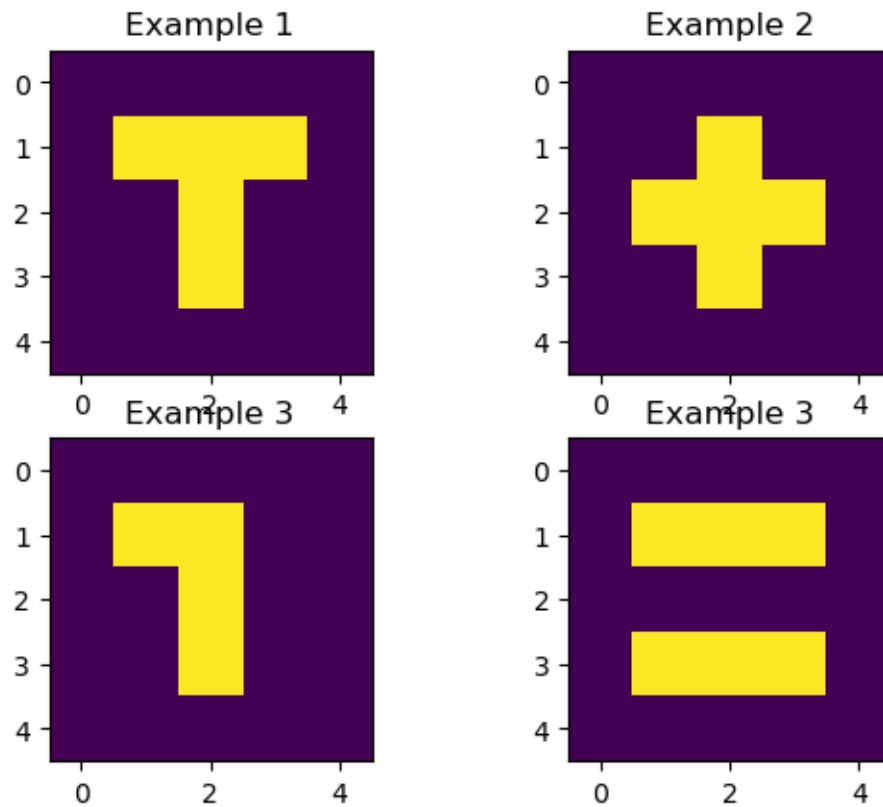
axs[0,1].imshow(dummy_images[1])
axs[0,1].set_title('Example 2')

axs[1,0].imshow(dummy_images[2])
axs[1,0].set_title('Example 3')

axs[1,1].imshow(dummy_images[3])
axs[1,1].set_title('Example 3')

plt.show()
```

Shape of axs object: (2, 2)



Another way to plot this is:

```
[21]: fig, axs = plt.subplots(2,2)

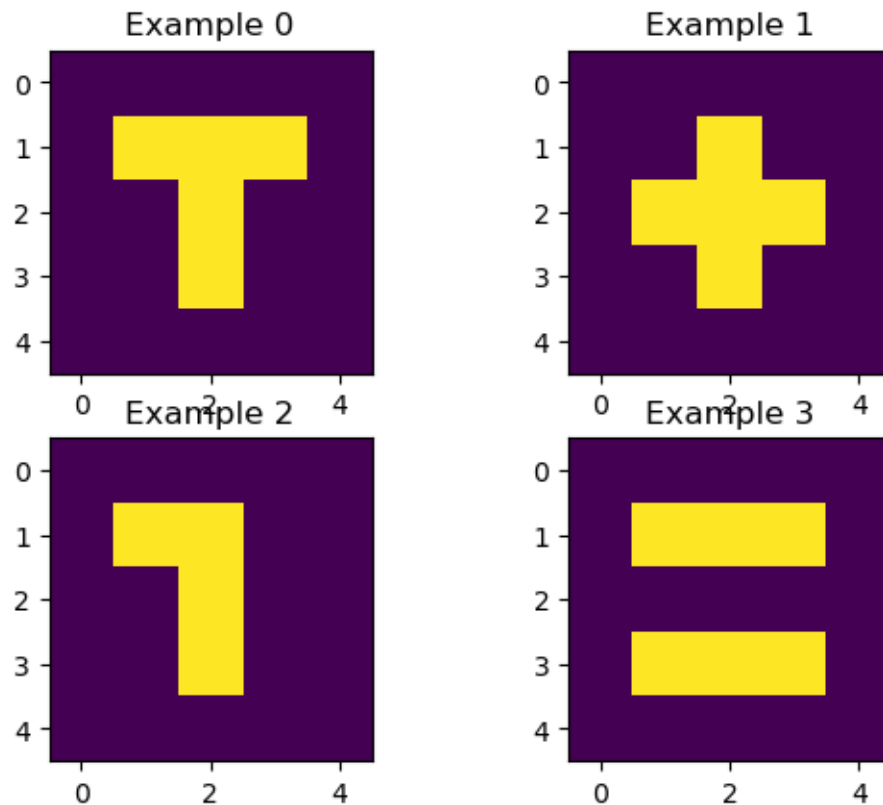
print("Shape of axs object:", axs.shape)
print("Shape of axs.ravel() object:", axs.ravel().shape)

for i, ax in enumerate(axs.ravel()):
    ax.imshow(dummy_images[i])
    ax.set_title(f"Example {i}")

plt.show()
```

Shape of axs object: (2, 2)

Shape of axs.ravel() object: (4,)



Note that you can iterate over subplots stored in object `axs`

In: `print(axs)`

Out: [`<Axes: >` `<Axes: >` `<Axes: >`]

with method `ravel` which creates a list of subplots.

There is a second way to create subplots which makes use of the `plt.figure.add_subplot` method. This is particularly useful when combining 3d and 2d sub-figures like in the following example.

```
[22]: data_3d = np.random.multivariate_normal(mean=[1,2,4], cov=np.eye(3), size=100)

fig = plt.figure(figsize=(10,2))

ax = fig.add_subplot(1,4,1, projection='3d')
ax.scatter(data_3d[:,0], data_3d[:,1], data_3d[:,2], marker='.')

ax.set_title('3D Example')

ax = fig.add_subplot(1,4,2)

ax.imshow(dummy_images[0])
```

```

ax.set_title("Example 1")

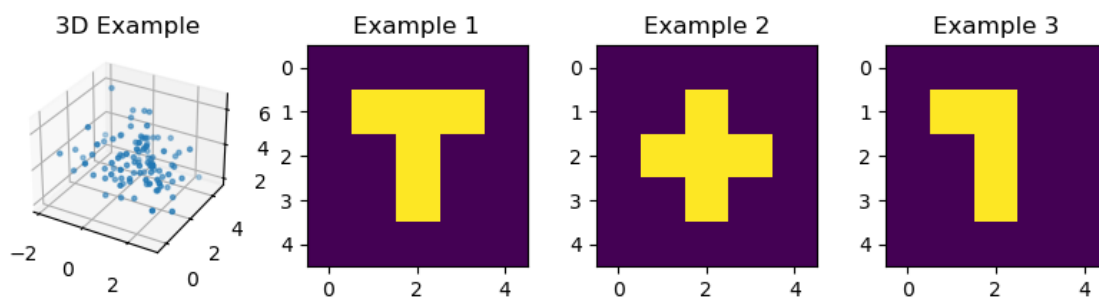
ax = fig.add_subplot(1,4,3)

ax.imshow(dummy_images[1])
ax.set_title("Example 2")

ax = fig.add_subplot(1,4,4)
ax.imshow(dummy_images[2])
ax.set_title("Example 3")

plt.show()

```



1.4 Type Annotations and Type Hints

You might have appreciated Python for its *dynamic typing*, i.e. that Python allows variables to change their data type during runtime. However, this kind of flexibility can make your code more prone to errors such that proficient coding in Python often resorts back to a flavour of *static typing* by using **type annotations**.

Consider the following example:

```

[24]: counter = 1

text = 'This is a test'

country_codes = {'Switzerland': 41, 'France': 33, 'Italy': 39,
                 'UK': 44, 'Germany': 49}

text = 100

```

The following might remind you of variable declarations you've seen in C, C++ or other statically typed programming languages:


```
[25]: counter: int = 1

text: str = 'This is a test'

country_codes: dict[str, int] = {'Switzerland': 41, 'France': 33, 'Italy': 39,
                                'UK': 44, 'Germany': 49}

text = 100
```

Advantages of type annotations include that typical IDEs support checking whether variables are assigned the expected type, and automated unit tests can be used more efficiently. In PyCharm, the above example would provide the following

This can be particularly useful when your code becomes increasingly abstract, e.g. with functions

```
[26]: def divide_10_by(some_numbers: list[float]) -> None:
        '''
        not very useful
        '''
        for num in some_numbers:
            print(f'10/{num} = {10/num}')

my_numbers: list[float] = [0.5, 20.0, 100.0]

divide_10_by(some_numbers = my_numbers)
```

```
10/0.5 = 20.0
10/20.0 = 0.5
10/100.0 = 0.1
```

or classes

```
[27]: class person:
        ...

alice: person = person()
```

1.5 Lambda Functions

Typically, we declare a function like this in Python:

```
[28]: def fahrenheit_to_celsius(deg_F):
        return (deg_F - 32) * 5/9
```

```
[29]: fahrenheit_to_celsius(98.6)
```

```
[29]: 37.0
```

For such simple **single expression functions** a lambda function might be a convenient choice.

```
[30]: fahrenheit_to_celsius_lambda = lambda deg_F: (deg_F - 32) * 5/9
```

```
[31]: fahrenheit_to_celsius_lambda(98.6)
```

```
[31]: 37.0
```

They are sometimes referred to as **anonymous functions**, as they are not required to be bound to a name, e.g.

```
[32]: (lambda deg_F: (deg_F - 32) * 5/9)(98.6)
```

```
[32]: 37.0
```

Of course, that's the same as doing

```
[33]: (98.6 - 32) * 5/9
```

```
[33]: 37.0
```

We will see a more interesting example below!

1.6 Generators

Generator functions are special functions particularly suited for generating sequences of various kind.

```
[34]: def squared_sequence(limit):
      num = 0

      while num < limit:
          yield num**2
          num += 1
```

where `num += 1` is shorthand for `num = num + 1`.

```
[35]: squared_sequence(10)
```

```
[35]: <generator object squared_sequence at 0x175d136b0>
```

```
[36]: gen = squared_sequence(10)
```

```
[47]: next(gen)
```

```
-----
StopIteration
```

```
Cell In[47], line 1
```

```
----> 1 next(gen)
```

```
Traceback (most recent call last)
```

StopIteration:

```
[48]: gen2 = squared_sequence(10)
```

```
[52]: next(gen2)
```

```
[52]: 9
```

```
[53]: for i in squared_sequence(10):
      print(i)
```

```
0
1
4
9
16
25
36
49
64
81
```

Note that the above is equivalent to

```
for i in range(10):
    print(i**2)
```

Note the differences to standard functions:

- Generators *yield* values, suspend the function and maintain the local state
- The values are generated when they are required (called with `next()`), i.e. we do not store a whole list but generate one element at a time
- Once all elements are generated, the iteration stops

However, you can still have all elements in a list if need be:

```
[54]: list( squared_sequence(10) )
```

```
[54]: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

This is similar to the **list comprehension**

```
[57]: [ num**2 for num in range(10) ]
```

```
[57]: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
[58]: empty_list = []
```

```

for num in range(10):
    res = num**2
    empty_list.append(res)

print(empty_list)

```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

1.7 Map and Filter

The built-in functions `map` and `filter` are convenient ways to apply a function to all elements of an iterable and avoid writing a loop for that.

With `map` we apply the function to each element of the iterable(s) and return the results. We use it in the following way:

```
map( function, iterable(s) )
```

Let's see some examples.

```

[59]: from matplotlib.colors import to_rgb

colours = ['green', 'red', 'blue', 'yellow']

for c in colours:
    print(f"{c} in RGB is: {to_rgb(c)}")

```

```

green in RGB is: (0.0, 0.5019607843137255, 0.0)
red in RGB is: (1.0, 0.0, 0.0)
blue in RGB is: (0.0, 0.0, 1.0)
yellow in RGB is: (1.0, 1.0, 0.0)

```

```

[60]: rgb_colours = map( to_rgb, colours )

print(f"Try to print rgb_colours: {rgb_colours}\n")

print( list(rgb_colours) )

```

```
Try to print rgb_colours: <map object at 0x175d23ac0>
```

```
[(0.0, 0.5019607843137255, 0.0), (1.0, 0.0, 0.0), (0.0, 0.0, 1.0), (1.0, 1.0, 0.0)]
```

```

[63]: decimals = [3.14159, 2.71828, 1.61803]

rounded = list(
    map( round, decimals, range(1,4) )
)

```

```
print(rounded)
```

```
[3.1, 2.72, 1.618]
```

```
[71]: decimals = [3.14159, 2.71828, 1.61803]
decimal_position = list(range(1,4))

empty_list = []

print(f"{decimals = }")
print(f"{decimal_position = }")
print()

for d, i in zip(decimals, decimal_position):
    empty_list.append( round(d,i) )

print(empty_list)
```

```
decimals = [3.14159, 2.71828, 1.61803]
decimal_position = [1, 2, 3]
```

```
[3.1, 2.72, 1.618]
```

With `filter` we apply a boolean function to an iterable and provide only the elements which returned `True`. We use it like this:

```
filter( function, iterable )
```

where `function` returns boolean values `True` or `False`.

```
[72]: def passed(grade):
      return grade > 4.0
```

```
[73]: grades = [5.5, 6.0, 2.0, 3.5, 4.5]

list( filter(passed, grades) )
```

```
[73]: [5.5, 6.0, 4.5]
```

A typical use case of lambda functions is actually in connection with `map` or `filter`.

```
[74]: list( filter( lambda grade: grade > 4.0, grades ) )
```

```
[74]: [5.5, 6.0, 4.5]
```

```
[75]: list( map( lambda grade: grade > 4.0, grades ) )
```

```
[75]: [True, True, False, False, True]
```

1.8 *args and **kwargs

Sometimes you see these `*args` and `**kwargs` arguments in functions and classes. We use them to write functions with variable number of arguments of positional and keyword arguments!

Here's an example:

```
[100]: def print_arguments(first, *args, **kwargs):
```

```
    print(first)
```

```
    if args:
```

```
        print(args)
```

```
    if kwargs:
```

```
        print(kwargs)
```

```
[101]: print_arguments()
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[101], line 1
----> 1 print_arguments()

TypeError: print_arguments() missing 1 required positional argument: 'first'
```

```
[102]: print_arguments(1)
```

```
1
```

```
[103]: print_arguments(1, 2, 3)
```

```
1
(2, 3)
```

```
[104]: print_arguments(1, 2, 3, 4, color='red', marker='x')
```

```
1
(2, 3, 4)
{'color': 'red', 'marker': 'x'}
```

Note that with `*args` we collect additional positional arguments in a tuple and with `**kwargs` the additional keyword arguments in a dictionary.

1.9 Decorators

Decorators are a great way to modify the behaviour of a function or class without changing the function or class itself.

```
[105]: def fahrenheit_to_celsius(deg_F):
        deg_C = (deg_F - 32) * 5/9
        return deg_C
```

```
[106]: fahrenheit_to_celsius(80)
```

```
[106]: 26.666666666666668
```

Let's define a decorator and "decorate" the previous function!

```
[107]: def just_int(func):

        def wrapper(*args, **kwargs):

            res = func(*args, **kwargs)

            print(int(res))

            return res

        return wrapper
```

```
[108]: @just_int
        def fahrenheit_to_celsius(deg_F):
            deg_C = (deg_F - 32) * 5/9
            return deg_C
```

```
[109]: fahrenheit_to_celsius(deg_F=80)
```

```
26
```

```
[109]: 26.666666666666668
```

```
[110]: def fahrenheit_to_celsius(deg_F):
        deg_C = (deg_F - 32) * 5/9
        return deg_C

        just_int(fahrenheit_to_celsius)(80)
```

```
26
```

```
[110]: 26.666666666666668
```

```
[111]: def just_int(print_out=None):

        def just_int_inner(func):

            def wrapper(*args, **kwargs):
```

```

        if print_out:
            print(print_out)

        res = func(*args,**kwargs)
        return int(res)

    return wrapper

return just_int_inner

```

```

[118]: @just_int(print_out='Get rid of decimals!')
def fahrenheit_to_celsius(deg_F):
    deg_C = (deg_F - 32) * 5/9
    return deg_C

```

```

[113]: # Alternatively

def fahrenheit_to_celsius(deg_F):
    deg_C = (deg_F - 32) * 5/9
    return deg_C

just_int(print_out='Get rid of decimals!')(fahrenheit_to_celsius)(80)

```

Get rid of decimals!

[113]: 26

```

[119]: fahrenheit_to_celsius(deg_F=80)

```

Get rid of decimals!

[119]: 26

1.10 Numba

Numba is a just-in-time compiler for Python which can speed up your code substantially if it is based on loops, NumPy arrays and NumPy functions. Its main feature is the `jit` decorator. Find more on Numba in the [official documentation](#).

Let's see what it does on a small example.

```

[120]: from numba import jit
import numpy as np

```

```

[123]: def calc_pi(total_num_points):
    np.random.seed(1234)

    num_circle_points = 0

```



```

for i in range(total_num_points):
    x = np.random.random()
    y = np.random.random()
    radius_squared = np.power(x,2) + np.power(y,2)

    if radius_squared < 1:
        num_circle_points += 1

pi_estimate = 4 * num_circle_points / total_num_points
return pi_estimate

```

```

[124]: @jit
def calc_pi_faster(total_num_points):
    np.random.seed(1234)

    num_circle_points = 0

    for i in range(total_num_points):
        x = np.random.random()
        y = np.random.random()
        radius_squared = np.power(x,2) + np.power(y,2)

        if radius_squared < 1:
            num_circle_points += 1

    pi_estimate = 4 * num_circle_points / total_num_points
    return pi_estimate

```

The function `calc_pi_faster` is compiled to machine code when called for the first time.

```

[125]: total_num_points = 100000

print(f"calc_pi: {calc_pi(total_num_points)}")
print(f"calc_pi_faster: {calc_pi_faster(total_num_points)}")

```

```

calc_pi: 3.14856
calc_pi_faster: 3.14856

```

```

[126]: %timeit calc_pi(total_num_points)

```

```

277 ms ± 2.6 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

```

```

[127]: %timeit calc_pi_faster(total_num_points)

```

```

605 µs ± 2.49 µs per loop (mean ± std. dev. of 7 runs, 1,000 loops each)

```

Note that Jupyter (IPython) provides some built-in Python functionality through *magic commands* `%command`.

1.11 Exercise Section

Generators are very useful when loading datasets, for instance. Suppose your dataset is too large to load into memory (RAM) (e.g. several tens of GB). Typically, a possible approach is to load parts of the dataset during training as needed.

In this exercise, we implement a generator to load *batches* of images from the **MNIST dataset**. MNIST is a standard Deep Learning benchmark dataset comprising 70 000 images (usually 28 x 28 pixels) of hand-written digits with the corresponding label as the classification target.

You can load the dataset (reduced to 5'000 samples) in the next cell.

```
[1]: import numpy as np

mnist_data = np.load('data/mnist_data_5k.npy', mmap_mode='r', allow_pickle=True)
mnist_targets = np.load('data/mnist_labels_5k.npy', mmap_mode='r',
    ↪allow_pickle=True)
```

Note that `mmap_mode='r'` specifies that we do not load data into memory but instead a memory-mapped array is constructed. This means, the data is read from the disk whenever accessed. Note that

```
In[1]: mnist_data = np.load('data/mnist_data_5k.npy', mmap_mode='r')
In[2]: type(mnist_data)
Out[2]: numpy.memmap
```

provides a `memmap` object, while

```
In[1]: mnist_data = np.load('data/mnist_data_5k.npy')
In[2]: type(mnist_data)
Out[2]: numpy.ndarray
```

provides the standard NumPy array. These are two different types of objects. `memmap` lives still on the disk, while the NumPy array is loaded into RAM. Note that in a `memmap` many but not all NumPy operations will work.

(1.) Define a generator which allows passing the data, targets, and the size of the batch in the following cell. For e.g. `batch_size=10`, the generator shall slice both arrays into the first 10 samples and provide them with the `next` call. With every following call, the next 10 samples are provided. So you need to think about how to access a slice of data in NumPy arrays and how consecutively get the next slice with the next call by adjusting the indices.

```
[2]: def batch_data(data, targets, batch_size):
    counter = 0
    length = data.shape[0]

    while counter < length:
        yield data[counter: counter + batch_size], targets[counter: counter +
    ↪batch_size]
        counter += batch_size
```

If you were successful, you can create the generator with

```
[3]: mnist_gen = batch_data(data=mnist_data, targets=mnist_targets, batch_size=10)
```

and retrieve a (new) batch of data with

```
[6]: new_data, new_targets = next(mnist_gen)

print(f"Target labels in this batch: {new_targets}")
```

Target labels in this batch: [4 0 9 1 1 2 4 3 2 7]

1.12 Proposed Solutions

Generators are very useful when loading datasets, for instance. Suppose your dataset is too large to load into memory (RAM) (e.g. several tens of GB). Typically, a possible approach is to load parts of the dataset during training as needed.

In this exercise, we implement a generator to load *batches* of images from the **MNIST dataset**. MNIST is a standard Deep Learning benchmark dataset comprising 70 000 images (usually 28 x 28 pixels) of hand-written digits with the corresponding label as the classification target.

You can load the dataset (reduced to 5 000 samples) in the next cell.

```
[1]: import numpy as np

mnist_data = np.load('data/mnist_data_5k.npy', mmap_mode='r', allow_pickle=True)
mnist_targets = np.load('data/mnist_labels_5k.npy', mmap_mode='r',
    ↪allow_pickle=True)
```

Note that `mmap_mode='r'` specifies that we do not load data into memory but instead a memory-mapped array is constructed. This means, the data is read from the disk whenever accessed. Note that

```
In[1]: mnist_data = np.load('data/mnist_data_5k.npy', mmap_mode='r')
In[2]: type(mnist_data)
Out[2]: numpy.memmap
```

provides a `memmap` object, while

```
In[1]: mnist_data = np.load('data/mnist_data_5k.npy')
In[2]: type(mnist_data)
Out[2]: numpy.ndarray
```

provides the standard NumPy array. These are two different types of objects. `memmap` lives still on the disk, while the NumPy array is loaded into RAM. Note that in a `memmap` many but not all NumPy operations will work.

(1.) Define a generator which allows passing the data, targets, and the size of the batch in the following cell. For e.g. `batch_size=10`, the generator shall slice both arrays into the first 10 samples and provide them with the `next` call. With every following call, the next 10 samples are provided.

So you need to think about how to access a slice of data in NumPy arrays and how consecutively get the next slice with the next call by adjusting the indices.

```
[2]: def batch_data(data, targets, batch_size):
    batch_num = 0
    tot_num_batches = int(data.shape[0] / batch_size)

    while batch_num < tot_num_batches:
        data_batch = data[batch_num*batch_size:(batch_num+1)*batch_size]
        targets_batch = targets[batch_num*batch_size:(batch_num+1)*batch_size]
        yield [np.array(data_batch), np.array(targets_batch)]
        batch_num = batch_num + 1
```

If you were successful, you can create the generator with

```
[3]: mnist_gen = batch_data(data=mnist_data, targets=mnist_targets, batch_size=10)
```

and retrieve a (new) batch of data with

```
[4]: new_data, new_targets = next(mnist_gen)

print(f"Target labels in this batch: {new_targets}")
```

Target labels in this batch: [5 0 4 1 9 2 1 3 1 4]

Class_3-Regression_and_Classification

May 20, 2026

1 3. Regression and Classification

Regression and classification are two fundamental tasks of supervised Machine Learning where labels allows us to guide the learning. This notebook reviews more standard approaches to regression and classification.

We provide the fundamental ideas behind

- linear regression and ridge regression
- logistic regression for classification
- decision trees and Random Forests

and apply these techniques to simulated data and a dataset example (wine quality assessment).

Keywords: OLS, MSE, Overfitting, Regularisation, `np.linalg.inv`, `sklearn.linear_model.LinearRegression`, `sklearn.linear_model.Ridge`, `sklearn.tree.DecisionTreeClassifier`, `sklearn.linear_model.LogisticRegression`, `sklearn.ensemble.RandomForestClassifier`, `sklearn.model_selection.train_test_split`

We start by fixing a random seed which controls the generation of (pseudo) random number sequences.

```
[1]: import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)
```

Note that this enables reproducibility of our results even in the presence of “randomness”. For example, the first three runs of the following cell

```
[4]: np.random.random(size=4)
```

```
[4]: array([0.4809319 , 0.39211752, 0.34317802, 0.72904971])
```

will always produce

1. `array([0.69646919, 0.28613933, 0.22685145, 0.55131477])`
2. `array([0.71946897, 0.42310646, 0.9807642 , 0.68482974])`

```
3. array([0.4809319 , 0.39211752, 0.34317802, 0.72904971])
```

1.1 Standard Linear Regression and Ordinary Least Squares

We assume a linear relationship between true **response** $Y_{\text{groundtruth}} = Xw$ and **predictors / co-variates** X . However, usually **our observation is not perfect**, i.e. there is some noise additional ϵ .

A simple linear regression model is then

$$Y = Xw + \epsilon. \quad (1)$$

Let us simulate $N = 10$ datapoints with Gaussian noise $\epsilon \sim \mathcal{N}(\mu = 0, \sigma = 20)$.

```
[8]: np.random.seed(123)

num_datapoints = 10

X = np.random.random(size=num_datapoints)*50
X = np.sort(X).reshape(-1,1)

print(X.shape)
print("X =", X)

w_groundtruth = 10
Y_groundtruth = w_groundtruth*X

epsilon_noise = np.random.normal(loc=0, scale=20.0, size=(num_datapoints,1))

Y = Y_groundtruth + epsilon_noise

print("Y =", Y)
```

```
(10, 1)
X = [[11.34257268]
      [14.30696675]
      [19.60587591]
      [21.15532301]
      [24.04659507]
      [27.56573845]
      [34.24148693]
      [34.82345928]
      [35.97344849]
      [49.03820992]]
Y = [[138.74445196]
      [125.73485943]
      [182.48103606]
```

```
[209.65905068]
[270.29374326]
[262.87934461]
[333.5352301 ]
[339.54756729]
[403.85308655]
[534.11782097]]
```

A typical objective function is given by the **mean squared error** (MSE) loss

$$\text{Loss}(w) = \frac{1}{2N} \|\epsilon\|_2^2 = \frac{1}{2N} \|Y - Xw\|_2^2 = \frac{1}{2N} \sum_{n=1}^N (y_n - x_n \cdot w)^2. \quad (2)$$

Under certain conditions, the solution to this equation will provide the **best linear (unbiased) estimator** for w ! We identify the **minimum** through

$$\nabla \text{Loss}(w) = X^\top Y - X^\top X w \equiv 0, \quad (3)$$

which provides the **ordinary least squares** (OLS) solution

$$w_{\text{OLS}} = (X^\top X)^{-1} X^\top Y. \quad (4)$$

```
[9]: XT = np.transpose(X)
     XTX_inverse = np.linalg.inv(XT @ X)

     w_OLS = XTX_inverse @ XT @ Y

     print(f"The solution is given by w_OLS = {w_OLS.squeeze()}.")
```

The solution is given by $w_{\text{OLS}} = 10.375338325958648$.

Note

- Operator `@` provides the matrix multiplication of two numpy array (similar to `np.matmul`)
- Function `numpy.array.squeeze()` removes all dimensions of size 1 from a NumPy array, i.e.

```
In: print(w_OLS.shape)
Out: (1, 1)
```

```
In: print(w_OLS)
Out: [[10.04673211]]
```

```
In: print(w_OLS.squeeze())
Out: 10.046732109803889
```

Let's provide some predictions Y_{OLS} based on this solution.

```
[10]: Y_OLS = w_OLS * X
```

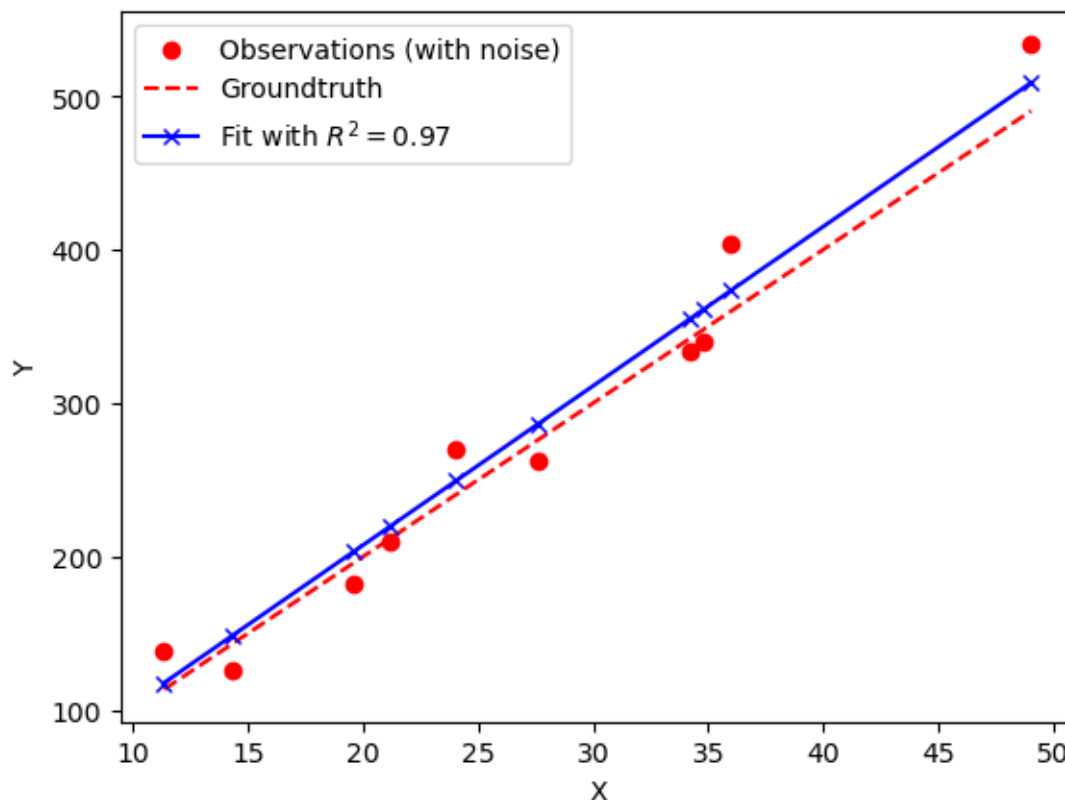
With this, we can calculate the **coefficient of determination** R^2 ([Wikipedia](#)) through

```
[11]: Rsquared = 1 - np.sum((Y - Y_OLS)**2) / np.sum((Y - np.mean(Y))**2)
      print(Rsquared)
```

0.965593882960831

```
[12]: plt.scatter(X, Y, color='red', label='Observations (with noise)')
      plt.plot(X, Y_groundtruth, color='red', linestyle='dashed',
               label=f'Groundtruth')
      plt.plot(X, Y_OLS, marker='x', color='blue', label=f'Fit with $R^2={Rsquared:.2f}$')

      plt.xlabel('X')
      plt.ylabel('Y')
      plt.legend()
      plt.show()
```



What happens if you have more “flexible models” at your disposal? In the following, we try to fit our observed responses Y with a polynomial model of the form

$$Y = X_{\text{poly}}w + \epsilon \quad (5)$$

$$= x \cdot w_1 + x^2 \cdot w_2 + x^3 \cdot w_3 + \dots + x^7 \cdot w_7 + x^8 \cdot w_8 + \epsilon \quad (6)$$

and create a hundred test datapoints.

```
[13]: X_poly = np.hstack((X, X ** 2, X ** 3, X ** 4, X ** 5, X ** 6, X ** 7, X ** 8))
print(X_poly.shape)

X_test = np.linspace(10,50,100).reshape(-1, 1)
X_test = np.hstack((X_test, X_test ** 2, X_test ** 3, X_test ** 4, X_test ** 5,
    ↪ X_test ** 6, X_test ** 7, X_test ** 8))
```

(10, 8)

Let's see the OLS solution for that case:

```
[14]: XT = np.transpose(X_poly)
XTX_inverse = np.linalg.inv(XT @ X_poly)

w_OLS = XTX_inverse @ XT @ Y

print(f"Solution w_OLS = {w_OLS.squeeze()}")

Y_OLS = X_poly @ w_OLS
Y_OLS_test = X_test @ w_OLS

Rsquared = 1 - np.sum((Y - Y_OLS)**2) / np.sum((Y - np.mean(Y))**2)

fig, axs = plt.subplots(1,2, figsize=(12,4))

axs[0].scatter(X, Y, marker='o', color='red', label='Observations (with noise)')
axs[0].plot(X, Y_groundtruth, color='red', linestyle='dashed',
    ↪ label=f'Groundtruth')
axs[0].scatter(X, Y_OLS, color='blue', marker='X', label=f'Train; fit with
    ↪ $R^2={Rsquared:.2f}$')

axs[1].scatter(X, Y, marker='o', color='red', label='Observations (with noise)')
axs[1].plot(X, Y_groundtruth, color='red', linestyle='dashed',
    ↪ label=f'Groundtruth')
axs[1].plot(X_test[:,0], Y_OLS_test, color='blue', label=f'Test')

axs[0].set_xlabel('X')
axs[0].set_ylabel('Y')
axs[0].legend()

axs[1].set_xlabel('X')
axs[1].set_ylabel('Y')
```

```

axs[1].legend()

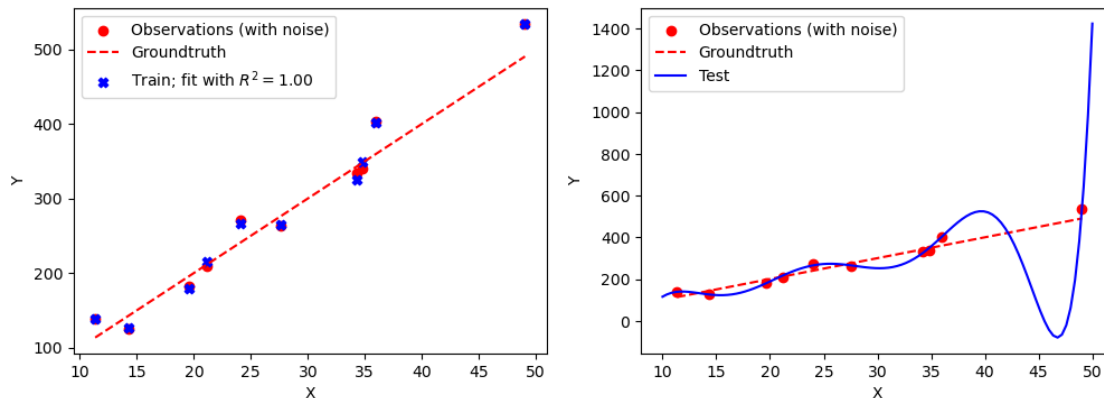
plt.show()

```

```

Solution w_OLS = [-7.51130753e+02  2.57966644e+02 -3.52218133e+01
2.52026936e+00
-1.02565199e-01  2.38549629e-03 -2.94929359e-05  1.50136544e-07]

```



1.2 Ridge Regression

Ridge regression adds an additional assumption on (the distribution of) weights w . We rewrite the loss as

$$\text{Loss}(w) = \frac{1}{2N} \|Y - Xw\|_2^2 + \frac{1}{2} \lambda' \|w\|_2^2 = \frac{1}{2N} \sum_{n=1}^N (y_n - x_n \cdot w)^2 + \frac{1}{2} \lambda' \sum_{i=1}^{d_x} w_i^2 \quad (7)$$

where the minimisation also takes into account the value (norm) of the weights w_i . Here, d_x indicates the dimensions of our input X , e.g. $d_x = 8$ in the case of the polynomial model above. The solution to this minimisation is given by

$$w_{\text{ridge}} = (\lambda \mathbf{1}_{d_x} + X^\top X)^{-1} X^\top Y. \quad (8)$$

Let's see what happens with this additional term:

```

[15]: lambda_ridge = 100000

XT = np.transpose(X_poly)
XTX_ridge_inverse = np.linalg.inv(XT @ X_poly + lambda_ridge * np.
    ↪identity(X_poly.shape[1]))

w_ridge = XTX_ridge_inverse @ XT @ Y

```

```
print(f"Solutions: \n w_OLS = \t {w_OLS.squeeze()},\n w_ridge = \t {w_ridge.
    ↪squeeze()}.")
```

Solutions:

```
w_OLS =          [-7.51130753e+02  2.57966644e+02 -3.52218133e+01
2.52026936e+00
-1.02565199e-01  2.38549629e-03 -2.94929359e-05  1.50136544e-07],
w_ridge =        [ 2.47873294e-03  1.58957362e-02  6.22240854e-02
4.74477828e-03
-7.57974781e-04  3.01976405e-05 -4.90619858e-07  2.84986667e-09].
```

```
[16]: Y_ridge = X_poly @ w_ridge
Y_ridge_test = X_test @ w_ridge

Rsquared_ridge = 1 - np.sum((Y - Y_ridge)**2) / np.sum((Y - np.mean(Y))**2)

print(Rsquared_ridge)

fig, axs = plt.subplots(1,2, figsize=(12,4))

axs[0].scatter(X, Y, marker='o', color='red', label='Observations (with noise)')
axs[0].plot(X, Y_groundtruth, color='red', linestyle='dashed',
    ↪label='Groundtruth')
axs[0].scatter(X, Y_OLS, color='blue', marker='X', label=f'Train; OLS with
    ↪$R^2={Rsquared:.2f}$')
axs[0].scatter(X, Y_ridge, color='green', marker='+', label=f'Train; ridge with
    ↪$R^2={Rsquared_ridge:.2f}$')

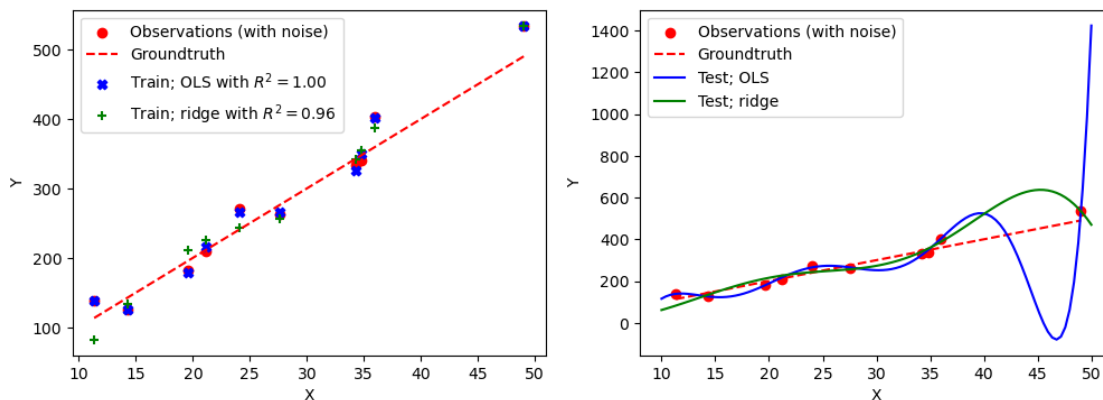
axs[1].scatter(X, Y, marker='o', color='red', label='Observations (with noise)')
axs[1].plot(X, Y_groundtruth, color='red', linestyle='dashed',
    ↪label='Groundtruth')
axs[1].plot(X_test[:,0], Y_OLS_test, color='blue', label=f'Test; OLS')
axs[1].plot(X_test[:,0], Y_ridge_test, color='green', label=f'Test; ridge')

axs[0].set_xlabel('X')
axs[0].set_ylabel('Y')
axs[0].legend()

axs[1].set_xlabel('X')
axs[1].set_ylabel('Y')
axs[1].legend()

plt.show()
```

0.9604147033799222



Note that the additional term $\lambda \|w\|_2^2 = \lambda \sum_{i=1}^{d_x=8} w_i^2$ in the loss function enables deviation from the observed noisy data points. That is models (i.e. weights w_i) which too closely *overfit* to the noisy observations are penalised. Parameter λ serves as a **penalty factor** biasing towards *simpler* models which **reduce overfitting** ([Wikipedia](#)).

Generally, having terms in your loss function favouring simpler models is referred to as **regularisation** ([Wikipedia](#)). Highly-flexible models (like the blue curve in the next figure) allow fitting observations (red dots) arbitrary well. Typically, we prefer simpler models (like the green curve) which usually provide better predictions to new, unseen data (**generalisation**).

Image source: Wikipedia

1.3 Using scikit-learn for Regression

You don't need to do all the work yourself! **scikit-learn** offers you the required functionality with the functions [LinearRegression](#) and [Ridge](#)!

```
[17]: from sklearn.linear_model import LinearRegression, Ridge
```

```
linreg = LinearRegression(fit_intercept=False)
linreg.fit(X,Y)
```

```
[17]: LinearRegression(fit_intercept=False)
```

Note that we used `fit_intercept=False` because we chose to neglect on off-set / intercept b on the y-axis. That is, we consider $Y = Xw + b + \epsilon$ with $b = 0$, which simplifies writing the equations out.

`linreg` is the linear fit which provides you the same OLS solution as seen above.

```
[18]: w_OLS_sk = linreg.coef_
      Y_OLS_sk = linreg.predict(X)
      Rsquared_sk = linreg.score(X,Y)
```

```
print(f"w_OLS_sk = {w_OLS_sk.squeeze()}, Rsquared_sk = {Rsquared_sk}")
```

```
w_OLS_sk = 10.375338325958642, Rsquared_sk = 0.965593882960831
```

We can do the same for the ridge regression case.

```
[19]: ridgereg = Ridge(alpha=10, fit_intercept=False)
      ridgereg.fit(X,Y)

      w_ridge_sk = ridgereg.coef_
      Y_ridge_sk = ridgereg.predict(X)
      Rsquared_ridge_sk = ridgereg.score(X,Y)

      print(f"w_ridge_sk = {w_ridge_sk.squeeze()}, Rsquared_ridge_sk = {Rsquared_sk}")
```

```
w_ridge_sk = 10.363270299382133, Rsquared_ridge_sk = 0.965593882960831
```

1.4 Logistic Regression for Classification

Next, we consider binary classification where our data falls into one of two classes which we will denote with class labels $Y = 0$ (class 1) and $Y = 1$ (class 2). Let's assume that observations of class 2 occur with an unknown probability of p_2 . As we only have two classes, the probability for class 1 is $p_1 = 1 - p_2$. A natural choice in such a setting is the **Bernoulli distribution** ([Wikipedia](#)) with probability (mass) function

$$p(Y) = p_2^Y p_1^{1-Y} = p_2^Y (1 - p_2)^{1-Y}. \quad (9)$$

The idea of logistic regression is to approximate the ratio of the two class probabilities with an exponential function of the form

$$\frac{p_2}{p_1} = \exp(x_1 \cdot w_1 + x_2 \cdot w_2 + \dots + x_{d_x} \cdot w_{d_x}) = \exp(Xw), \quad (10)$$

so again a linear model! Note that we can write this as

$$\exp(Xw) = \frac{p_2}{1 - p_2} \Rightarrow p_2 = \frac{1}{1 + \exp(-Xw)}. \quad (11)$$

This is the **logistic function**. In order to identify the weights w , i.e. a matching model, we use the **logistic loss** (also known as **negative log-likelihood** or **cross entropy loss**, depending on the context)

$$\text{Loss}(w) = -Y \log p_2 - (1-Y) \log(1-p_2) = - \sum_{n=1}^N y_n \log \frac{1}{1 + \exp(-x_n \cdot w)} + (1-y_n) \log \left(1 - \frac{1}{1 + \exp(-x_n \cdot w)} \right) \quad (12)$$

for N datapoints.

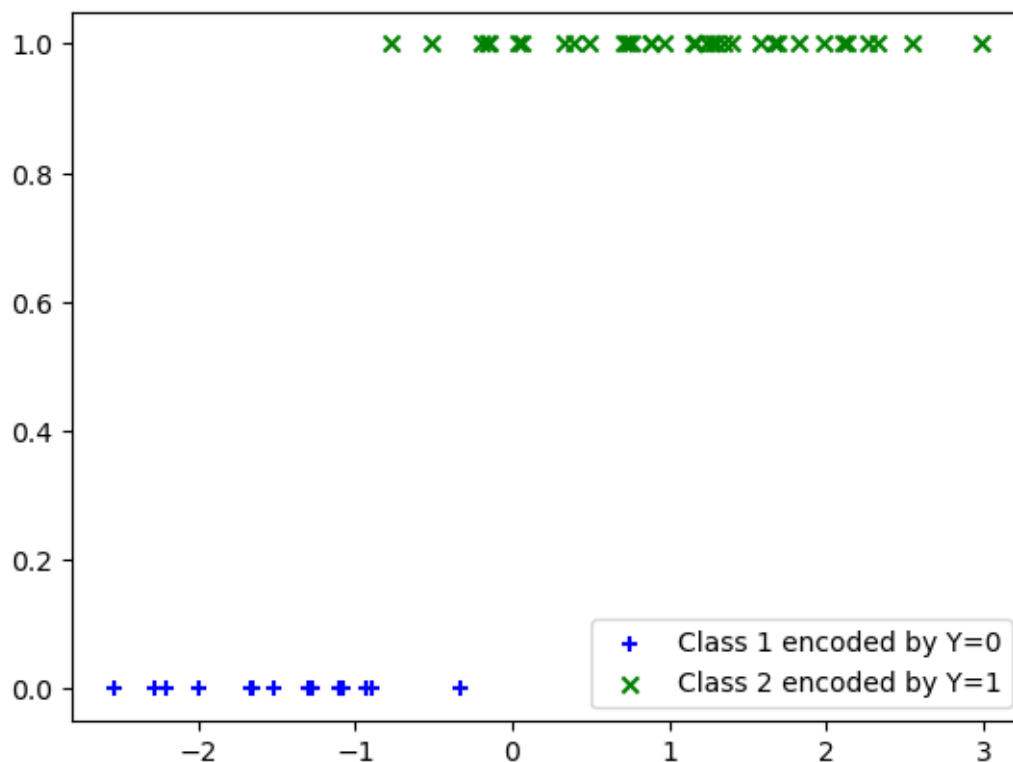
Let's create a simple binary classification problem with 50 datapoints.

```
[20]: from sklearn.datasets import make_classification

num_datapoints = 50

X, Y = make_classification(n_samples=num_datapoints,
                           n_features=1,
                           n_informative=1,
                           n_redundant=0,
                           n_classes=2,
                           n_clusters_per_class=1,
                           weights=[0.3, 0.7],
                           random_state=0)

plt.scatter(X[Y == 0], Y[Y == 0], color='blue', marker='+', label='Class 1_␣
↳encoded by Y=0')
plt.scatter(X[Y == 1], Y[Y == 1], color='green', marker='x', label='Class 2_␣
↳encoded by Y=1')
plt.legend()
plt.show()
```



Here we directly use `scikit-learn` function `LogisticRegression`.

```
[22]: from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression()
logreg.fit(X, Y)
```

```
[22]: LogisticRegression()
```

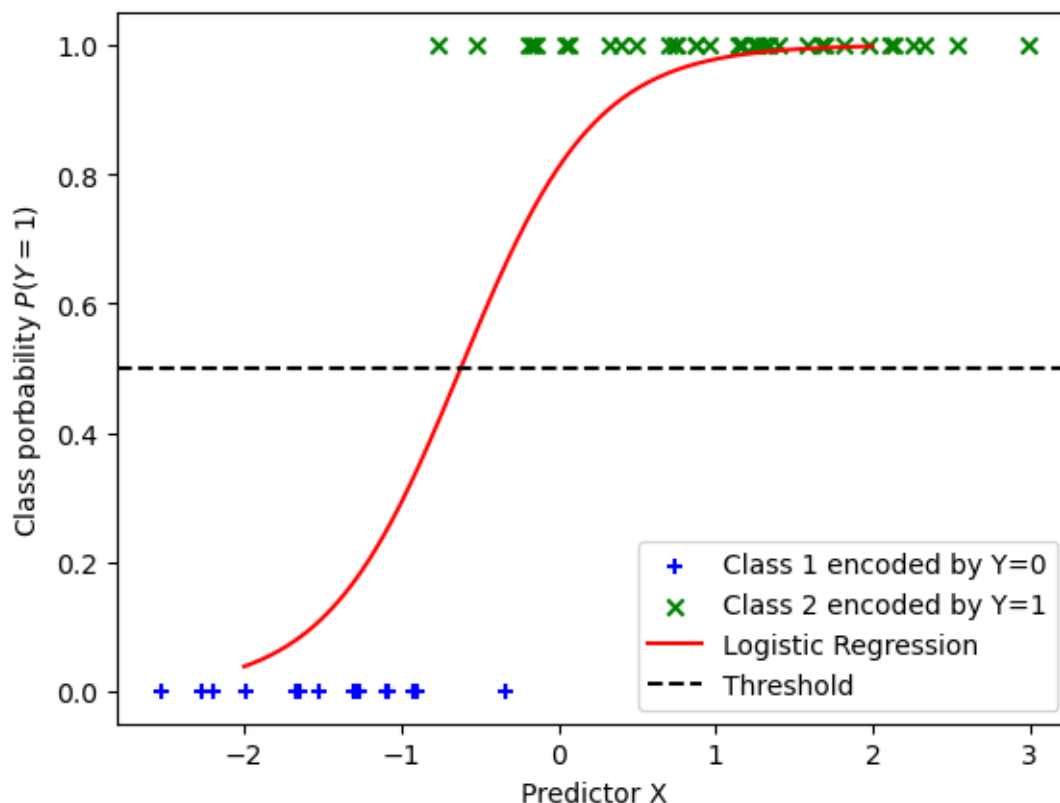
Using the method `predict_proba` allows predicting the probability $p(Y = 1)$ for class 2 of our logistic regression model `logreg`.

```
[23]: X_logreg = np.linspace(-2, 2, 1000).reshape(-1, 1)
Y_logreg = logreg.predict_proba(X_logreg)
```

Let's plot the result:

```
[24]: plt.scatter(X[Y == 0], Y[Y == 0], color='blue', marker='+', label='Class 1_
↳encoded by Y=0')
plt.scatter(X[Y == 1], Y[Y == 1], color='green', marker='x', label='Class 2_
↳encoded by Y=1')
plt.plot(X_logreg[:, 0], Y_logreg[:, 1], color='red', label='Logistic_
↳Regression')
plt.axhline(0.5, color='black', linestyle='--', label='Threshold')

plt.xlabel('Predictor X')
plt.ylabel(r'Class porbability $P(Y=1)$')
plt.legend()
plt.show()
```



Overfitting example in classification tasks

Image source: Medium

1.5 Decision Trees and Random Forests

An more classical approach to classification problems are decision trees. In the next example, we will try to classify premium red wine from their measured properties / features which serve as the predictors / covariates.

```
[42]: import pandas as pd
import matplotlib.pyplot as plt

wine_data = pd.read_csv('data/winequality-red.csv', sep=';')

# quality_threshold = 6
quality_threshold = 5

wine_data['premium'] = wine_data['quality'] > quality_threshold
```



```
wine_data
```

```
[42]:      fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0           7.4           0.700           0.00           1.9           0.076
1           7.8           0.880           0.00           2.6           0.098
2           7.8           0.760           0.04           2.3           0.092
3          11.2           0.280           0.56           1.9           0.075
4           7.4           0.700           0.00           1.9           0.076
...
1594         6.2           0.600           0.08           2.0           0.090
1595         5.9           0.550           0.10           2.2           0.062
1596         6.3           0.510           0.13           2.3           0.076
1597         5.9           0.645           0.12           2.0           0.075
1598         6.0           0.310           0.47           3.6           0.067

      free sulfur dioxide  total sulfur dioxide  density  pH  sulphates  \
0           11.0           34.0  0.99780  3.51           0.56
1           25.0           67.0  0.99680  3.20           0.68
2           15.0           54.0  0.99700  3.26           0.65
3           17.0           60.0  0.99800  3.16           0.58
4           11.0           34.0  0.99780  3.51           0.56
...
1594           32.0           44.0  0.99490  3.45           0.58
1595           39.0           51.0  0.99512  3.52           0.76
1596           29.0           40.0  0.99574  3.42           0.75
1597           32.0           44.0  0.99547  3.57           0.71
1598           18.0           42.0  0.99549  3.39           0.66

      alcohol  quality  premium
0           9.4         5    False
1           9.8         5    False
2           9.8         5    False
3           9.8         6     True
4           9.4         5    False
...
1594        10.5         5    False
1595        11.2         6     True
1596        11.0         6     True
1597        10.2         5    False
1598        11.0         6     True
```

```
[1599 rows x 13 columns]
```

We focus on the quality assessment.

```
[43]: quality_counts = wine_data['quality'].value_counts()

if quality_threshold == 5:
```

```

colours = ['red', 'blue', 'blue', 'red', 'blue', 'red']
elif quality_threshold == 6:
    colours = ['red', 'red', 'blue', 'red', 'blue', 'red']

print(quality_counts)

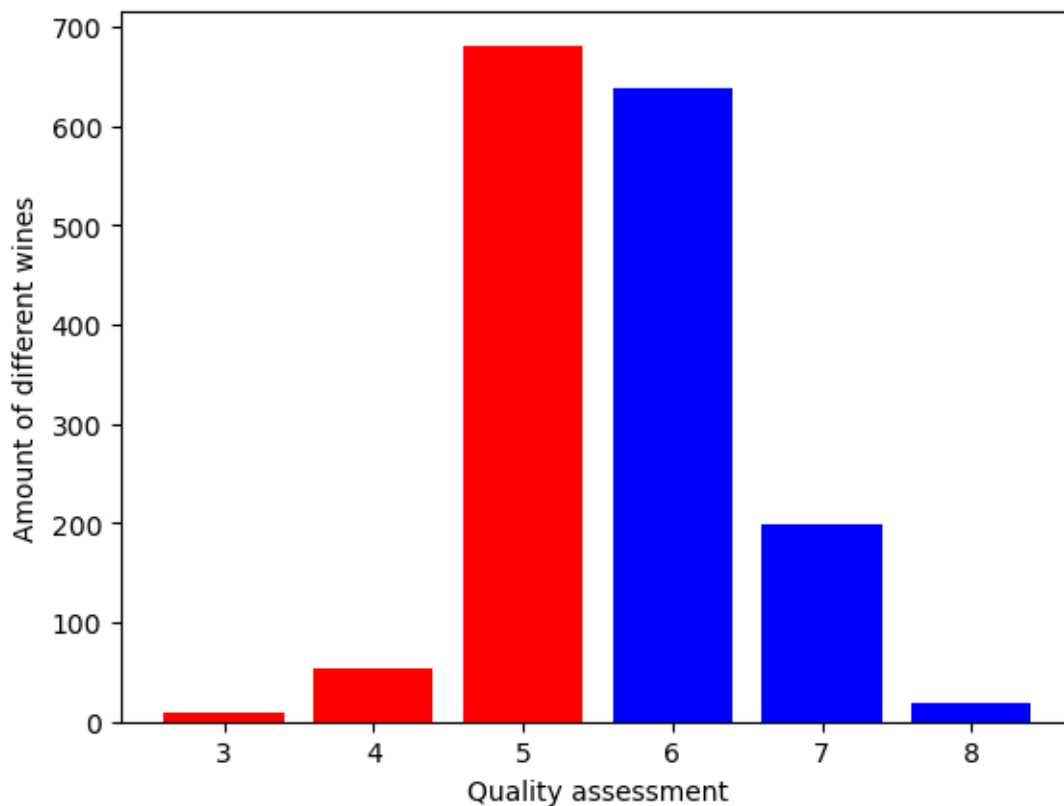
plt.bar(quality_counts.index, quality_counts, color=colours)
plt.xlabel('Quality assessment')
plt.ylabel('Amount of different wines')
plt.show()

```

```

quality
5    681
6    638
7    199
4     53
8     18
3     10
Name: count, dtype: int64

```



Perpare the features and target:

```
[44]: target = wine_data['premium'].astype(int)
      wine_features = wine_data.drop( ['quality','premium'], axis=1)

      print("Shape of wine_features:\t{}\nShape of target:\t{}\n"
            .format(wine_features.shape, target.shape)
            )

      target.head(5)
```

```
Shape of wine_features: (1599, 11)
Shape of target:       (1599,)
```

```
[44]: 0    0
      1    0
      2    0
      3    1
      4    0
      Name: premium, dtype: int64
```

The Goal is to learn a classifier which predicts whether a wine is a premium / non-premium wine on the basis of the measured wine features.

Select (randomly) a set on which the classifier is calibrated / trained on and a test set on which the performance is assessed. We consider a test set size of 30% of the original data set.

```
[45]: from sklearn.model_selection import train_test_split

      feat_train, feat_test, target_train, target_test = train_test_split(
          wine_features, target, test_size = 0.3, random_state=123)

      print("After splitting into train and test sets:\n\n"
            f"Shape of feat_train:\t{feat_train.shape}\nShape of target_train:
            ↪\t{target_train.shape}\n"
            f"Shape of feat_test:\t{feat_test.shape}\nShape of target_test:
            ↪\t{target_test.shape}"
            )
```

After splitting into train and test sets:

```
Shape of feat_train:    (1119, 11)
Shape of target_train: (1119,)
Shape of feat_test:    (480, 11)
Shape of target_test:  (480,)
```

We have a first look on a decision tree and just consider the features providing the alcohol and sulphates content. We use the [DecisionTreeClassifier](#).

```
[46]: from sklearn.tree import DecisionTreeClassifier, plot_tree

feat_train_subset = feat_train[ ['alcohol', 'sulphates'] ]

wine_tree = DecisionTreeClassifier(max_depth=2)
wine_tree.fit(feat_train_subset, target_train)
```

```
[46]: DecisionTreeClassifier(max_depth=2)
```

```
[47]: from sklearn.inspection import DecisionBoundaryDisplay

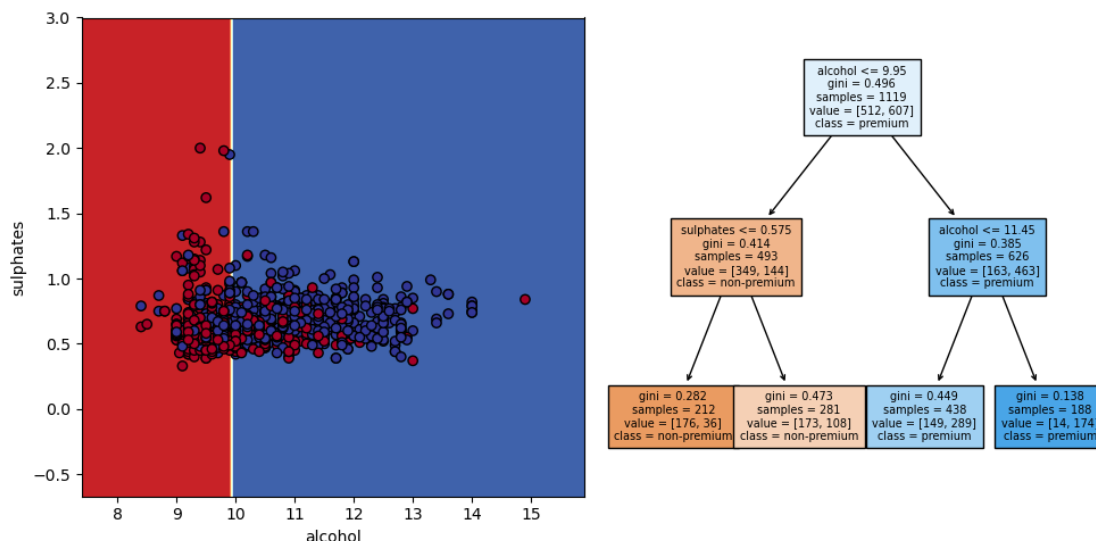
fig, axs = plt.subplots(1,2, figsize=(10,5))

display = DecisionBoundaryDisplay.from_estimator(
    wine_tree,
    feat_train_subset,
    cmap='RdYlBu',
    response_method="predict",
    ax=axs[0],
    xlabel='alcohol',
    ylabel='sulphates',
)

plot_tree(wine_tree, max_depth=2,
          feature_names=['alcohol', 'sulphates'],
          class_names=['non-premium', 'premium'],
          ax=axs[1], fontsize=7, filled = True,
          )

axs[0].scatter(feat_train['alcohol'], feat_train['sulphates'],
               c=target_train, cmap='RdYlBu', edgecolor='black'
               )

plt.tight_layout()
plt.show()
```



Note that decision trees partition the input space into different regions with **decision boundaries** separating different classes, like premium wines (blue data / region) and non-premium wines (red data / region)!

Now let's consider the full dataset!

```
[48]: tree_height = 8

wine_tree = DecisionTreeClassifier(max_depth=tree_height)
wine_tree.fit(feet_train, target_train)

correct_pred = wine_tree.predict(feet_test) == target_test

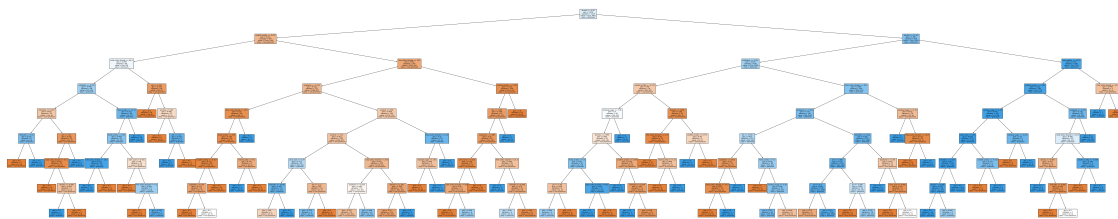
correct = correct_pred.value_counts()

accuracy = (correct[True] / (correct[True] + correct[False]))*100
print(f"The decision tree identified premium / non-premium wines with_
↳{accuracy}% accuracy!")

fig, ax = plt.subplots(1,1, figsize=(100,20))

plot_tree(wine_tree,
          feature_names=wine_features.columns.to_list(),
          class_names=['non-premium', 'premium'],
          filled = True,
          ax=ax,
          fontsize=8
        )
plt.show()
```

The decision tree identified premium / non-premium wines with 70.83333333333334% accuracy!



You can learn more on decision trees from [scikit-learn](#) and the [mathematical criteria](#) for the decision splits.

Random Forests are now very powerful machine learning methods which build on decision trees. Random Forests construct a multitude of decision trees like the one above. The individual decision trees provide a classification result and by majority voting the final classification is obtained. We use the [RandomForestClassifier](#).

```
[49]: from sklearn.ensemble import RandomForestClassifier

num_trees = 100
tree_height = 8

wine_forest = RandomForestClassifier(n_estimators=num_trees,
                                    max_depth=tree_height,
                                    random_state=123)
wine_forest.fit(feet_train, target_train)
```

```
[49]: RandomForestClassifier(max_depth=8, random_state=123)
```

```
[50]: correct_pred = wine_forest.predict(feet_test) == target_test

correct = correct_pred.value_counts()

accuracy = (correct[True] / (correct[True] + correct[False]))*100
print(f"The random forest identified premium / non-premium wines with
      ↳{accuracy}% accuracy!")
```

The random forest identified premium / non-premium wines with 78.125% accuracy!

That's a great accuracy! But let's check what is actually predicted wrongly:

```
[51]: rel_incorrect_pred = target_test[correct_pred == False].value_counts() /
      ↳target_test.value_counts()
```

```

print(f"Incorrect predictions by premium quality:\n{rel_incorrect_pred}")
print(f"\nRatio of premium / non-premium wines in test set:\n{target_test.
↪value_counts(normalize=True)}")
print(f"\nRatio of premium / non-premium wines in train set:\n{target_train.
↪value_counts(normalize=True)}")

```

Incorrect predictions by premium quality:

premium

0 0.275862

1 0.165323

Name: count, dtype: float64

Ratio of premium / non-premium wines in test set:

premium

1 0.516667

0 0.483333

Name: proportion, dtype: float64

Ratio of premium / non-premium wines in train set:

premium

1 0.542449

0 0.457551

Name: proportion, dtype: float64

You can still plot one of the trees!

```

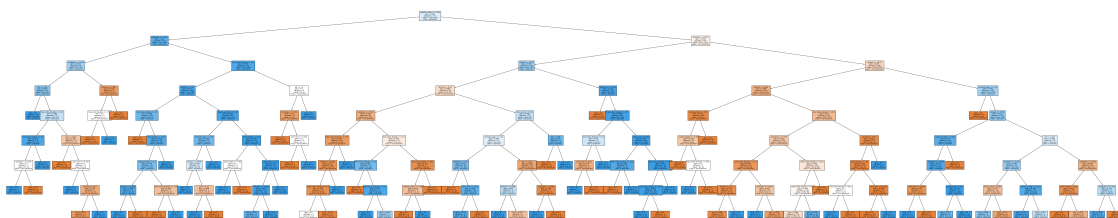
[54]: single_tree = wine_forest.estimators_[5]

fig, ax = plt.subplots(1,1, figsize=(100,20))

plot_tree(single_tree,
          feature_names=wine_features.columns.to_list(),
          class_names=['non-premium', 'premium'],
          filled = True,
          ax=ax,
          fontsize=8
        )

plt.show()

```

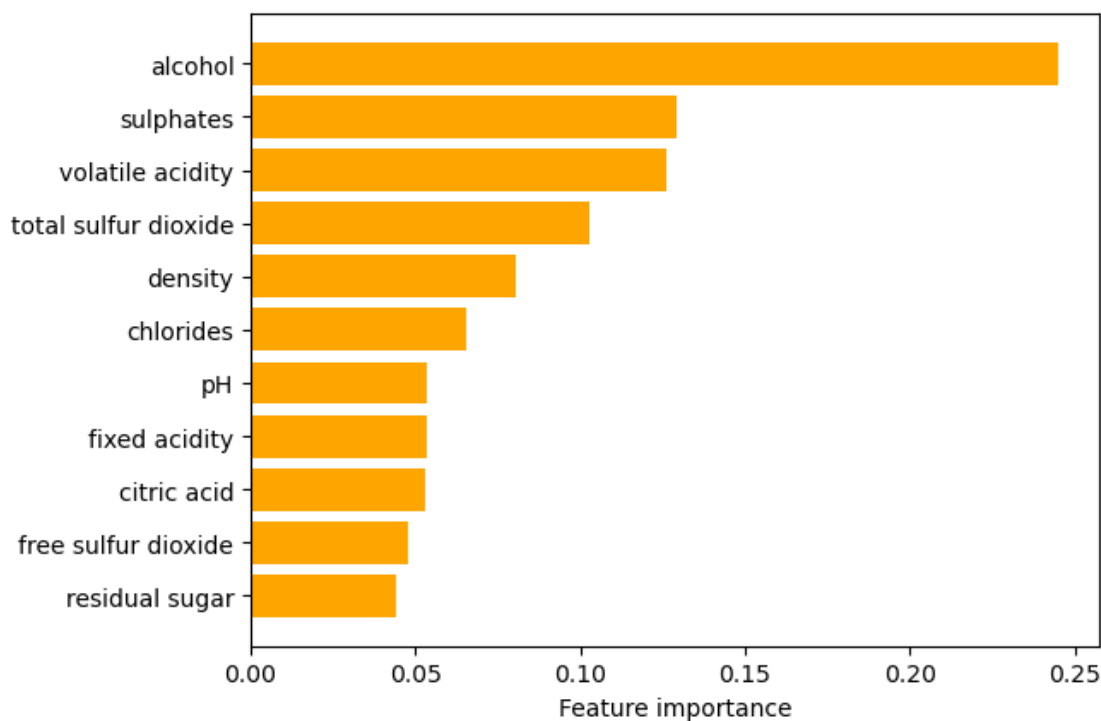


However, this is less insightful, as the decision is made by vote of many (different) trees! Still, we can study the **feature importance score**:

```
[55]: feature_scores = wine_forest.feature_importances_
feature_names = list(wine_features.columns)

important_features = pd.Series(feature_scores, index=feature_names).
    ↪ sort_values()

plt.barh(important_features.index, important_features, color='Orange')
plt.xlabel('Feature importance')
plt.show()
```



1.5.1 How to address imbalanced datasets?

As we have seen with Random Forests, looking only at the accuracy in imbalanced datasets can deceive. There are different approaches to rectify this problem; find the following link different [resampling strategies for imbalanced datasets](#).

1.5.2 How to tune hyperparameters?

A particular challenge with more advanced machine learning techniques is the choice of hyperparameters like `max_depth` / `tree_height` for decision trees and random forests. Typically, one attempts to define a reasonable scope for the range of different hyperparameters and identify more suitable values based on the **validation set** performance.

There exist tools which allow you an automated way to perform hyperparameter search. Check out for example this scikit-learn documentation on [Tuning the hyper-parameters of an estimator](#) and an [example for a random forest](#).

1.6 Exercise Section

(1.) In this exercise, we train a Ridge regressor for predicting the `quality` values on the test set `feat_test`. First, load the following cell:

```
[56]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

wine_data = pd.read_csv('data/winequality-red.csv', sep=';')

ex_target = wine_data['quality']
ex_features = wine_data.drop(['quality'], axis=1)

feat_train, feat_test, target_train, target_test = train_test_split(
    ex_features, ex_target, test_size = 0.3, random_state=123)
```

```
[57]: feat_train.head(5)
```

```
[57]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	\
374	14.0	0.410	0.63	3.8	0.089	
800	7.2	0.610	0.08	4.0	0.082	
1441	7.4	0.785	0.19	5.2	0.094	
1269	5.5	0.490	0.03	1.8	0.044	
691	9.2	0.920	0.24	2.6	0.087	

	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	\
374	6.0	47.0	1.00140	3.01	0.81	
800	26.0	108.0	0.99641	3.25	0.51	
1441	19.0	98.0	0.99713	3.16	0.52	
1269	28.0	87.0	0.99080	3.50	0.82	
691	12.0	93.0	0.99980	3.48	0.54	


```
alcohol
```

```

374    10.800000
800     9.400000
1441    9.566667
1269   14.000000
691     9.800000

```

```
[58]: target_train.head(5)
```

```

[58]: 374     6
      800     5
      1441    6
      1269    8
      691     5
      Name: quality, dtype: int64

```

Put your result in the next cell and use `ex_pred_ridge` for the predicted quality values.

```

[ ]: from sklearn.linear_model import Ridge

     # Fill in

     ex_pred_ridge = # Fill in

```

Execute the next cell to save the results in a summary data frame.

```

[ ]: target_test = target_test.to_frame()
     target_test['Ridge_predicted_quality'] = np.around(ex_pred_ridge, decimals=2)
     target_test['Ridge_absolut_deviation'] = abs(target_test['quality'] -
     ↪target_test['Ridge_predicted_quality'])
     target_test

```

(2.) You can create a Random Forest not only for classification, but also regression. Make use of the `scikit-learn` method

```
[ ]: from sklearn.ensemble import RandomForestRegressor
```

to make predictions on the wine quality based on the other features / predictors. Load the previous cell and train a the `RandomForestRegressor` for predicting the quality values on the test set `feat_test`. Put your result in the next cell and use `ex_pred_RF` for the predicted quality values.

Hint: You might want to revisit the steps for RF classification we saw above.

```

[ ]: num_trees = 100
     tree_height = 8

     # Fill in

     ex_pred_RF = # Fill in

```

```
[ ]: target_test['RF_predicted_quality'] = np.around(ex_pred_RF, decimals=2)
target_test['RF_absolut_deviation'] = abs(target_test['quality'] -
    ↪target_test['RF_predicted_quality'])
target_test
```

Finally, compare how the Random Forest and ridge regressor performed in comparison. For this, just execute the next cell.

```
[ ]: RF_pred_MSE = (target_test['RF_absolut_deviation']**2).mean()
Ridge_pred_MSE = (target_test['Ridge_absolut_deviation']**2).mean()

print(f"Mean squared error of RandomForestRegressor: {RF_pred_MSE:.2f}")
print(f"Mean squared error of Ridge: {Ridge_pred_MSE:.2f}")
```

1.7 Proposed Solutions

(1.) In this exercise, we train a Ridge regressor for predicting the quality values on the test set `feat_test`. First, load the following cell:

```
[32]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

wine_data = pd.read_csv('data/winequality-red.csv', sep=';')

ex_target = wine_data['quality']
ex_features = wine_data.drop(['quality'], axis=1)

feat_train, feat_test, target_train, target_test = train_test_split(
    ex_features, ex_target, test_size = 0.3, random_state=123)
```

Put your result in the next cell and use `ex_pred_ridge` for the predicted quality values.

```
[33]: from sklearn.linear_model import Ridge

ridgereg = Ridge(alpha=1)
ridgereg.fit(feat_train, target_train)

ex_pred_ridge = ridgereg.predict(feat_test)

print(f"Rsquared_ridgereg: {ridgereg.score(feat_train, target_train)}")
```

Rsquared_ridgereg: 0.3627326339932849

Execute the next cell to save the results in a summary data frame.

```
[34]: target_test = target_test.to_frame()
target_test['Ridge_predicted_quality'] = np.around(ex_pred_ridge, decimals=2)
target_test['Ridge_absolut_deviation'] = abs(target_test['quality'] -
↳target_test['Ridge_predicted_quality'])
target_test
```

```
[34]:      quality  Ridge_predicted_quality  Ridge_absolut_deviation
912         6                6.32          0.32
772         5                4.97          0.03
1037        5                4.77          0.23
1106        6                6.57          0.57
263         5                5.51          0.51
...
1466        7                5.57          1.43
580         5                5.32          0.32
1082        6                5.40          0.60
1279        7                6.31          0.69
1155        5                5.32          0.32
```

[480 rows x 3 columns]

(2.) You can create a Random Forest not only for classification, but also regression. Make use of the `scikit-learn` method

```
[35]: from sklearn.ensemble import RandomForestRegressor
```

to make predictions on the wine quality based on the other features / predictors. Load the previous cell and train a the `RandomForestRegressor` for predicting the quality values on the test set `feat_test`. Put your result in the next cell and use `ex_pred_RF` for the predicted quality values.

Hint: You might want to revisit the steps for RF classification we saw above.

```
[36]: num_trees = 100
tree_height = 8

ex_forest = RandomForestRegressor(n_estimators=num_trees,
↳max_depth=tree_height, random_state=123)
ex_forest.fit(feat_train, target_train)
ex_pred_RF = ex_forest.predict(feat_test)

print(f"Rsquared_RF: {ex_forest.score(feat_train, target_train)}")
```

Rsquared_RF: 0.7545065944574373

```
[37]: target_test['RF_predicted_quality'] = np.around(ex_pred_RF, decimals=2)
target_test['RF_absolut_deviation'] = abs(target_test['quality'] -
↳target_test['RF_predicted_quality'])
target_test
```

```
[37]:
```

	quality	Ridge_predicted_quality	Ridge_absolut_deviation	\
912	6	6.32	0.32	
772	5	4.97	0.03	
1037	5	4.77	0.23	
1106	6	6.57	0.57	
263	5	5.51	0.51	
...	...	...	...	
1466	7	5.57	1.43	
580	5	5.32	0.32	
1082	6	5.40	0.60	
1279	7	6.31	0.69	
1155	5	5.32	0.32	

	RF_predicted_quality	RF_absolut_deviation
912	6.41	0.41
772	5.10	0.10
1037	4.98	0.02
1106	6.44	0.44
263	5.73	0.73
...	...	...
1466	5.84	1.16
580	4.98	0.02
1082	5.49	0.51
1279	6.41	0.59
1155	5.18	0.18

```
[480 rows x 5 columns]
```

Finally, compare how the Random Forest and ridge regressor performed in comparison. For this, just execute the next cell.

```
[38]: RF_pred_MSE = (target_test['RF_absolut_deviation']**2).mean()
Ridge_pred_MSE = (target_test['Ridge_absolut_deviation']**2).mean()

print(f"Mean squared error of RandomForestRegressor: {RF_pred_MSE:.2f}")
print(f"Mean squared error of Ridge: {Ridge_pred_MSE:.2f}")
```

```
Mean squared error of RandomForestRegressor: 0.34
```

```
Mean squared error of Ridge: 0.42
```

Class_4-Dim_Reduction_Clustering

May 20, 2026

1 4. Dimensionality Reduction and Clustering

In the previous notebook, we could rely on labels which supervised the learning algorithm. Next, we consider the unsupervised setting and knowledge discovery. We focus on the idea of identifying a subset of relevant dimensions which capture the most variation in the data. This is closely related to the idea of compression.

In this notebook, we cover

- Principal Component Analysis (PCA) for dimensionality reduction
- k-means clustering
- Gaussian Mixture Models (GMMs)

and apply these techniques to simulated data and two dataset examples (digit recognition).

Keywords: `MNIST`, `np.linalg.eig`, `sklearn.decomposition.PCA`, `sklearn.cluster.KMeans`, `sklearn.mixture.GaussianMixture`

1.1 Dimensionality Reduction

The topic of dimensionality reduction plays a vital role in Machine Learning. Typically, our datasets collect a great number of predictors or covariates or consist of images with hundreds of pixels. Getting an idea of the input space and relevant features is challenging in these high-dimensional problems. Three main use cases include:

1. **Identifying relevant features / dimensions** in the input space which as much as possible preserves the variance in the data. In the following simplified example, the black arrows indicate the dimensions of highest variance:

The longer arrow can be viewed as the **main direction / dimension of variance** which accounts for about 90% of total variance and thus as the “more relevant” feature.

2. **Data compression:** In many applications we can reduce the number of dimensions to capture the relevant information. In the previous image, we might choose the longer arrow as a compressed version of our data. Another compression example is shown in the next image:

Here, we encoded an image of Maxim’s cat (with an approach called *Fast Fourier Transformation*) into a much lower dimensional compressed image which only requires 9% of pixels compared to the original image. Decoding this compressed image reconstructs the original image almost perfectly.

3. **Visualising** high-dimensional data is generally difficult for dimensions larger than 3. Thus, dimensionality reduction is an important tool for visualisation and identifying patterns.
-

1.2 Principal Component Analysis

Principal Component Analysis (PCA) ([Wikipedia](#)) is such a standard technique used for the above purposes and is of great relevance in a great variety of different disciplines.

Let us generate simple 2-dimensional dataset to identify the main dimensions of variation.

```
[1]: import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)

num_datapoints = 1000

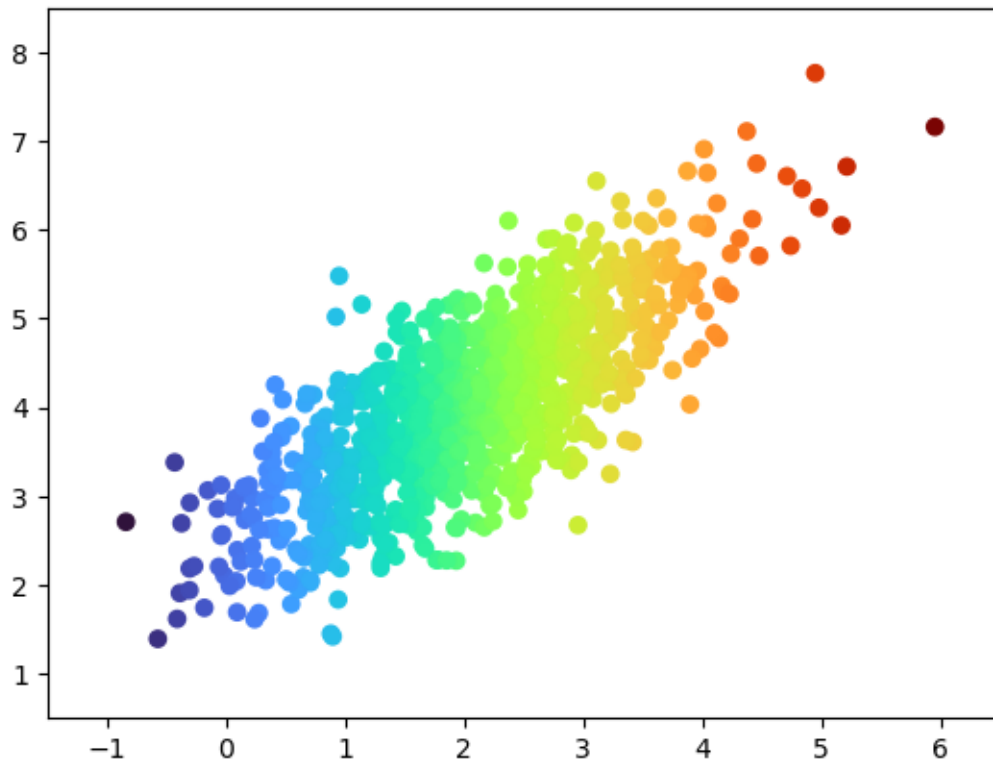
mu_generation = [2,4]
covariance_matrix_generation = np.array([[1, 0.75],
                                         [0.75, 1, ]
                                         ])

gauss_2d_rotated = np.random.multivariate_normal(mean=mu_generation,
                                                  ↵
                                                  ↵cov=covariance_matrix_generation,
                                                  size=num_datapoints)

plt.scatter(gauss_2d_rotated[:,0], gauss_2d_rotated[:,1], c=gauss_2d_rotated[:,
↵,0], cmap='turbo')

plt.ylim(0.5,8.5)
plt.xlim(-1.5,6.5)

plt.show()
```



The fundamental idea of PCA is to perform an eigendecomposition on the data covariance matrix $\text{cov}(X, X)$. If our dataset X is **centered**, i.e. the mean of all datapoints N is 0, we can write the (sample) covariance matrix as

$$\text{cov}(X, X) = \frac{1}{N-1} XX^\top. \quad (1)$$

The corresponding eigenvectors to the largest eigenvalues of this matrix provide the main axes of variation (principle components) in our dataset X .

Let's calculate these for our example and start by centering the dataset.

```
[2]: data_mean = np.mean(gauss_2d_rotated, axis=0)
     data_centered = gauss_2d_rotated - data_mean
```

Next, calculate the eigenvalues and eigenvectors using `np.linalg.eig` and sort them in descending order.

```
[3]: cov_matrix = np.cov(data_centered, rowvar=False)

     eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

     idx = eigenvalues.argsort()[::-1]
     eigenvalues = eigenvalues[idx]
```



```
eigenvectors = eigenvectors[:, idx]
```

Let's compare the calculated covariance and the covariance matrix we used to generate the data:

```
[4]: print(f"Calculated covariance:\n{cov_matrix}\n\nCovariance used for generation:\n\n{covariance_matrix_generation}")
```

Calculated covariance:

```
[[0.98354551 0.74236276]
 [0.74236276 0.97100139]]
```

Covariance used for generation:

```
[[1.  0.75]
 [0.75 1.  ]]
```

Choose the two eigenvectors with the highest eigenvalues and project the data onto these vectors. These are our “new” x- and y-axes.

```
[5]: data_pca_transform = np.dot(data_centered, eigenvectors[:, :2])

fig, axs = plt.subplots(1,2, figsize=(10,3))

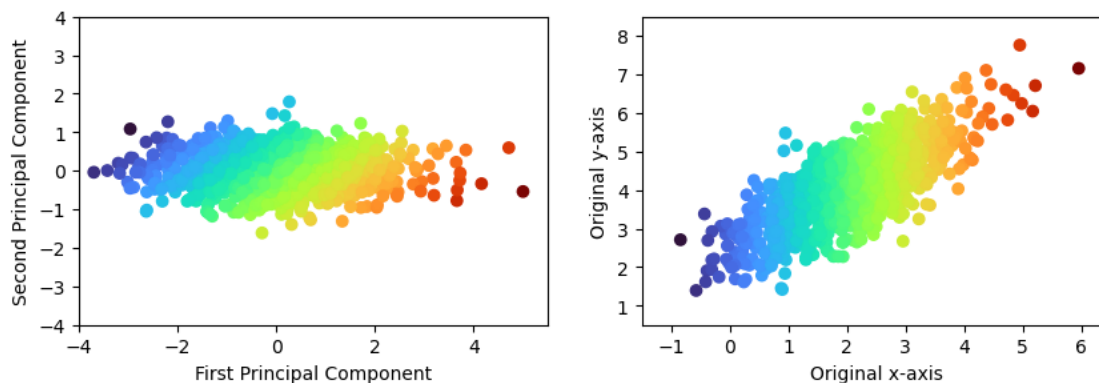
axs[0].scatter(data_pca_transform[:, 0], data_pca_transform[:, 1],
               c=gauss_2d_rotated[:,0], cmap='turbo')

axs[0].set_xlabel('First Principal Component')
axs[0].set_ylabel('Second Principal Component')
axs[0].set_xlim(-4,5.5)
axs[0].set_ylim(-4,4)

axs[1].scatter(gauss_2d_rotated[:,0], gauss_2d_rotated[:,1],
               c=gauss_2d_rotated[:,0], cmap='turbo')

axs[1].set_xlabel('Original x-axis')
axs[1].set_ylabel('Original y-axis')
axs[1].set_ylim(0.5,8.5)
axs[1].set_xlim(-1.5,6.5)

plt.show()
```



Note that in this 2d example, the **projection on the two eigenvectors corresponds to a rotation** of our dataset. This will be different, if the original dataset has more dimensions.

1.2.1 As before, we can just use `scikit-learn`

and its functionality with the function `PCA`!

```
[6]: from sklearn.decomposition import PCA

num_eigvectors = 2

data_pca = PCA(n_components=num_eigvectors)
data_pca.fit(gauss_2d_rotated)

data_pca_transform = data_pca.transform(gauss_2d_rotated)
```

We can even check the explained variance ratio of the first two principal components:

```
[7]: print(data_pca.explained_variance_ratio_)
```

```
[0.87982678 0.12017322]
```

1.2.2 MNIST Example

As a first, more interesting example, we examine the MNIST dataset!

```
[8]: mnist_data = np.load('data/mnist_data_5k.npy', allow_pickle=True)
mnist_targets = np.load('data/mnist_labels_5k.npy', allow_pickle=True)

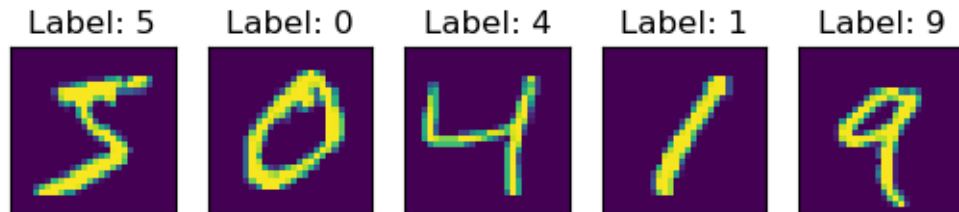
fig, axs = plt.subplots(1,5)

for i, ax in enumerate(axs.ravel()):
    ax.imshow(mnist_data[i].reshape(28,28))

    ax.set_title(f"Label: {mnist_targets[i]}")
```

```
ax.set_xticks([])
ax.set_yticks([])

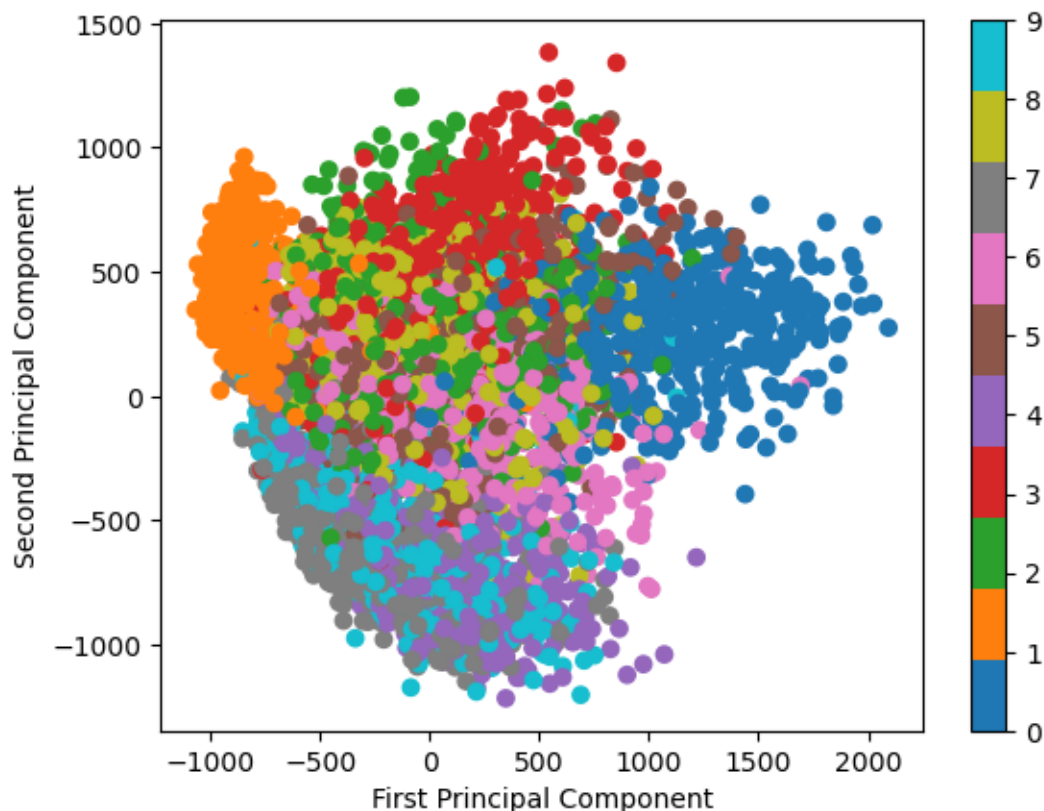
plt.show()
```



Let's see what the projection on the first two eigenvectors looks like in MNIST:

```
[9]: data_pca = PCA(n_components=2)
data_pca.fit(mnist_data)
data_pca_transform_mnist = data_pca.transform(mnist_data)

plt.scatter(data_pca_transform_mnist[:, 0], data_pca_transform_mnist[:, 1],
            c=mnist_targets, cmap='tab10')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.colorbar()
plt.show()
```



The result shows that **similar numbers are clustered** and thus that samples from the **same class share similar features**.

Let's examine the explained variance ratios:

```
[10]: print(data_pca.explained_variance_ratio_)
```

```
[0.09867566 0.07404546]
```

These two dimensions account for **less than 20% of the variance**. In other words, we might miss about 80% of relevant variation in our data and should not base our computation solely on these first two principal components!

1.3 Clustering

Another important approach to identifying patterns in an unsupervised fashion is clustering. In the previous examples, we have already seen instances where datapoints might form clusters indicating similar features.

Two popular approaches to clustering are **k-means** and **Gaussian Mixture Models** which we consider in the following.

k-means The underlying idea of k-means ([Wikipedia](#)) is assign datapoints to k cluster means / centroids in the dataset. The following GIF illustrates this for $k = 3$ centroids:

GIF source: Wikipedia

More formally, we consider a set of N datapoints $X = (x_1, \dots, x_N)$ which we try to group into k groups forming sets $S_1, S_2, \dots, S_k$. The objective function is given by

$$\operatorname{argmin}_S \sum_{i=1}^k \sum_{x \in S_i} \|x - \mu_i\|^2 \quad (2)$$

where $\mu_i = \frac{1}{|S_i|} \sum_{x \in S_i} x$ is the mean of the datapoints in cluster S_i .

The algorithm **starts** with placing the cluster means / centroids μ_i in a random position (see GIF above at Iteration #0). Datapoints are assigned to the closest mean μ_i , i.e. to cluster S_i , and a **new cluster mean** is calculated. The algorithm **iterates** this step several times until the cluster assignments remain unchanged.

Note that in an unsupervised setting, **we generally do not know the number of clusters k !** Thus, k is a *hyper-parameter* we need to choose!

Let's create a clustered dataset with two clusters of 100 datapoints each, i.e. $N = 200$.

```
[11]: import numpy as np
import matplotlib.pyplot as plt

np.random.seed(123)

num_datapoints = 100

cluster_1 = np.random.normal( 2, 1, size=(num_datapoints, 2))
cluster_2 = np.random.normal(-2, 1, size=(num_datapoints, 2))

cluster_data = np.concatenate([cluster_1, cluster_2])
```

We make use of the `scikit-learn` implementation of `KMeans` and choose $k = 2$.

```
[12]: from sklearn.cluster import KMeans

k = 2

kmeans = KMeans(n_clusters=k, random_state=123, n_init=10)
kmeans.fit(cluster_data)

cluster_prediction = kmeans.predict(cluster_data)

plt.scatter(cluster_data[:,0], cluster_data[:,1], c=cluster_prediction)
plt.show()
```


where $\mu = (\mu_1, \dots, \mu_k)$, $\Sigma = (\Sigma_1, \dots, \Sigma_k)$, and $w = (w_1, \dots, w_k)$ is the weighting of the different mixture components.

As before, we start by choosing a number of k clusters and setting the parameters μ, Σ to some initial values. The cluster assignment and parameters are then updated iteratively with the **expectation-maximisation (EM) algorithm** until cluster assignments, i.e. the parameters μ, Σ , do not change any longer.

Let's consider the same dataset as for k-means and make use of the `scikit-learn` `GaussianMixture` method.

```
[13]: from sklearn.mixture import GaussianMixture

k = 2

GMM = GaussianMixture(n_components=k)
GMM.fit(cluster_data)
```

```
[13]: GaussianMixture(n_components=2)
```

In order to provide the probabilities, we need to obtain a grid of datapoints which can do with `numpy.meshgrid`.

```
[14]: x_min = cluster_data[:, 0].min() - 1
x_max = cluster_data[:, 0].max() + 1

y_min = cluster_data[:, 1].min() - 1
y_max = cluster_data[:, 1].max() + 1

xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                     np.linspace(y_min, y_max, 100))
```

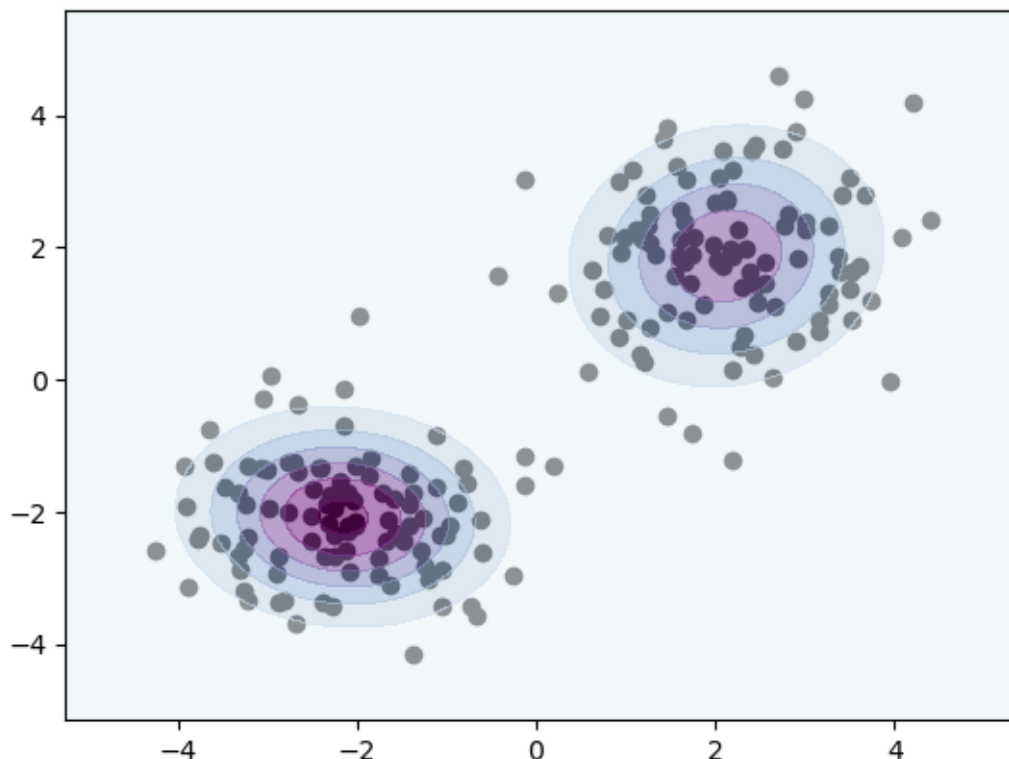
Finally, we can obtain the probability predictions with `score_samples` and plot the results. Note that this method provides the log-likelihood, i.e. we need to exponentiate these outputs!

```
[15]: probab = GMM.score_samples(np.c_[xx.ravel(), yy.ravel()])
probab = np.exp(probab)
probab = probab.reshape(xx.shape)

plt.scatter(cluster_data[:, 0], cluster_data[:, 1], c='black')
plt.contourf(xx, yy, probab, alpha=0.6, cmap='BuPu')

plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)

plt.show()
```



Note that `np.c_[A,B]` concatenates NumPy arrays A and B along the second axis.

1.4 Exercise Section

In the next two exercises, we use k-means and a Gaussian Mixture Model on the projection of the MNIST data on its two first principal components.

(1.) Firstly, to simplify the clustering task, we will only consider digits 0, 3, 6, and 9. For this make us of `%3` to identify all of these labels in `mnist_targets` and select only the respective elements in `data_pca_transform_mnist` and `mnist_targets` (defined above). Provide the filtered datasets `ex_data` and the corresponding labels in `ex_targets` in the following cell.

```
[ ]: # fill in
```

If you were successful, you can plot the filtered dataset below, which should look like the following plot:

```
[ ]: plt.scatter(ex_data[:, 0], ex_data[:, 1], c=ex_targets, cmap='tab10')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.colorbar()
plt.show()
```


(2.) Implement a k-means approach for clustering `ex_data` and provide the k-means object as `kmeans` and cluster predictions in `cluster_prediction`.

```
[ ]: # fill in
```

Use `cluster_prediction` and the above defined `kmeans` to plot the clusters with the following cell.

```
[ ]: plt.scatter(ex_data[:,0], ex_data[:,1], c=cluster_prediction)
plt.scatter(kmeans.cluster_centers_[0], kmeans.cluster_centers_[1],
            marker='X', c='red')
plt.show()
```

(3.) Implement an Gaussian Mixture Model approach for clustering `ex_data`. Provide the Gaussian Mixture Model as object `GMM`, the grid coordinates `xx` and `yy` (as seen before) and the predictions in `probab`.

```
[ ]: # fill in
```

If you have defined `GMM`, `xx`, `yy`, and `probab` in the cell, the next cell will allow you to plot your result.

```
[ ]: plt.scatter(ex_data[:, 0], ex_data[:, 1], c='black')
plt.contourf(xx, yy, probab, alpha=0.6, cmap='BuPu')
plt.scatter(GMM.means_[0], GMM.means_[1], marker='X', c='orange')

plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)

plt.show()
```

1.5 Proposed Solutions

In the next two exercises, we use k-means and a Gaussian Mixture Model on the projection of the MNIST data on its two first principal components.

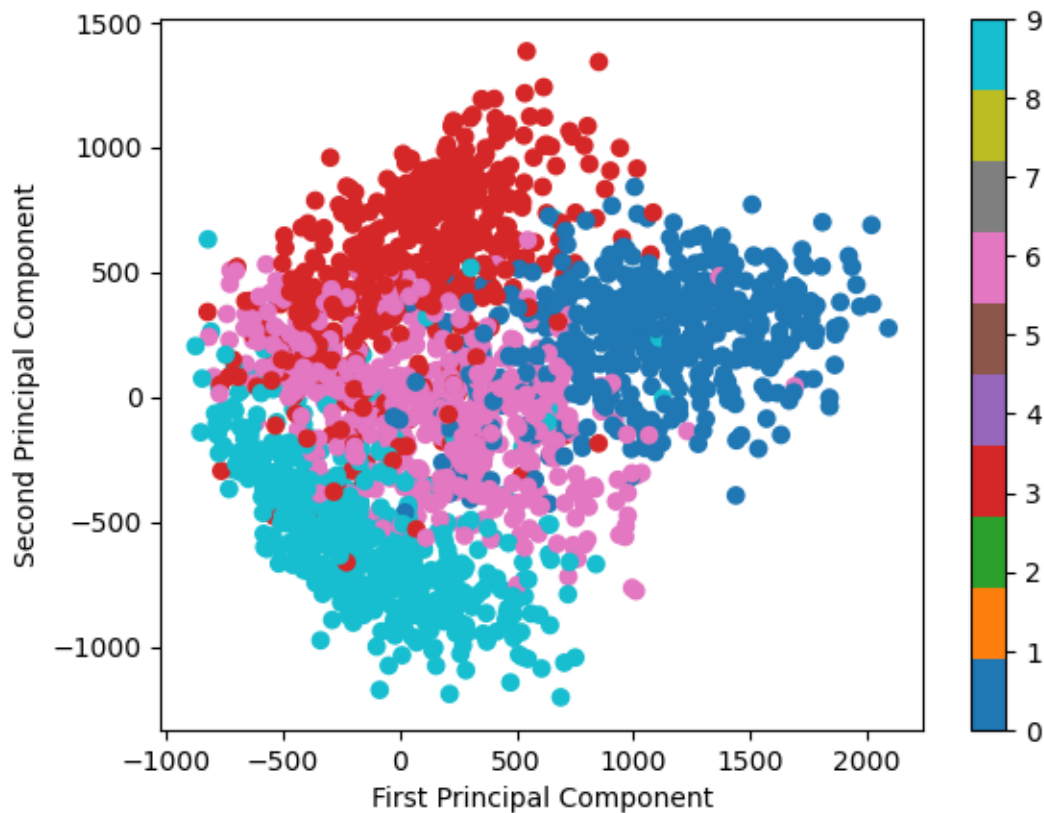
(1.) Firstly, to simplify the clustering task, we will only consider digits 0, 3, 6, and 9. For this make us of `%3` to identify all of these labels in `mnist_targets` and select only the respective elements in `data_pca_transform_mnist` and `mnist_targets` (defined above). Provide the filtered datasets `ex_data` and the corresponding labels in `ex_targets` in the following cell

```
[18]: ex_data = data_pca_transform_mnist[mnist_targets%3==0]
ex_targets = mnist_targets[mnist_targets%3==0]
```

If you were successful, you can plot the filtered dataset below, which should look like the following plot:

```
[19]: plt.scatter(ex_data[:, 0], ex_data[:, 1], c=ex_targets, cmap='tab10')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
```

```
plt.colorbar()
plt.show()
```



(2.) Implement an k-means approach for clustering `ex_data` and provide the k-means object as `kmeans` and cluster predictions in `cluster_prediction`.

```
[20]: from sklearn.cluster import KMeans

k = 4

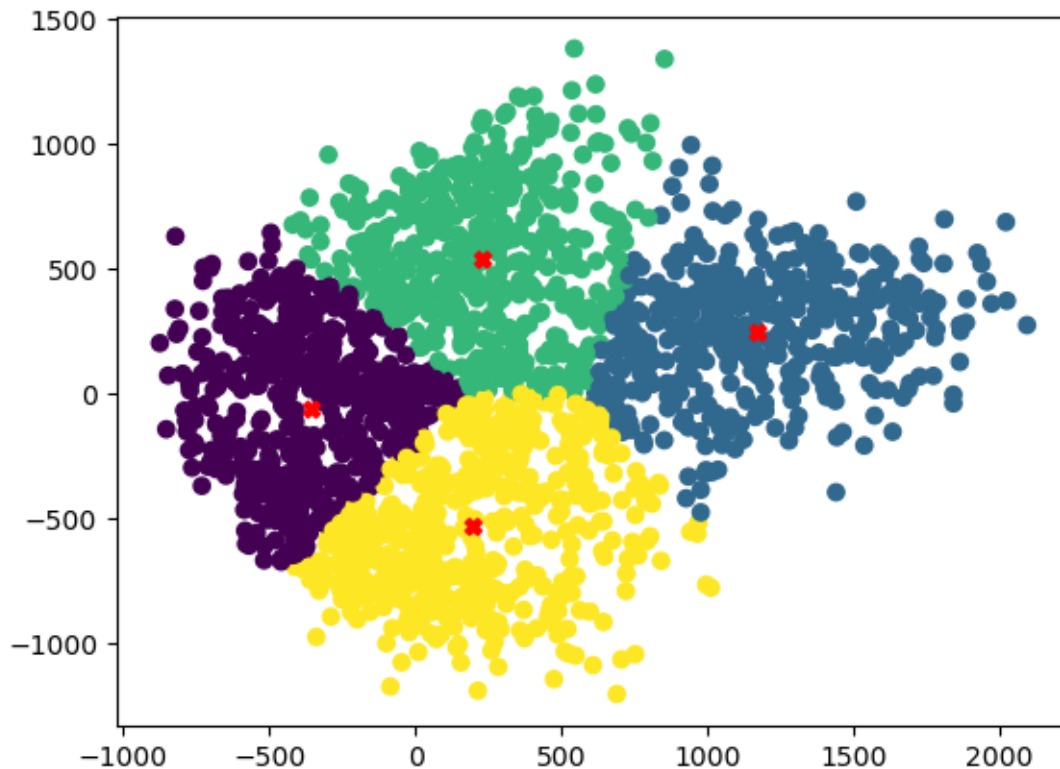
kmeans = KMeans(n_clusters=k, random_state=123)
kmeans.fit(ex_data)

cluster_prediction = kmeans.predict(ex_data)
```

```
/Users/samarinm/anaconda3/APML/lib/python3.11/site-
packages/sklearn/cluster/_kmeans.py:1412: FutureWarning: The default value of
`n_init` will change from 10 to 'auto' in 1.4. Set the value of `n_init`
explicitly to suppress the warning
    super()._check_params_vs_input(X, default_n_init=10)
```

Use `cluster_prediction` and the above defined `kmeans` to plot the clusters with the following cell.

```
[21]: plt.scatter(ex_data[:,0], ex_data[:,1], c=cluster_prediction)
plt.scatter(kmeans.cluster_centers[:,0], kmeans.cluster_centers[:,1],
            marker='X', c='red')
plt.show()
```



(3.) Implement an Gaussian Mixture Model approach for clustering `ex_data`. Provide the Gaussian Mixture Model as object `GMM`, the grid coordinates `xx` and `yy` (as seen before) and the predictions in probab.

```
[22]: from sklearn.mixture import GaussianMixture

k = 4

GMM = GaussianMixture(n_components=k)
GMM.fit(ex_data)

x_min = ex_data[:, 0].min() - 1
x_max = ex_data[:, 0].max() + 1

y_min = ex_data[:, 1].min() - 1
y_max = ex_data[:, 1].max() + 1
```

```
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                     np.linspace(y_min, y_max, 100))

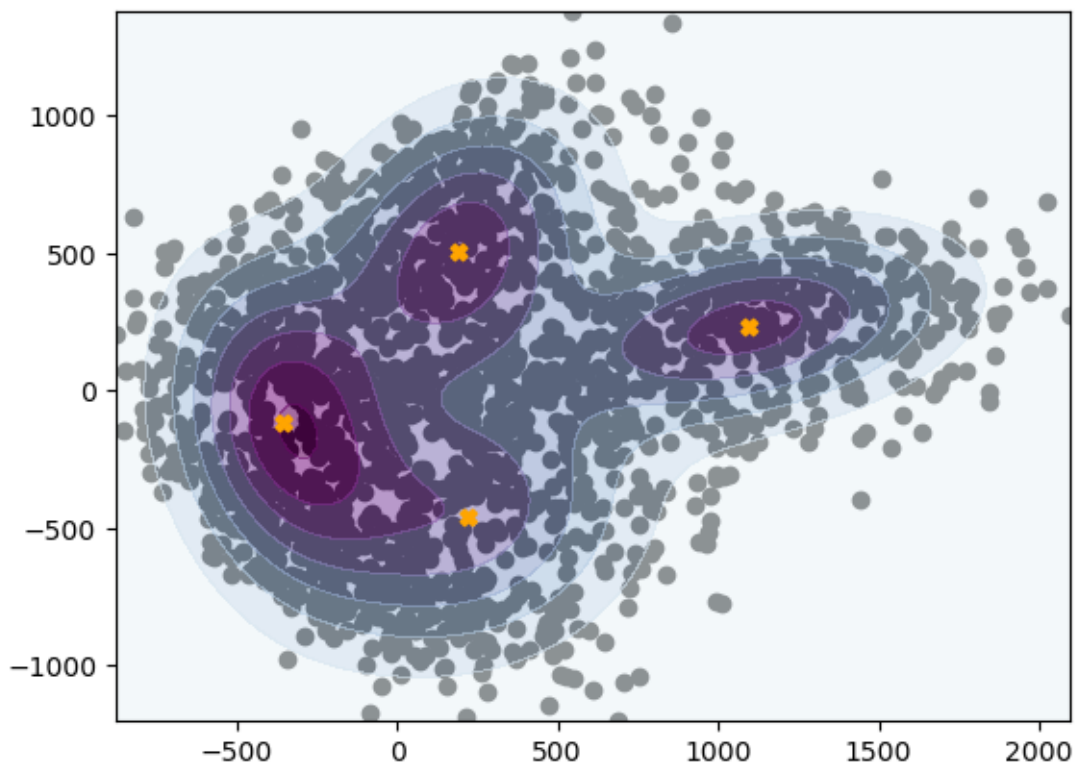
probab = GMM.score_samples(np.c_[xx.ravel(), yy.ravel()])
probab = np.exp(probab)
probab = probab.reshape(xx.shape)
```

If you have defined GMM, xx, yy, and probab in the cell, the next cell will allow you to plot your result.

```
[23]: plt.scatter(ex_data[:, 0], ex_data[:, 1], c='black')
plt.contourf(xx, yy, probab, alpha=0.6, cmap='BuPu')
plt.scatter(GMM.means_[:,0], GMM.means_[:,1], marker='X', c='orange')

plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)

plt.show()
```



```
[24]: print(f"GMM cluster means = \n{GMM.means_}\n")
print(f"k-means cluster means = \n{kmeans.cluster_centers_}")
```

GMM cluster means =

```
[[ 189.23126455  509.27534117]
 [-353.88561238 -119.4424008 ]
 [1096.73196992  233.27373145]
 [ 221.66004826 -461.50031972]]
```

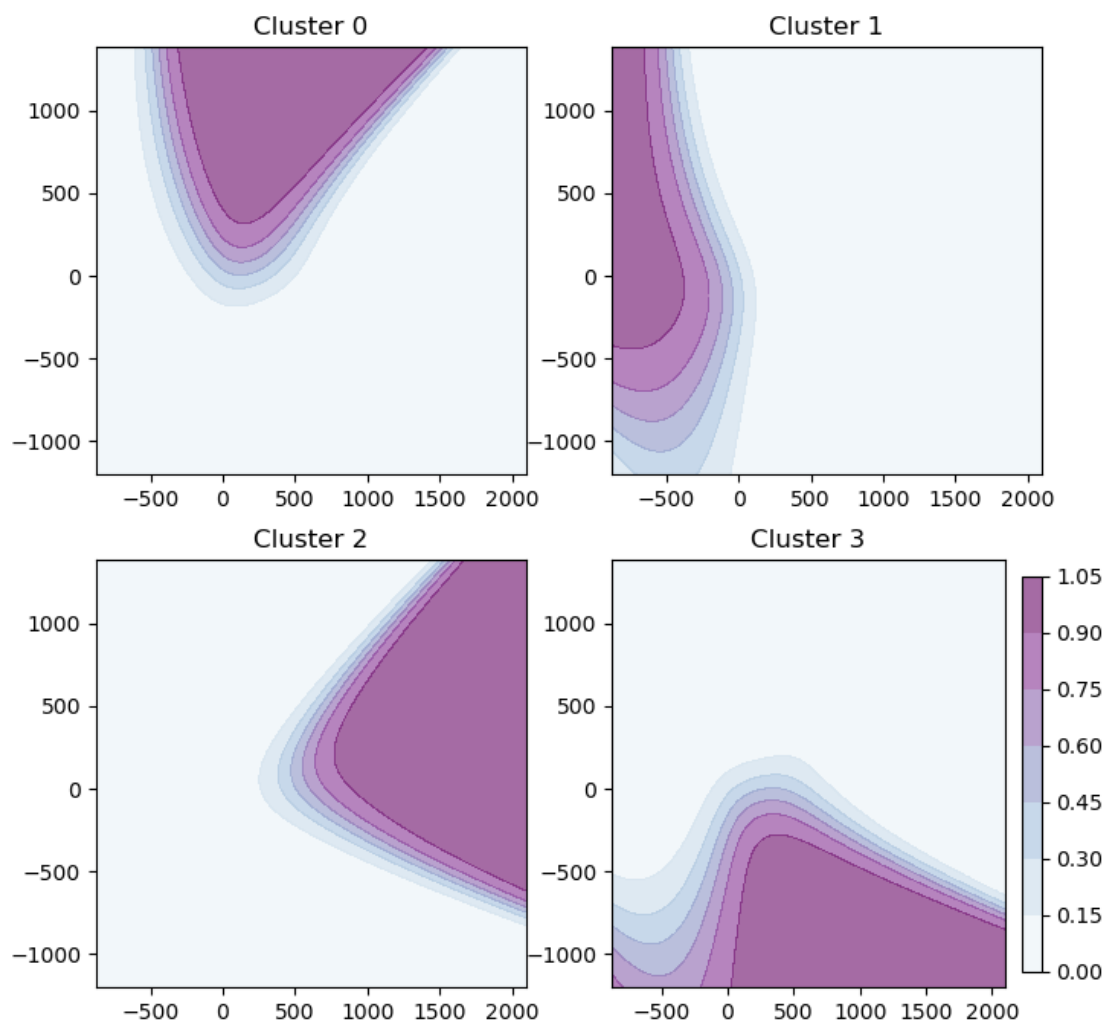
```
k-means cluster means =
[[-358.65065773  -62.03116211]
 [1172.16616214  245.21794471]
 [ 231.10450973  539.9021651 ]
 [ 197.06796498 -531.87278348]]
```

```
[47]: probab_on_grid = GMM.predict_proba(np.c_[xx.ravel(), yy.ravel()])

fig, axs = plt.subplots(2,2, figsize = (8,8))

for i, ax in enumerate(axs.ravel()):
    probab_cluster = probab_on_grid[:,i].reshape(xx.shape)
    im = ax.contourf(xx, yy, probab_cluster, alpha=0.6, cmap='BuPu')
    ax.set_title(f"Cluster {i}")

fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
plt.show()
```



```
[ ]:
```

Class_5-Kernel_Methods

May 20, 2026

1 5. Kernel Methods

In contrast to the idea of dimensionality reduction and compression, another popular direction in Machine Learning considers the opposite approach of solving problems in a higher-dimensional space. In this notebook we present the general idea of **kernel methods** on the example of

- Kernel ridge regression and
- Gaussian Processes

mostly focusing on simulated data to highlight their main benefits.

Keywords: `feature mapping`, `RBF kernel`, `sklearn.kernel_ridge.KernelRidge`, `sklearn.gaussian_process.GaussianProcessRegressor`, `sklearn.gaussian_process.kernels.RBF`, `sklearn.pipeline.make_pipeline`, `sklearn.preprocessing.StandardScaler`

1.1 Lifting Data into higher Dimensions

Most models considered in the previous notebooks can be regarded as essentially linear models. Although linear models have their advantages (in particular with respect to mathematical derivations and guarantees), they severely limit the kind of functions we can represent. A common approach therefore is to **map** the problem from the input space into a **higher-dimensional feature space** as illustrated in the next figure for a classification setting.

The feature mapping ϕ allows us to map data from a lower-dimensional input space into a higher-dimensional feature space. The key idea is that identifying the complicated decision boundary in input space might be challenging, but **in feature space a linear model suffices** to identify the planar decision boundary.

More importantly, we can show that we do not need to ever evaluate $\phi(x)$. We can rely on the **kernel trick**! This implies that we only need to evaluate scalar products of feature mappings – called kernels – $\kappa(x, x') = \langle \phi(x), \phi(x') \rangle$ for pairs of datapoints x, x' in our dataset X .

One such wildly used kernel is the **radial basis function (RBF) kernel**

$$\kappa(x, x') = \exp \left(-\frac{1}{2\sigma^2} \|x - x'\|_2^2 \right) \quad (1)$$

which corresponds to an infinite-dimensional feature space. In general, we can design task-dependent kernels which might extract features we deem relevant to our problem. Furthermore, we can “kernelise” standard linear approaches to become more powerful!

1.2 Kernel Ridge Regression

We revisit our standard regression techniques and see how the kernelised version extends what we have seen before. We formulated the linear regression model before as

$$Y = Xw + \epsilon. \quad (2)$$

Let's simulate a sinusoidal dataset (with noise) of $N = 200$ datapoints.

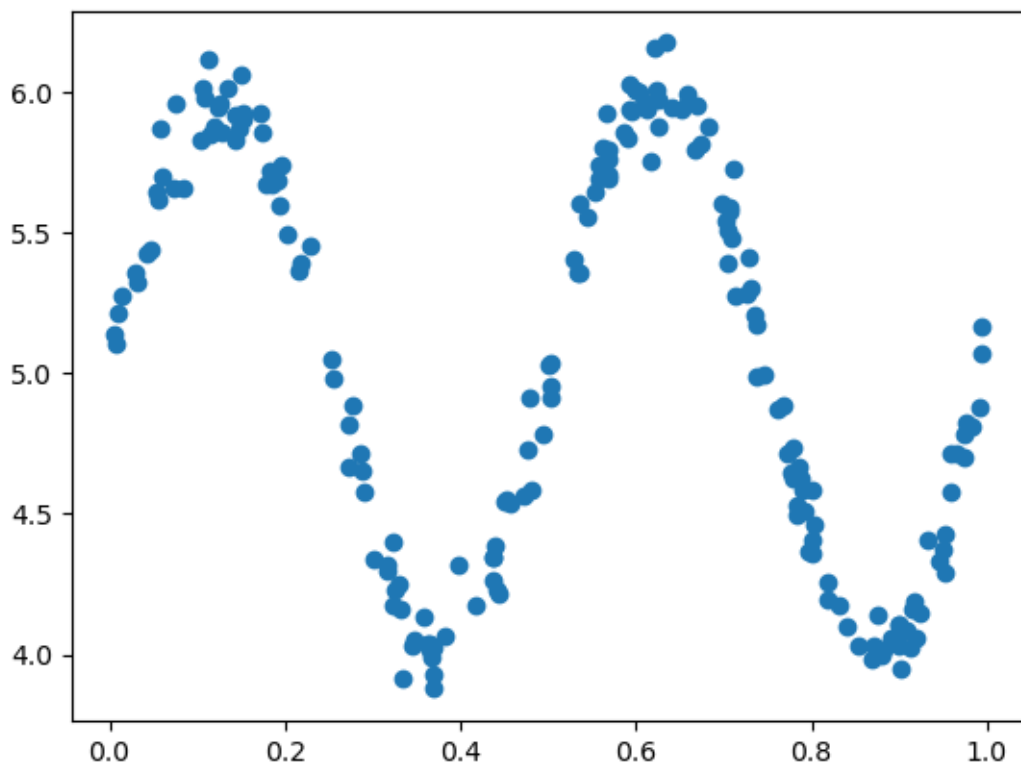
```
[1]: import numpy as np
import matplotlib.pyplot as plt

np.random.seed(1234)

num_datapoints = 200
X = np.random.rand(num_datapoints)
X = np.sort(X).reshape(-1,1)

Y = np.sin(4 * np.pi * X) + np.random.randn(num_datapoints, 1) * 0.1 + 5

plt.scatter(X,Y)
plt.show()
```



As we have seen before, we will try to fit these observations with a polynomial with linear and ridge regression.

In the following, we make use of an advanced `scikit-learn` tool. We build a **pipeline** (see [documentation](#)) which allows data processing steps to be executed with an estimator like `LinearRegression` or `Ridge`. In particular, we use the `StandardScaler` method (see [documentation](#)), which is a method to standardise our dataset, i.e. subtract the sample mean and divide by the sample standard deviation. With such a pipeline we can do some “data curation” before training!

```
[2]: from sklearn.linear_model import LinearRegression, Ridge
      from sklearn.pipeline import make_pipeline
      from sklearn.preprocessing import StandardScaler

      X_poly = np.hstack((X, X ** 2, X ** 3, X ** 4, X ** 5))

      linreg = make_pipeline(StandardScaler(), LinearRegression())
      linreg.fit(X_poly, Y)
      Y_OLS = linreg.predict(X_poly)
      Rsquared_lin = linreg.score(X_poly, Y)

      ridgereg = make_pipeline(StandardScaler(), Ridge(alpha=0.001))
      ridgereg.fit(X_poly, Y)
      Y_ridge = ridgereg.predict(X_poly)
      Rsquared_ridge = ridgereg.score(X_poly, Y)

      print(f"Rsquared_lin = {Rsquared_lin:.2f}, Rsquared_ridge = {Rsquared_ridge:.
        ↪2f}")

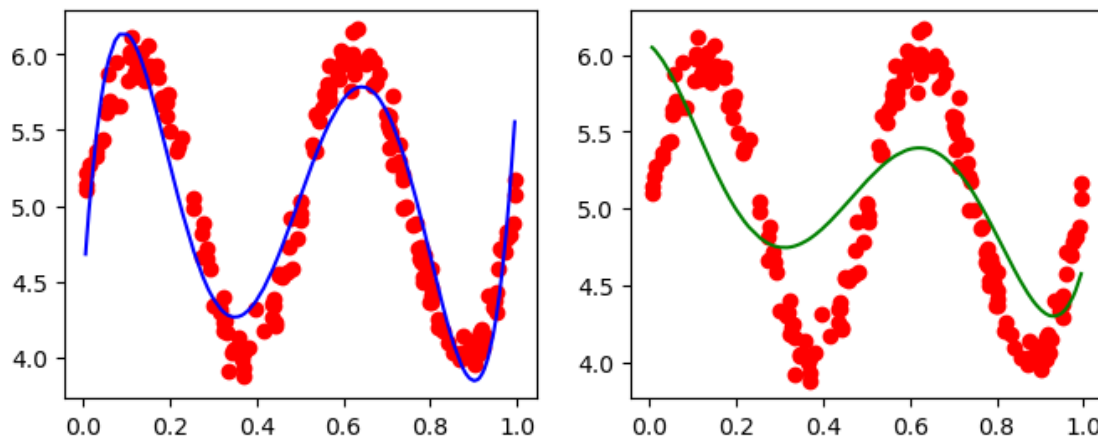
      fig, axs = plt.subplots(1, 2, figsize=(8, 3))

      axs[0].scatter(X[:, 0], Y, color='red')
      axs[0].plot(X[:, 0], Y_OLS, color='blue')

      axs[1].scatter(X[:, 0], Y, color='red')
      axs[1].plot(X[:, 0], Y_ridge, color='green')

      plt.show()
```

Rsquared_lin = 0.89, Rsquared_ridge = 0.55



1.2.1 Kernel Ridge Regression (KRR)

incorporates the idea of employing a feature mapping ϕ and we rewrite the linear regression model as

$$Y = \phi(X)w + \epsilon = f(x) + \epsilon. \quad (3)$$

Similar to the ridge regression result, the solution can be shown to be

$$w_{\text{KRR}} = \phi(X)^\top (\phi(X)^\top \phi(X) + \lambda \mathbb{1}_N)^{-1} Y. \quad (4)$$

and the learned function (without going into the mathematical details) is then given by

$$f(x) = K(x, X) (K(X, X) + \lambda \mathbb{1}_{d_N})^{-1} Y \quad (5)$$

with K being the kernel matrix $K_{nm} = \kappa(x_n, x_m) = \langle \phi(x_n), \phi(x_m) \rangle$.

Let's see how we can make use of for the above dataset and define the RBF kernel.

```
[3]: def rbf_kernel(x1, x2, sigma=1.0):
      return np.exp(-np.linalg.norm(x1 - x2) ** 2 / (2 * sigma ** 2))
```

As usually, we use a `scikit-learn` estimator: [KernelRidge](#).

```
[4]: from sklearn.kernel_ridge import KernelRidge

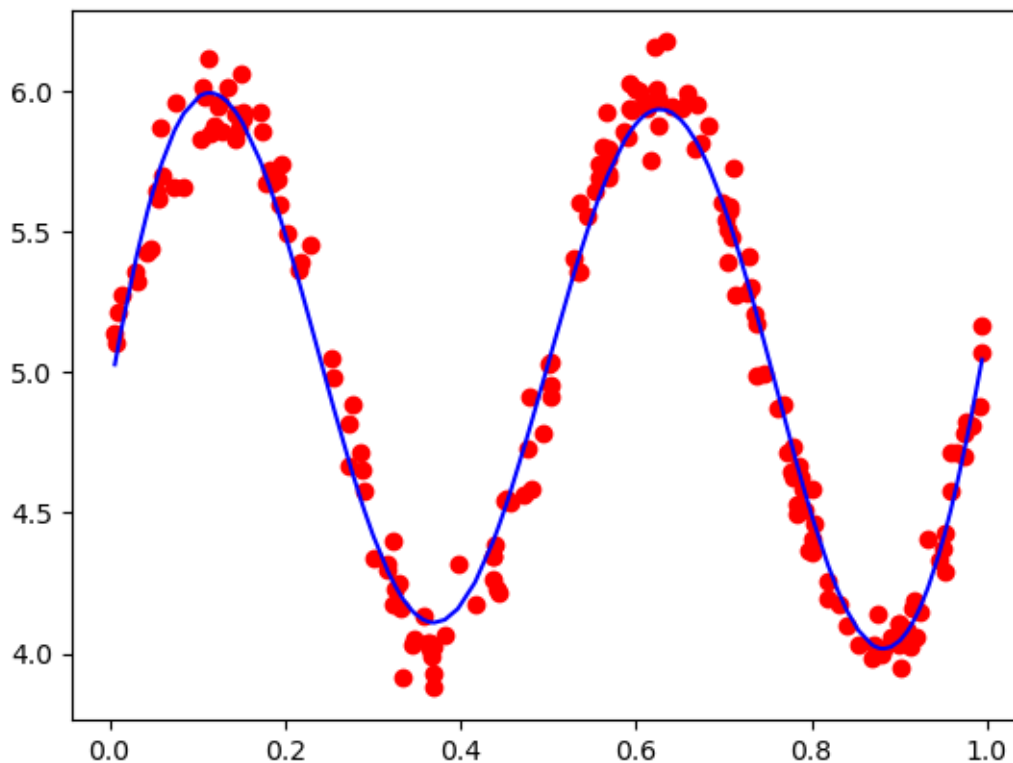
KRR = make_pipeline(StandardScaler(), KernelRidge(kernel=rbf_kernel, alpha=0.
    ↳ 01, gamma=0.1))
KRR.fit(X, Y)
Y_KRR = KRR.predict(X)

Rsquared_KRR = KRR.score(X, Y)
```

```
print(f"Rsquared_lin = {Rsquared_lin:.2f}, Rsquared_ridge = {Rsquared_ridge:.2f}, Rsquared_KRR = {Rsquared_KRR:.2f}")

plt.scatter(X[:, 0], Y, color='red')
plt.plot(X[:, 0], Y_KRR, color='blue')
plt.show()
```

Rsquared_lin = 0.89, Rsquared_ridge = 0.55, Rsquared_KRR = 0.98



1.3 Gaussian Processes

Kernel Ridge Regression has a close connection to **Gaussian Processes (GPs)** ([Wikipedia](#) or [scikit-learn](#)). Informally, a GP extends the notion of random variables to random functions. In a regression setting with input matrix X , the GP prior distribution $p(f | X) = \mathcal{N}(0, K)$ allows us to model the regression function $f(X) = [f(x_1), \dots, f(x_N)]^\top$ as

$$(f(x_1), \dots, f(x_N)) \sim \mathcal{N}(0, K) \quad (6)$$

with sample covariance matrix $K = \text{cov}(X, X)$. Let us consider a GP regression model with noisy responses Y as before, i.e.

$$Y = f(x) + \epsilon. \quad (7)$$

Leaving the mathematical derivation aside, in order to make a prediction for function $f(x)$ with new (test) data x , we can show that the posterior predictive distribution ([Wikipedia](#)) has the form

$$p(f | x, X, Y) = \mathcal{N}(\mu(x), \Sigma(x, X)) \quad (8)$$

$$\mu(x) = K(x, X)(K(X, X) + \lambda \mathbb{1}_N)^{-1} Y \quad (9)$$

$$\Sigma(x, X) = K(x, x) - K(x, X)^\top (K(X, X) + \lambda \mathbb{1}_N)^{-1} K(x, X) \quad (10)$$

Note that what we have written earlier for $f(x)$ in KRR **corresponds exactly** to $\mu(x)$ in GP regression, i.e. there is a close connection between KRR and GP regression (see e.g. [this comparison](#)). This motivates that the **covariance matrix K is a kernel**, too! Compared to KRR, GPs can be more flexible (as we can sample functions from the posterior) and we can make use of the GP formalism for training. Also, GPs enable us to quantify the uncertainty of a prediction due to $\Sigma(x, X)$.

Let's study this in a simplified example with $N = 4$ datapoints. As always, we can use `scikit-learn` methods for the [GaussianProcessRegressor](#) and different kernels. We employ the [RBF](#) kernel as well as a [ConstantKernel](#), i.e. $\kappa(x, x') = \text{const.}$

```
[5]: from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, ConstantKernel

np.random.seed(1234)

num_datapoints = 4

X = np.random.rand(num_datapoints, 1)*2
Y = np.sin(2 * np.pi * X) + np.random.randn(num_datapoints, 1) * 0.1 + 5

kernel = ConstantKernel(constant_value=1.0, constant_value_bounds=(0.001, 1000))
kernel = kernel * RBF(length_scale=0.5, length_scale_bounds=(0.01, 100))

GPreg = GaussianProcessRegressor(kernel=kernel, alpha=0.1,
    ↪n_restarts_optimizer=10)
GPreg.fit(X, Y)

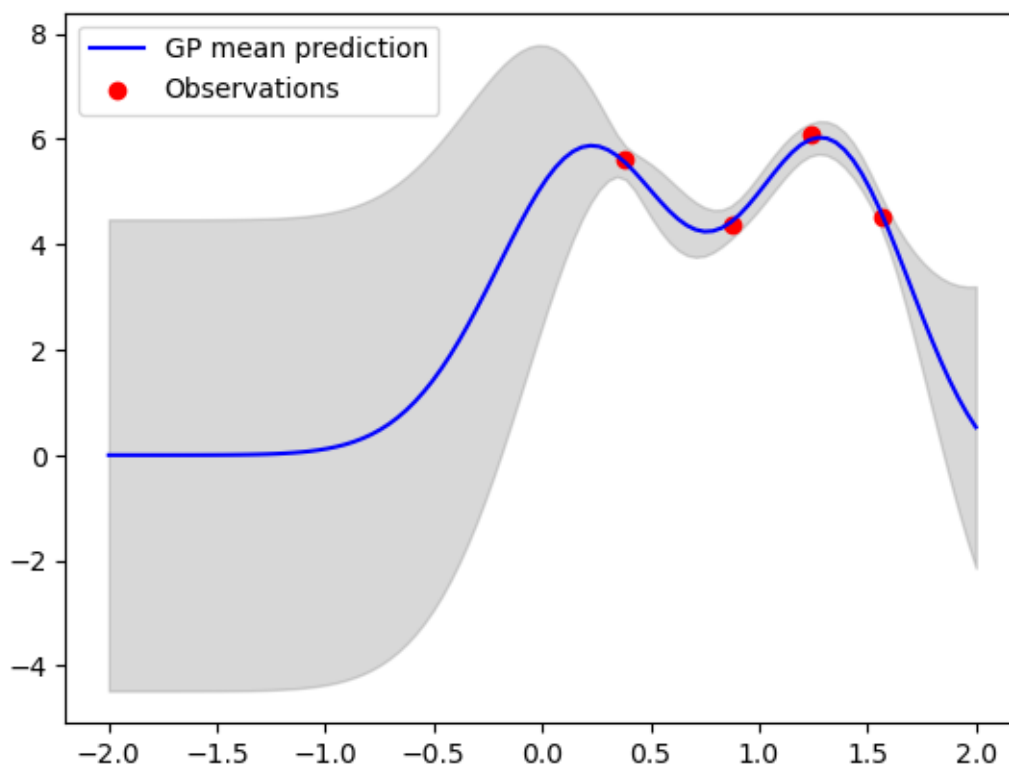
X_pred = np.linspace(-2.0, 2.0, 100).reshape(-1, 1)
Y_GPreg, stddev_GPreg = GPreg.predict(X_pred, return_std=True)

X_pred = X_pred.reshape(-1)
Y_GPreg = Y_GPreg.reshape(-1)

plt.fill_between(X_pred, Y_GPreg - stddev_GPreg, Y_GPreg + stddev_GPreg,
    ↪alpha=0.3, color='grey')
```

```
plt.plot(X_pred, Y_GPreg, color='blue', label='GP mean prediction')
plt.scatter(X, Y, color='red', label='Observations')

plt.legend(loc='upper left')
plt.show()
```



1.4 Exercise Section

For the exercises, we consider again the red wine dataset but this time train a (1.) kernel ridge regressor and (2.) Gaussian Process regressor. As in notebook 2, we want to predict the quality values on the test set `feat_test` and compare the mean squared errors (MSEs) of KRR and GP regression to the previous results for ridge regression (0.42) and Random Forest regression (0.34).

In order, to prepare the data and kernel, load the following cell:

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.gaussian_process.kernels import RBF, ConstantKernel

np.random.seed(1234)

ex_kernel = RBF(length_scale=5, length_scale_bounds=(0.01, 100))

wine_data = pd.read_csv('data/winequality-red.csv', sep=';')

ex_target = wine_data['quality']
ex_features = wine_data.drop(['quality'], axis=1)

feat_train, feat_test, target_train, target_test = train_test_split(
    ex_features, ex_target, test_size = 0.3, random_state=123)

target_test = target_test.to_frame()

```

```
[ ]: feat_train.head(5)
```

```
[ ]: target_train.head(5)
```

(1.) Implement kernel ridge regression as we have seen it in this notebook. Please provide the predictions for the test set `feat_test` in `ex_pred_KRR`.

```
[ ]: # Fill in
```

We use `ex_pred_KRR` in the following cell to provide the results. You can just execute the next cell.

```

[ ]: target_test['KRR_predicted_quality'] = np.around(ex_pred_KRR, decimals=2)
target_test['KRR_absolut_deviation'] = abs(target_test['quality'] -
    ↪target_test['KRR_predicted_quality'])
print(target_test)

KRR_pred_MSE = (target_test['KRR_absolut_deviation']**2).mean()
print(f"\nMean squared error of KernelRidge: {KRR_pred_MSE:.2f}")

```

(2.) Implement GP regression as we have seen it in this notebook. Please provide the predictions for the test set `feat_test` in `ex_pred_GPreg`.

Hints: If the computation might take too long, interrupt the execution with `/ Interrupt Kernel` in the panel above and set `n_restarts_optimizer=1`.

```
[ ]: # Fill in
```

We use `ex_pred_GPreg` in the following cell to provide the results. You can just execute the next cell.

```

[ ]: target_test['GPreg_predicted_quality'] = np.around(ex_pred_GPreg, decimals=2)
target_test['GPreg_absolut_deviation'] = abs(target_test['quality'] -
    ↪target_test['GPreg_predicted_quality'])

```

```
print(target_test)

GPreg_pred_MSE = (target_test['GPreg_absolut_deviation']**2).mean()

print(f"\nMean squared error of GaussianProcessRegressor: {GPreg_pred_MSE:.2f}")
print(f"Mean squared error of KernelRidge: {KRR_pred_MSE:.2f}")
```

1.5 Proposed Solutions

For the exercises, we consider again the red wine dataset but this time train a (1.) kernel ridge regressor and (2.) Gaussian Process regressor. As in notebook 2, we want to predict the quality values on the test set `feat_test` and compare the mean squared errors (MSEs) of KRR and GP regression to the previous results for ridge regression (0.42) and Random Forest regression (0.34).

In order, to prepare the data and kernel, load the following cell:

```
[11]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.gaussian_process.kernels import RBF, ConstantKernel

np.random.seed(1234)

ex_kernel = RBF(length_scale=5, length_scale_bounds=(0.01, 100))

wine_data = pd.read_csv('data/winequality-red.csv', sep=';')

ex_target = wine_data['quality']
ex_features = wine_data.drop(['quality'], axis=1)

feat_train, feat_test, target_train, target_test = train_test_split(
    ex_features, ex_target, test_size = 0.3, random_state=123)

target_test = target_test.to_frame()
```

(1.) Implement kernel ridge regression as we have seen it in this notebook. Please provide the predictions for the test set `feat_test` in `ex_pred_KRR`.

```
[25]: from sklearn.kernel_ridge import KernelRidge

KRR_ex = make_pipeline(StandardScaler(), KernelRidge(kernel=ex_kernel, alpha=0.
    ↪1))
KRR_ex.fit(feat_train, target_train)
```

```
ex_pred_KRR = KRR_ex.predict(feat_test)

Rsquared_KRR_ex = KRR_ex.score(feat_train, target_train)
```

We use `ex_pred_KRR` in the following cell to provide the results. You can just execute the next cell.

```
[26]: target_test['KRR_predicted_quality'] = np.around(ex_pred_KRR, decimals=2)
target_test['KRR_absolut_deviation'] = abs(target_test['quality'] -
↳ target_test['KRR_predicted_quality'])
print(target_test)

KRR_pred_MSE = (target_test['KRR_absolut_deviation']**2).mean()
print(f"\nMean squared error of KernelRidge: {KRR_pred_MSE:.2f}")
```

	quality	KRR_predicted_quality	KRR_absolut_deviation	\
912	6	6.32	0.32	
772	5	4.87	0.13	
1037	5	5.04	0.04	
1106	6	6.45	0.45	
263	5	5.58	0.58	
...	...	...	...	
1466	7	5.60	1.40	
580	5	4.71	0.29	
1082	6	5.37	0.63	
1279	7	6.25	0.75	
1155	5	5.37	0.37	

	GPreg_predicted_quality	GPreg_absolut_deviation
912	6.31	0.31
772	4.86	0.14
1037	5.04	0.04
1106	6.44	0.44
263	5.58	0.58
...	...	...
1466	5.59	1.41
580	4.74	0.26
1082	5.36	0.64
1279	6.23	0.77
1155	5.38	0.38

[480 rows x 5 columns]

Mean squared error of KernelRidge: 0.38

(2.) Implement GP regression as we have seen it in this notebook. Please provide the predictions for the test set `feat_test` in `ex_pred_GPreg`.

Hints: If the computation might take too long, interrupt the execution with `/ Interrupt Kernel` in the panel above and set `n_restarts_optimizer=1`.


```
[27]: from sklearn.gaussian_process import GaussianProcessRegressor

GPreg_ex = make_pipeline(StandardScaler(),
    ↳ GaussianProcessRegressor(kernel=ex_kernel, alpha=0.15,
    ↳ n_restarts_optimizer=5))
GPreg_ex.fit(feet_train, target_train)
ex_pred_GPreg = GPreg_ex.predict(feet_test)

Rsquared_GPreg_ex = GPreg_ex.score(feet_train, target_train)
```

We use `ex_pred_GPreg` in the following cell to provide the results. You can just execute the next cell.

```
[28]: target_test['GPreg_predicted_quality'] = np.around(ex_pred_GPreg, decimals=2)
target_test['GPreg_absolut_deviation'] = abs(target_test['quality'] -
    ↳ target_test['GPreg_predicted_quality'])
print(target_test)

GPreg_pred_MSE = (target_test['GPreg_absolut_deviation']**2).mean()

print(f"\nMean squared error of GaussianProcessRegressor: {GPreg_pred_MSE:.2f}")
print(f"Mean squared error of KernelRidge: {KRR_pred_MSE:.2f}")
```

	quality	KRR_predicted_quality	KRR_absolut_deviation	\
912	6	6.32	0.32	
772	5	4.87	0.13	
1037	5	5.04	0.04	
1106	6	6.45	0.45	
263	5	5.58	0.58	
...	...	...	...	
1466	7	5.60	1.40	
580	5	4.71	0.29	
1082	6	5.37	0.63	
1279	7	6.25	0.75	
1155	5	5.37	0.37	

	GPreg_predicted_quality	GPreg_absolut_deviation
912	6.31	0.31
772	4.86	0.14
1037	5.04	0.04
1106	6.44	0.44
263	5.58	0.58
...	...	...
1466	5.59	1.41
580	4.74	0.26
1082	5.36	0.64
1279	6.23	0.77
1155	5.38	0.38

```
[480 rows x 5 columns]
```

```
Mean squared error of GaussianProcessRegressor: 0.38
```

```
Mean squared error of KernelRidge: 0.38
```

Class_6-Neural_Networks

May 20, 2026

1 6. Neural Networks

In the previous notebook, we have seen that we can choose feature mappings ϕ or kernels κ to design powerful non-linear models. Informally, deep learning can be characterised as trying to learn the feature mapping ϕ from data.

In this notebook we will discuss and implement our first

- feed-forward neural network / multi-layer perceptron and
- convolutional neural network

for the MNIST and wine quality classification tasks.

Keywords: Gradient descent, Backpropagation, Cross-entropy loss, ReLU, One-hot encoding, Max pooling, Convolution, `keras.layers`, `model.compile`, `model.fit`, `model.evaluate`, `model.save`, `model.summary`, `models.load_model`, `keras.optimizers.SGD`

1.1 Neural Network Basics

We start with the classical **Feed-Forward Neural Networks (FFNNs)** which are also referred to as multi-layer perceptrons (MLPs). FFNNs were originally proposed as a computational model simulating mechanisms of the human brain. The general idea is that inputs are processed in a sequential manner by **artificial neurons**. The common architecture consists of an input layer, one to several hidden layers, and an output layer, as illustrated in the following figure.

As before, we try to learn the function $f(X)$. The displayed three-layer FFNN **parameterises** this function in the following way

$$f(X, W) = \sigma(\sigma(XW_1)W_2)W_3 = \phi(X, W_1, W_2)W_3, \quad (1)$$

where weights $W = (W_1, W_2, W_3)$ are the matrices which we multiply layer-wise to the output of the previous layer. The outputs are called **activations**. An important part of why neural networks are so powerful is the use of **non-linear activation functions**. The **rectified linear unit (ReLU)** $\sigma(z) = \max\{0, z\}$ is a particularly popular choice, e.g. $\sigma(z = -2) = 0$ and $\sigma(z = 2) = 2$, with z being the preactivation.

We can make a connection between neural networks and what we have seen before. Let's consider a deeper L -layer FFNN. In the regression setting, our goal is again to find a model for

$$Y = f(X, W) + \epsilon = \phi(X, W_1, \dots, W_{L-1})W_L + \epsilon \quad (2)$$

where this time ϕ is learned! One can show that with this type of model we can in principle approximate any kind of (continuous and compact) function, i.e. neural networks are **universal function approximators**! We calibrate our model to solve the task by using **(Stochastic) Gradient Descent** for updating the weights $W = (W_1, \dots, W_L)$:

$$W^{(t+1)} = W^{(t)} - \eta \nabla \text{Loss}(W^{(t)}). \quad (3)$$

As illustrated in the figure below, the idea is to start at some position of our loss landscape described by initially random weights $W^{(t=0)}$ drawn from a Gaussian distribution. With every update step, we approach a (local) minimum until the weights no longer change. Note that this requires an appropriate choice of the step size or **learning rate** η (which again is a hyperparameter like λ and k which we've seen before).

As before, we can use the mean-squared error loss for regression tasks

$$\text{Loss}(W) = \frac{1}{2N} \|Y - f(X, W)\|_2^2 = \frac{1}{2N} \sum_{n=1}^N (y_n - f(x_n, W))^2. \quad (4)$$

In the classification setting, we use the multi-class version of the logistic loss encountered earlier. In deep learning context, we usually call it the **cross-entropy loss**

$$\text{Loss}(W) = - \sum_{n=1}^N \sum_{c=1}^C y_{n,c} \log p_{n,c} = - \sum_{n=1}^N \sum_{c=1}^C y_{n,c} \log f_c(x_n, W), \quad (5)$$

where we consider C different classes our N datapoints belong to. The neural network provides in the output layer the probabilities for the different classes $p_{n,c} = f_c(x_n, W)$. For this, we use another kind of activation function known as the **softmax activation function** which maps the activation to the values between 0 and 1. The activation $\sigma(z)_i$ at output neuron i is given by

$$\sigma(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^C \exp(z_j)}, \quad (6)$$

where C is the number of neurons in the output layer, i.e. the number of classes, and $i, j \in 1, \dots, C$. Typically, we change the class labels into a **one-hot encoding** which provides something like a class probability associated with the label. Suppose our datapoint x_n shows the digit 9 and the corresponding target is $y_n = 9$. In the one-hot encoding we replace y_n with the vector

$$y_n = \begin{pmatrix} y_{n,0} \\ y_{n,1} \\ \vdots \\ y_{n,9} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}, \quad (7)$$

where we have 0s at every index but the class index, i.e. 9 in this case.

The basic principle of how neural networks learn can be summarised as an iteration over the following steps:

- Forward pass: provide inputs and compute predictions
- Calculate loss: measure the error of predictions compared to expected output
- **Backpropagation**: compute gradients through the chain rule
- Gradient descent: updated weights
- Repeat until the loss does not change (too much) anymore

Let's use all this to build an FFNN to classify MNIST digits!

```
[1]: import numpy as np
      from tensorflow import keras

      mnist_data = np.load('data/mnist_data_5k.npy', allow_pickle=True)
      mnist_targets = np.load('data/mnist_labels_5k.npy', allow_pickle=True)
```

2026-05-18 11:36:58.057420: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE4.1 SSE4.2, in other operations, rebuild TensorFlow with the appropriate compiler flags.

In the following, we normalise the vector of pixel values to the $[0, 1]$ interval. It is typically advisable to scale all your predictors in X (i.e. the columns) such that values range between 0 and 1. Furthermore, we split the data into training, validation, and test datasets.

```
[2]: mnist_data_flat = mnist_data.reshape(-1, 784, 1) / 255

      num_train = 3000
      num_val = 1000
      num_test = 1000

      x_train = mnist_data_flat[:num_train]
      y_train = mnist_targets[:num_train]

      x_val = mnist_data_flat[num_train:num_train+num_val]
      y_val = mnist_targets[num_train:num_train+num_val]

      x_test = mnist_data_flat[-num_test:]
      y_test = mnist_targets[-num_test:]
```

Now, we define a three-layer FFNN with the following layers sizes:

- 784 neurons in the input layer
- 1024 (hidden) neurons in the first hidden layer with ReLU activation
- 512 (hidden) neurons in the second hidden layer with ReLU activation
- 10 output neurons for our classes in the output layer with a softmax activation

```
[3]: ffnn = keras.Sequential([
    keras.layers.InputLayer(input_shape=(784,)),
    keras.layers.Dense(units=1024, activation='relu'),
    keras.layers.Dense(units=512, activation='relu'),
    keras.layers.Dense(units=10, activation='softmax')
], name='FFNN_example')
```

Next, we specify hyperparameters. As outlined before, the **learning rate** η is an important hyperparameter. In addition, we typically cannot (but also do not want to) use all datapoints for training at the same time. We choose small batches of the dataset and thus need to specify the **batch size**.

```
[4]: learning_rate = 0.01
    batch_size = 16
    num_epochs = 10

    print(f"Shape of training set: {x_train.shape}")
    print(f"Number of batches: {x_train.shape[0] / batch_size}")
```

Shape of training set: (3000, 784, 1)

Number of batches: 187.5

A training **epoch** is concluded after we have seen the total number of batches. We set multiple epochs to iteratively refine the neural network weights, i.e. training samples are seen several times during training.

Now, let's define our optimisation method where we use [Stochastic Gradient Descent](#).

```
[5]: keras.utils.set_random_seed(123)

    ffnn.compile(optimizer=keras.optimizers.SGD(learning_rate=learning_rate),
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
```

We use as the loss the `sparse_categorical_crossentropy` where

- *categorical* means that the neural network outputs are expected to be softmax values between 0 and 1
- *sparse* indicates that we have did not one-hot encode our y-targets; the loss will take care of that

```
[6]: print(y_train)
```

```
[5 0 4 ... 9 1 5]
```

We can now use the methods `fit` and `evaluate` to train our FFNN and evaluate it on the test set.

```
[7]: ffnn_history = ffnn.fit(x=x_train, y=y_train, batch_size=batch_size,
                             validation_data=(x_val, y_val), epochs=num_epochs)

    test_loss, test_acc = ffnn.evaluate(x_test, y_test, verbose=0)
```

```
print(f'\nTest accuracy: {test_acc:.4f}')
```

```
ffnn.save('output/MNIST_FFNN.h5')
```

```
Epoch 1/10
188/188 [=====] - 1s 4ms/step - loss: 1.5439 -
accuracy: 0.6567 - val_loss: 0.9696 - val_accuracy: 0.8100
Epoch 2/10
188/188 [=====] - 1s 3ms/step - loss: 0.6926 -
accuracy: 0.8480 - val_loss: 0.6111 - val_accuracy: 0.8660
Epoch 3/10
188/188 [=====] - 1s 3ms/step - loss: 0.4796 -
accuracy: 0.8807 - val_loss: 0.4748 - val_accuracy: 0.8840
Epoch 4/10
188/188 [=====] - 1s 3ms/step - loss: 0.3894 -
accuracy: 0.8990 - val_loss: 0.4189 - val_accuracy: 0.8980
Epoch 5/10
188/188 [=====] - 1s 3ms/step - loss: 0.3399 -
accuracy: 0.9123 - val_loss: 0.3715 - val_accuracy: 0.9100
Epoch 6/10
188/188 [=====] - 1s 3ms/step - loss: 0.3037 -
accuracy: 0.9223 - val_loss: 0.3554 - val_accuracy: 0.9040
Epoch 7/10
188/188 [=====] - 1s 3ms/step - loss: 0.2751 -
accuracy: 0.9307 - val_loss: 0.3435 - val_accuracy: 0.9110
Epoch 8/10
188/188 [=====] - 1s 3ms/step - loss: 0.2518 -
accuracy: 0.9347 - val_loss: 0.3110 - val_accuracy: 0.9180
Epoch 9/10
188/188 [=====] - 1s 3ms/step - loss: 0.2331 -
accuracy: 0.9363 - val_loss: 0.3088 - val_accuracy: 0.9190
Epoch 10/10
188/188 [=====] - 1s 3ms/step - loss: 0.2158 -
accuracy: 0.9427 - val_loss: 0.2946 - val_accuracy: 0.9270
```

Test accuracy: 0.9100

We can summarise the network parameters:

```
[8]: ffnn.summary()
```

Model: "FFNN_example"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 1024)	803840
dense_1 (Dense)	(None, 512)	524800

dense_2 (Dense) (None, 10) 5130

```
=====
Total params: 1,333,770
Trainable params: 1,333,770
Non-trainable params: 0
-----
```

Let's see how the accuracy and loss developed over training making use of the `ffnn_history` object we assigned after training.

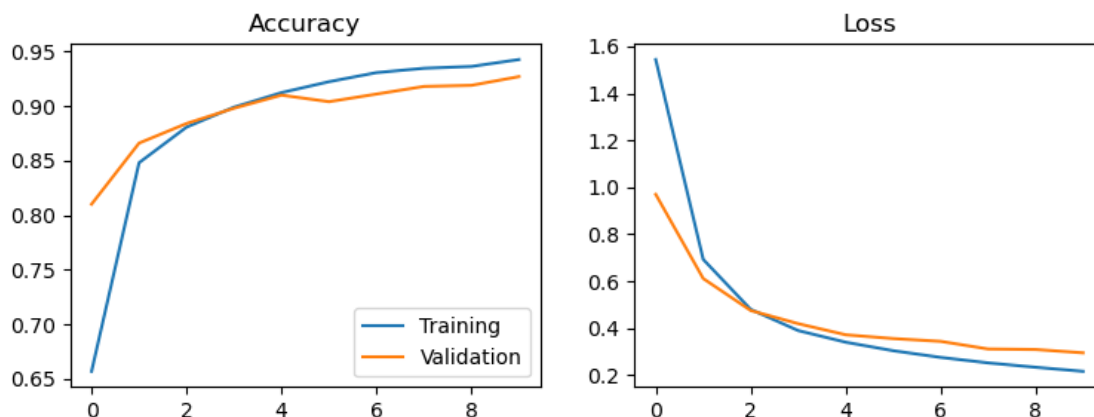
```
[9]: import matplotlib.pyplot as plt

fig, axs = plt.subplots(1,2, figsize=(9,3))

axs[0].plot(ffnn_history.history['accuracy'], label='Training')
axs[0].plot(ffnn_history.history['val_accuracy'], label='Validation')
axs[0].set_title('Accuracy')
axs[0].legend()

axs[1].plot(ffnn_history.history['loss'])
axs[1].plot(ffnn_history.history['val_loss'])
axs[1].set_title('Loss')

plt.show()
```



We use the `predict` method to provide predictions to new data. You can use the model you just trained or you can load a previously **pretrained** model.

```
[10]: # Our pretrained model (100 epochs):
ffnn = keras.models.load_model('output/MNIST_FFNN_pretrained.h5')
```



```

# ... or load the model you trained yourself:
# ffnn = keras.models.load_model('output/MNIST_FFNN.h5')

test_loss, test_acc = ffnn.evaluate(x_test, y_test, verbose=0)

print(f'Test accuracy: {test_acc:.4f}\n')

num_examples = 10

predictions_1hot = ffnn.predict(x_test, verbose=0)

# print(predictions_1hot[0])

nicer_predictions_1hot_0 = list(map(np.format_float_positional,
    ↪ predictions_1hot[0], [5]*10))

print(f"Shape of the predictions: {predictions_1hot.shape}\n")
print(f"First prediction: {nicer_predictions_1hot_0}\n")

predictions = np.argmax(predictions_1hot, axis=1)

print(f"First class prediction: {predictions[0]}")

```

Test accuracy: 0.9260

Shape of the predictions: (1000, 10)

First prediction: ['0.00000', '0.00000', '0.00004', '0.00043', '0.00000',
'0.00000', '0.00000', '0.99954', '0.00000', '0.00000']

First class prediction: 7

Note that the neural network outputs vectors with class probabilities. We choose the class with the highest probability.

Let's plot some of the predicted labels.

```

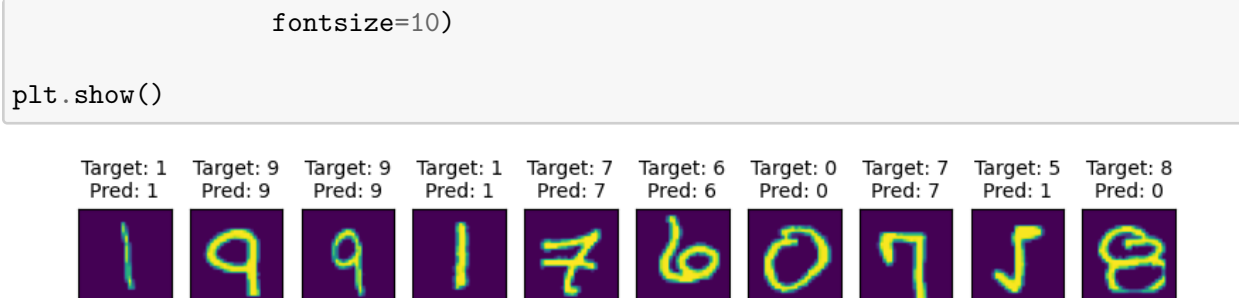
[11]: fig, axs = plt.subplots(1, num_examples, figsize=(num_examples, 5))

start_index = 900

for i, ax in enumerate(axs.ravel()):
    ax.imshow(x_test[start_index+i].reshape(28, 28))

    ax.set_xticks([])
    ax.set_yticks([])
    ax.set_title(f"Target: {y_test[start_index+i]}\nPred: ↪
    ↪ {predictions[start_index+i]}")

```



1.2 Convolutional Neural Networks

An important class of neural network architectures are **convolutional neural networks (CNNs)**. CNNs are inspired by biological models for the primary visual cortex and their resemblance to the mathematical convolution operation. CNNs can be viewed as a particular kind of FFNNs where certain connections are missing and some neurons share the same weights. This gives rise to architectures as illustrated in the following figure.

CNNs are particularly suited to work with images and generally data **where some kind of local relationship / correlation** are present, e.g. adjacent pixels in an image or adjacent time steps in time series data. Two key components of CNNs are pooling and convolutional layers illustrated in the following.

Convolutional layers use a **convolutional filter** (sometimes called *kernel*, too) which processes the input images with respect to particular features (like edges) producing **feature maps**. Sequentially applying convolutional layers allows for **identifying increasingly complex features**. A typical size of a convolutional filter is 3×3 which can be thought of as a matrix of weights (in red above) which are adjusted during training to identify features in a data-driven fashion.

Pooling layers reduce the resolution of feature maps and don't have any associated weights. In particular, max pooling layers of size 2×2 are used to identify the main activation in adjacent pixels. This allows objects to be in different positions in the image but still be recognised as the same object.

Let's implement such a CNN and reshape the image vectors to actual images again.

```

[12]: mnist_data_image = mnist_data.reshape(-1, 28, 28, 1) / 255

x_train = mnist_data_image[:num_train]
x_val = mnist_data_image[num_train:num_train+num_val]
x_test = mnist_data_image[-num_test:]

```

We define a CNN with the following layers and train as before:

- $28 \times 28 \times 1$ neurons in the input layer for the greyscale image
- 6 conv. filters of size 3×3 in the first hidden layer with ReLU activation followed by a 2×2 max pooling

- 16 conv. filters of size 3×3 in the second hidden layer with ReLU activation followed by a 2×2 max pooling
- flatten these feature maps into a single vector
- 1024 (hidden) neurons in the third hidden layer with ReLU activation
- 512 (hidden) neurons in the fourth hidden layer with ReLU activation
- 10 output neurons for our classes in the output layer with a softmax activation

```
[13]: keras.utils.set_random_seed(123)

cnn = keras.Sequential([
    keras.layers.InputLayer(input_shape=(28, 28, 1)),
    keras.layers.Conv2D(filters=6, kernel_size=(3, 3), activation='relu'),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    keras.layers.Conv2D(filters=16, kernel_size=(3, 3), activation='relu'),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    keras.layers.Flatten(),
    keras.layers.Dense(units=1024, activation='relu'),
    keras.layers.Dense(units=512, activation='relu'),
    keras.layers.Dense(units=10, activation='softmax')
], name='CNN_example')

cnn.compile(optimizer=keras.optimizers.SGD(learning_rate=learning_rate),
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])

cnn_history = cnn.fit(x=x_train, y=y_train, batch_size=batch_size,
                    validation_data=(x_val, y_val), epochs=num_epochs)

test_loss, test_acc = cnn.evaluate(x_test, y_test, verbose=0)

print(f'\nTest accuracy: {test_acc:.4f}\n')

cnn.save('output/MNIST_CNN.h5')

cnn.summary()
```

Epoch 1/10

188/188 [=====] - 1s 5ms/step - loss: 2.0536 - accuracy: 0.4477 - val_loss: 1.3182 - val_accuracy: 0.6740

Epoch 2/10

188/188 [=====] - 1s 5ms/step - loss: 0.7038 - accuracy: 0.8103 - val_loss: 0.6041 - val_accuracy: 0.8130

Epoch 3/10

188/188 [=====] - 1s 5ms/step - loss: 0.4115 - accuracy: 0.8753 - val_loss: 0.3994 - val_accuracy: 0.8690

Epoch 4/10

188/188 [=====] - 1s 5ms/step - loss: 0.3023 - accuracy: 0.9117 - val_loss: 0.3031 - val_accuracy: 0.8970

```

Epoch 5/10
188/188 [=====] - 1s 5ms/step - loss: 0.2517 -
accuracy: 0.9270 - val_loss: 0.2094 - val_accuracy: 0.9470
Epoch 6/10
188/188 [=====] - 1s 5ms/step - loss: 0.2132 -
accuracy: 0.9387 - val_loss: 0.2146 - val_accuracy: 0.9420
Epoch 7/10
188/188 [=====] - 1s 5ms/step - loss: 0.1818 -
accuracy: 0.9453 - val_loss: 0.2419 - val_accuracy: 0.9280
Epoch 8/10
188/188 [=====] - 1s 5ms/step - loss: 0.1586 -
accuracy: 0.9543 - val_loss: 0.1588 - val_accuracy: 0.9550
Epoch 9/10
188/188 [=====] - 1s 5ms/step - loss: 0.1421 -
accuracy: 0.9587 - val_loss: 0.1701 - val_accuracy: 0.9500
Epoch 10/10
188/188 [=====] - 1s 5ms/step - loss: 0.1206 -
accuracy: 0.9673 - val_loss: 0.1448 - val_accuracy: 0.9580

```

Test accuracy: 0.9360

Model: "CNN_example"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 6)	60
max_pooling2d (MaxPooling2D)	(None, 13, 13, 6)	0
conv2d_1 (Conv2D)	(None, 11, 11, 16)	880
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 16)	0
flatten (Flatten)	(None, 400)	0
dense_3 (Dense)	(None, 1024)	410624
dense_4 (Dense)	(None, 512)	524800
dense_5 (Dense)	(None, 10)	5130

Total params: 941,494
 Trainable params: 941,494
 Non-trainable params: 0

Let's study the performance of the CNN.

```
[14]: import matplotlib.pyplot as plt

fig, axs = plt.subplots(1,2, figsize=(9,3))

axs[0].plot(cnn_history.history['accuracy'], label='Training')
axs[0].plot(cnn_history.history['val_accuracy'], label='Validation')
axs[0].set_title('Accuracy')
axs[0].legend()

axs[1].plot(cnn_history.history['loss'])
axs[1].plot(cnn_history.history['val_loss'])
axs[1].set_title('Loss')

plt.show()

# Our pretrained model (100 epochs):
cnn = keras.models.load_model('output/MNIST_CNN_pretrained.h5')

# ... or load your model:
# cnn = keras.models.load_model('output/MNIST_CNN.h5')

test_loss, test_acc = cnn.evaluate(x_test, y_test, verbose=0)

print(f'Test accuracy: {test_acc:.4f}\n')

num_examples = 10

predictions_1hot = cnn.predict(x_test, verbose=0)
predictions = np.argmax(predictions_1hot, axis=1)

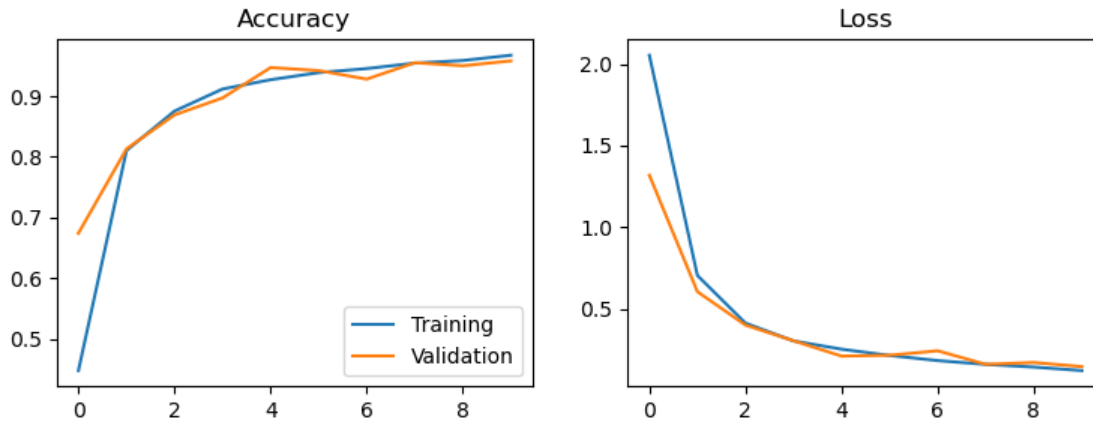
fig, axs = plt.subplots(1, num_examples, figsize=(num_examples,5))

start_index = 900

for i, ax in enumerate(axs.ravel()):
    ax.imshow(x_test[start_index+i])

    ax.set_xticks([])
    ax.set_yticks([])
    ax.set_title(f"Target: {y_test[start_index+i]}\nPred: ⬇
    ↪{predictions[start_index+i]}",
                fontsize=10)

plt.show()
```



Test accuracy: 0.9560



1.2.1 How to tune hyperparameters?

As mentioned in [Notebook 2](#), neural network performance is sensitive to the hyperparameter choice. Again, there are tools to perform hyperparameter search in a more automated manner like:

1. TensorFlow / Keras: [Keras Tuner](#)
2. PyTorch: [Optuna](#)

1.3 Exercise Section

For a last time, we consider the red wine dataset with our new regression method. As before, we want to predict the `quality` values on the test set `feat_test` and compare it to the previous results:

- ridge regression MSE 0.42
- Random Forest regression MSE 0.34
- kernel ridge regression MSE 0.38
- Gaussian Process regression MSE 0.38

Prepare the data by load the following cell:

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from tensorflow import keras

np.random.seed(123)
keras.utils.set_random_seed(123)

wine_data = pd.read_csv('data/winequality-red.csv', sep=';')

ex_target = wine_data['quality']
ex_features = wine_data.drop(['quality'], axis=1)

ex_features = (ex_features - ex_features.min()) / (ex_features.max() -
↳ex_features.min())

feat_train, feat_test, target_train, target_test = train_test_split(
    ex_features, ex_target, test_size = 0.3, random_state=123)

target_test = target_test.to_frame()
```

(1.) Implement a feed-forward neural network with the following number of artificial neurons in the hidden layers: 1024, 512, 256, 128, 64. Use ReLU activations and the following hyperparameters:

```
[ ]: learning_rate = 0.005
batch_size = 64
num_epochs = 50
```

Put your result in the following cell and note:

- Provide the model as `ffnn_reg`, the history of the training in `ffnn_reg_history`, and the predicted quality values in `ex_pred_ffnn`.
- The output in the regression setting is just 1-dimensional and no activation is required.
- Use as the MSE loss `keras.losses.MeanSquaredError()` and do not use the `metrics` argument.
- You can use the test set for `validation_data`.

```
[ ]: # fill in
```

Try to check the accuracy and loss during training with the following plot.

```
[ ]: plt.plot(ffnn_reg_history.history['loss'][1:], label='Training')
plt.plot(ffnn_reg_history.history['val_loss'][1:], label='Test')
plt.title('Loss')
plt.legend()
```

```
plt.show()
```

As we have done earlier, the following cell provides a data frame with an overview of the predictions.

```
[ ]: target_test['NN_predicted_quality'] = np.around(ex_pred_ffnn, decimals=2)
target_test['NN_absolut_deviation'] = np.around(abs(target_test['quality'] -
    ↪target_test['NN_predicted_quality']), decimals=2)
print(target_test)

NN_pred_MSE = (target_test['NN_absolut_deviation']**2).mean()
print(f"\nMean squared error of Feedforward Neural Network regression:
    ↪{NN_pred_MSE:.2f}")
```

1.4 Proposed Solutions

For a final time, we consider the red wine dataset with our new regression method. As before, we want to predict the **quality** values on the test set **feat_test** and compare it to the previous results:

- ridge regression MSE 0.42
- Random Forest regression MSE 0.34
- kernel ridge regression MSE 0.38
- Gaussian Process regression MSE 0.38

Prepare the data by loading the following cell:

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from tensorflow import keras

np.random.seed(123)
keras.utils.set_random_seed(123)

wine_data = pd.read_csv('data/winequality-red.csv', sep=';')

ex_target = wine_data['quality']
ex_features = wine_data.drop(['quality'], axis=1)

ex_features = (ex_features - ex_features.min()) / (ex_features.max() -
    ↪ex_features.min())

feat_train, feat_test, target_train, target_test = train_test_split(
    ex_features, ex_target, test_size = 0.3, random_state=123)
```



```
target_test = target_test.to_frame()
```

2023-03-21 12:53:11.016835: I tensorflow/core/platform/cpu_feature_guard.cc:193] This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critical operations: SSE4.1 SSE4.2 AVX AVX2 FMA To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.

(1.) Implement a feed-forward neural network with the following number of artificial neurons in the hidden layers: 1024, 512, 256, 128, 64. Use ReLU activations and the following hyperparameters:

```
[2]: learning_rate = 0.005
      batch_size = 64
      num_epochs = 50
```

Put your result in the following cell and note:

- Provide the model as `ffnn_reg`, the history of the training in `ffnn_reg_history`, and the predicted quality values in `ex_pred_ffnn`.
- The output in the regression setting is just 1-dimensional and no activation is required.
- Use as the MSE loss `keras.losses.MeanSquaredError()` and do not use the `metrics` argument.
- You can use the test set for `validation_data`.

```
[3]: ffnn_reg = keras.Sequential(
      [
        keras.layers.Dense(1024, activation='relu', input_shape=(11,)),
        keras.layers.Dense(512, activation='relu'),
        keras.layers.Dense(256, activation='relu'),
        keras.layers.Dense(128, activation='relu'),
        keras.layers.Dense(64, activation='relu'),
        keras.layers.Dense(1)
      ],
      name='FFNN_regression')

ffnn_reg.compile(optimizer=keras.optimizers.SGD(learning_rate=learning_rate),
                 loss=keras.losses.MeanSquaredError())

ffnn_reg_history = ffnn_reg.fit(x=feat_train, y=target_train,
                               validation_data=(feat_test, target_test),
                               batch_size=batch_size, epochs=num_epochs)

ex_pred_ffnn = ffnn_reg.predict(feat_test, verbose=0)

ffnn_reg.save(f'output/MNIST_FFNN_reg.h5')
```

Epoch 1/50

2023-03-21 12:53:13.838010: I tensorflow/core/platform/cpu_feature_guard.cc:193] This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critical operations: SSE4.1 SSE4.2 AVX AVX2 FMA To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.

```
18/18 [=====] - 1s 15ms/step - loss: 17.3084 -
val_loss: 0.6507
Epoch 2/50
18/18 [=====] - 0s 7ms/step - loss: 0.6783 - val_loss:
0.6384
Epoch 3/50
18/18 [=====] - 0s 7ms/step - loss: 0.6341 - val_loss:
0.5898
Epoch 4/50
18/18 [=====] - 0s 7ms/step - loss: 0.6149 - val_loss:
0.5527
Epoch 5/50
18/18 [=====] - 0s 9ms/step - loss: 0.5946 - val_loss:
0.6049
Epoch 6/50
18/18 [=====] - 0s 7ms/step - loss: 0.5648 - val_loss:
0.5109
Epoch 7/50
18/18 [=====] - 0s 8ms/step - loss: 0.5551 - val_loss:
0.5547
Epoch 8/50
18/18 [=====] - 0s 9ms/step - loss: 0.5414 - val_loss:
0.4864
Epoch 9/50
18/18 [=====] - 0s 7ms/step - loss: 0.5218 - val_loss:
0.4878
Epoch 10/50
18/18 [=====] - 0s 8ms/step - loss: 0.4996 - val_loss:
0.4796
Epoch 11/50
18/18 [=====] - 0s 8ms/step - loss: 0.4966 - val_loss:
0.4754
Epoch 12/50
18/18 [=====] - 0s 7ms/step - loss: 0.4806 - val_loss:
0.4774
Epoch 13/50
18/18 [=====] - 0s 8ms/step - loss: 0.4737 - val_loss:
0.4592
Epoch 14/50
18/18 [=====] - 0s 7ms/step - loss: 0.4678 - val_loss:
0.4589
```

Epoch 15/50
18/18 [=====] - 0s 7ms/step - loss: 0.4623 - val_loss: 0.4516
Epoch 16/50
18/18 [=====] - 0s 7ms/step - loss: 0.4580 - val_loss: 0.4459
Epoch 17/50
18/18 [=====] - 0s 7ms/step - loss: 0.4597 - val_loss: 0.4725
Epoch 18/50
18/18 [=====] - 0s 7ms/step - loss: 0.4566 - val_loss: 0.4398
Epoch 19/50
18/18 [=====] - 0s 9ms/step - loss: 0.4522 - val_loss: 0.4500
Epoch 20/50
18/18 [=====] - 0s 7ms/step - loss: 0.4472 - val_loss: 0.4411
Epoch 21/50
18/18 [=====] - 0s 8ms/step - loss: 0.4493 - val_loss: 0.4379
Epoch 22/50
18/18 [=====] - 0s 7ms/step - loss: 0.4452 - val_loss: 0.4434
Epoch 23/50
18/18 [=====] - 0s 9ms/step - loss: 0.4521 - val_loss: 0.4310
Epoch 24/50
18/18 [=====] - 0s 7ms/step - loss: 0.4452 - val_loss: 0.4322
Epoch 25/50
18/18 [=====] - 0s 7ms/step - loss: 0.4375 - val_loss: 0.4297
Epoch 26/50
18/18 [=====] - 0s 8ms/step - loss: 0.4407 - val_loss: 0.4349
Epoch 27/50
18/18 [=====] - 0s 7ms/step - loss: 0.4442 - val_loss: 0.4442
Epoch 28/50
18/18 [=====] - 0s 8ms/step - loss: 0.4392 - val_loss: 0.4277
Epoch 29/50
18/18 [=====] - 0s 9ms/step - loss: 0.4327 - val_loss: 0.4261
Epoch 30/50
18/18 [=====] - 0s 7ms/step - loss: 0.4319 - val_loss: 0.4344

Epoch 31/50
18/18 [=====] - 0s 8ms/step - loss: 0.4332 - val_loss: 0.4253
Epoch 32/50
18/18 [=====] - 0s 8ms/step - loss: 0.4410 - val_loss: 0.4346
Epoch 33/50
18/18 [=====] - 0s 7ms/step - loss: 0.4311 - val_loss: 0.4234
Epoch 34/50
18/18 [=====] - 0s 8ms/step - loss: 0.4352 - val_loss: 0.4254
Epoch 35/50
18/18 [=====] - 0s 8ms/step - loss: 0.4307 - val_loss: 0.4209
Epoch 36/50
18/18 [=====] - 0s 7ms/step - loss: 0.4306 - val_loss: 0.4193
Epoch 37/50
18/18 [=====] - 0s 9ms/step - loss: 0.4286 - val_loss: 0.4357
Epoch 38/50
18/18 [=====] - 0s 7ms/step - loss: 0.4274 - val_loss: 0.4181
Epoch 39/50
18/18 [=====] - 0s 9ms/step - loss: 0.4250 - val_loss: 0.4379
Epoch 40/50
18/18 [=====] - 0s 8ms/step - loss: 0.4273 - val_loss: 0.4373
Epoch 41/50
18/18 [=====] - 0s 7ms/step - loss: 0.4255 - val_loss: 0.4215
Epoch 42/50
18/18 [=====] - 0s 7ms/step - loss: 0.4221 - val_loss: 0.4426
Epoch 43/50
18/18 [=====] - 0s 8ms/step - loss: 0.4222 - val_loss: 0.4574
Epoch 44/50
18/18 [=====] - 0s 9ms/step - loss: 0.4264 - val_loss: 0.4336
Epoch 45/50
18/18 [=====] - 0s 8ms/step - loss: 0.4211 - val_loss: 0.4274
Epoch 46/50
18/18 [=====] - 0s 7ms/step - loss: 0.4182 - val_loss: 0.4171

```

Epoch 47/50
18/18 [=====] - 0s 7ms/step - loss: 0.4186 - val_loss:
0.4361
Epoch 48/50
18/18 [=====] - 0s 7ms/step - loss: 0.4185 - val_loss:
0.4233
Epoch 49/50
18/18 [=====] - 0s 9ms/step - loss: 0.4229 - val_loss:
0.4288
Epoch 50/50
18/18 [=====] - 0s 7ms/step - loss: 0.4174 - val_loss:
0.4273

```

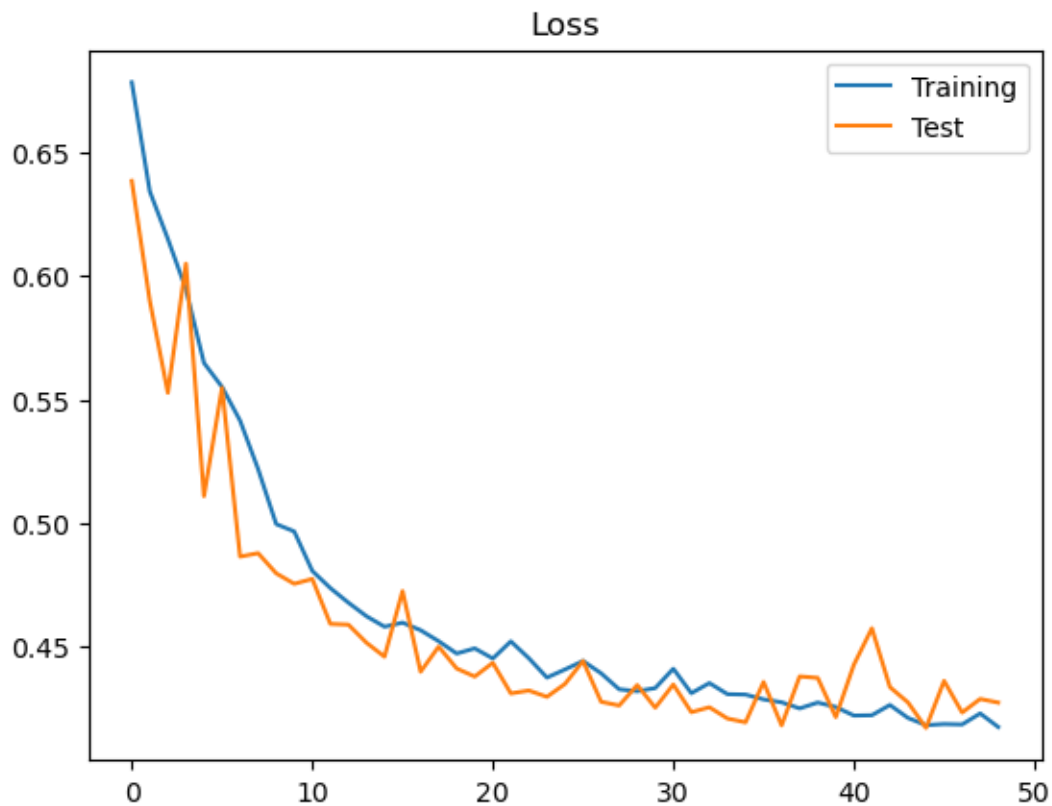
Try to check the accuracy and loss during training with the following plot.

```

[4]: plt.plot(ffnn_reg_history.history['loss'][1:], label='Training')
plt.plot(ffnn_reg_history.history['val_loss'][1:], label='Test')
plt.title('Loss')
plt.legend()

plt.show()

```



As we have done earlier, the following cell provides a data frame with an overview of the predictions.

```
[5]: # Pretrained for 1000 epochs
#ffnn_reg = keras.models.load_model('output/MNIST_FFNN_reg_pretrained.h5')
#ex_pred_ffnn = ffnn_reg.predict(feet_test, verbose=0)

[6]: target_test['NN_predicted_quality'] = np.around(ex_pred_ffnn, decimals=2)
target_test['NN_absolut_deviation'] = np.around(abs(target_test['quality'] -
    ↪target_test['NN_predicted_quality']), decimals=2)
print(target_test)

NN_pred_MSE = (target_test['NN_absolut_deviation']**2).mean()
print(f"\nMean squared error of Feedforward Neural Network regression:
    ↪{NN_pred_MSE:.2f}")
```

	quality	NN_predicted_quality	NN_absolut_deviation
912	6	6.52	0.52
772	5	5.09	0.09
1037	5	4.91	0.09
1106	6	6.60	0.60
263	5	5.51	0.51
...	...	...	...
1466	7	5.61	1.39
580	5	5.44	0.44
1082	6	5.47	0.53
1279	7	6.35	0.65
1155	5	5.34	0.34

[480 rows x 3 columns]

Mean squared error of Feedforward Neural Network regression: 0.43

Class_7-VAE

May 20, 2026

1 7. Variational Autoencoder

We have seen approaches for compression previously. In this notebook, we cover a related approach based on neural networks which also enables us to generate novel (artificial) data.

In particular, we discuss

- latent variable models and
- the Variational Autoencoder

and provide a more advanced CNN implementation for MNIST and a molecule dataset.

Keywords: VAE, ELBO, reparameterisation trick, keras.Model, keras.layers.Conv2DTranspose, tensorflow.GradientTape, tf.sigmoid, model.save_weights, model.load_weights, keras.optimizers.Adam

1.1 Latent Variable Models

Latent variable models (LVMs) are statistical models which consider random variables. Suppose that our observations X and Y are random variables following a probability distribution $p(X, Y)$. LVMs assume that there are **latent, i.e. unobserved or hidden, variables** Z which have a relationship to our observed data X and Y . A typical assumption, for instance, is that our predictors X are generated by **lower-dimensional latent variables** Z , e.g. $p(X | Z)$.

Note that we have seen models such as these already. For example, in a **Gaussian Mixture Model** datapoints X_i belonging to cluster i with mean μ_i and covariance matrix Σ_i are generated by sampling from $\mathcal{N}(\mu_i, \Sigma_i)$. In other words, the low-dimensional latent variables are $Z = (\mu, \Sigma)$. **PCA** is another example where few principal components can allow describing the majority of variation in a dataset. However, these **linear LVMs** assume linear relationships between latent variable Z and the observations.

Due to the highly non-linear nature of neural networks, we can use neural networks to extend models such as PCA to **non-linear LVMs**. Here, we pick up the previously encountered idea of compression:

The **Variational Autoencoder (VAE)** is an unsupervised neural network approach which attempts to identify a meaningful **latent representation** Z capable of reconstructing the original input as perfectly as possible. The VAE can be viewed as a probabilistic non-linear PCA, with the general idea being illustrated in the following figure.

The VAE has three important components:

1. **Encoder:** The encoder neural network attempts to compress the relevant information about X in latent representation Z with dimensions $d_X > d_Z$. The goal of the encoder is to parameterise a distribution $q_\phi(Z | X)$ which provides us a (variational) approximation for the distribution of the (compressed) latent variables Z given an input X . Here, ϕ denotes the weights of the encoder network. We typically assume that this approximation is given by a Gaussian distribution $\mathcal{N}(\mu_Z, \Sigma_Z)$ where $\mu_Z = (\mu_1, \dots, \mu_{d_Z})$ and $\Sigma_Z = (\sigma_1, \dots, \sigma_{d_Z})$ containing only the diagonal entries of covariance matrix Σ_Z (all other entries are 0). For technical reasons (details omitted), we require the **reparameterisation trick** to sample latent variables Z :

$$z_n(\varepsilon) = \mu_Z(x_n) + \Sigma_Z(x_n)\varepsilon$$

with $\varepsilon \sim \mathcal{N}(0, 1)$.

2. **Latent representation / space Z :** Datapoints from the input space are mapped into the lower-dimensional latent space Z . To start with, we typically assume that the latent variables Z are normally distributed, i.e. a prior distribution $p(Z) = \mathcal{N}(0, \mathbb{1}_{d_Z})$ with d_Z being the number of latent dimensions. We train the encoder such that the (posterior) distribution $q_\phi(Z | X)$ matches the prior $p(Z)$ as much as possible. The deviations from the prior distribution will capture important features of our dataset. We measure the difference in these distributions with the (non-negative) [Kullback-Leibler divergence](#)

$$D_{\text{KL}}(q_\phi(Z | X) \| p(Z)) = -\frac{1}{2} \sum_{i=1}^{d_Z} (1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2),$$

where the last expression follows from the Gaussian distribution assumptions outlined before.

3. **Decoder:** The decoder neural network attempts to reconstruct the original input X from the compressed representation Z . The goal of the decoder is to parameterise the likelihood $p_\theta(X | Z)$ with decoder network weights θ . The **reconstruction error** is measured by the log-likelihood $\log(p_\theta(X | Z))$ which in the case of regression tasks is typically given by the mean-squared error loss and in classification tasks by the cross-entropy loss.

Combining these ingredients, we obtain the VAE loss function which is usually referred to as negative **evidence lower bound (ELBO)**

$$\text{Loss}(\phi, \theta) = \frac{1}{s} \sum_{i=1}^s \underbrace{\mathbb{E}_{q_\phi(z|x_i)}(-\log(p_\theta(x_i|z)))}_{\text{reconstruction loss}} + \underbrace{D_{\text{KL}}(q_\phi(z|x_i) \| p(z))}_{\text{latent loss}} \quad (1)$$

where s is the batch size (denoting that we sum the results for different batches). Note that the VAE objective has two parts which we try to balance:

1. The **reconstruction loss** measures how well we can reconstruct the input X from the compression Z . In other words, this term is an incentive to provide as much information of X in Z as possible, i.e. less compression.
2. The **latent loss** measures how well the posterior $q_\phi(Z | X)$ agrees with the prior $p(Z)$. This term favours stronger compressions.

With this probabilistic framework, we can now explore the latent space Z and, in particular, pick latent points in Z to generate novel artificial objects by decoding the picked latent point!

Let's implement and apply a VAE on MNIST! For this, we first define a class for the model which provides the individual parts as outlined above.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras

class VAE(keras.Model):

    def __init__(self, latent_dim, batch_size):
        super(VAE, self).__init__()

        self.latent_dim = latent_dim
        self.batch_size = batch_size

        # Encoder network:
        self.encoder = keras.Sequential(
            [
                keras.layers.InputLayer(input_shape=(28, 28, 1)),
                # using tuples to indicate kernel size and strides
                keras.layers.Conv2D(filters=32, kernel_size=(3,3), strides=(2,2), activation='relu'),
                # using just an integer to indicate kernel size and strides,
                # but doing the same
                keras.layers.Conv2D(filters=64, kernel_size=3, strides=2, activation='relu'),
                keras.layers.Flatten(),
                keras.layers.Dense(latent_dim + latent_dim),
            ]
        )

        # Decoder network:
        self.decoder = keras.Sequential(
            [
                keras.layers.InputLayer(input_shape=(latent_dim,)),
                keras.layers.Dense(units=7 * 7 * 32, activation='relu'),
                keras.layers.Reshape(target_shape=(7, 7, 32)),
                # 7x7 x32
                keras.layers.Conv2DTranspose(filters=32, kernel_size=3,
                strides=2, padding='same', activation='relu'),
                # 14x14 x64
                keras.layers.Conv2DTranspose(filters=64, kernel_size=3,
                strides=2, padding='same', activation='relu'),
                # 28x28 x1
                keras.layers.Conv2DTranspose(filters=1, kernel_size=3,
                strides=1, padding='same'),
```

```

    ]
)

def encode(self, x):
    mean, logvar = tf.split(self.encoder(x), num_or_size_splits=2, axis=1)
    return mean, logvar

def decode(self, z, apply_sigmoid=False):
    logits = self.decoder(z)
    if apply_sigmoid:
        probs = tf.sigmoid(logits)
        return probs
    return logits

@tf.function
def sample(self, eps=None):
    if eps is None:
        eps = tf.random.normal(shape=(self.batch_size, self.latent_dim))
    return self.decode(eps, apply_sigmoid=True)

# Reparameterisation trick:
def reparameterize(self, mean, logvar):
    eps = tf.random.normal(shape=(self.batch_size, self.latent_dim))
    return eps * tf.exp(logvar * .5) + mean

def call(self, x):
    mean, logvar = tf.split(self.encoder(x), num_or_size_splits=2, axis=1)
    z = self.reparameterize(mean, logvar)
    return self.decoder(z), mean, logvar

```

2026-05-19 09:33:57.909700: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE4.1 SSE4.2, in other operations, rebuild TensorFlow with the appropriate compiler flags.

Note the effect of **padding** in the figure below (left: padding='valid'; right: padding='same'):

Furthermore, we define some functions we will use for computing the loss and the training steps. Note that in the previous notebook we used some built-in keras functions for that.

```
[2]: def log_normal_pdf(sample, mean, logvar, raxis=1):
      log2pi = tf.math.log(2.0 * np.pi)
```

```

    return tf.reduce_sum(-0.5 * ((sample - mean) ** 2.0 * tf.exp(-logvar) +
↪logvar + log2pi), axis=raxis)

def compute_loss(model, x):
    mean, logvar = model.encode(x)
    z = model.reparameterize(mean, logvar)
    x_logit = model.decode(z)
    cross_ent = tf.nn.sigmoid_cross_entropy_with_logits(logits=x_logit,
↪labels=x)
    logpx_z = -tf.reduce_sum(cross_ent, axis=[1, 2, 3])
    logpz = log_normal_pdf(z, 0., 0.)
    logqz_x = log_normal_pdf(z, mean, logvar)

    return -tf.reduce_mean(logpx_z + logpz - logqz_x)

@tf.function
def train_step(model, x, optimizer):

    with tf.GradientTape() as tape:
        loss = compute_loss(model, x)

    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

```

Now, we can load MNIST as before.

```

[3]: mnist_data = np.load('data/mnist_data_5k.npy', allow_pickle=True).
    ↪astype('float32')
mnist_targets = np.load('data/mnist_labels_5k.npy', allow_pickle=True).
    ↪astype('float32')

mnist_data = mnist_data.reshape(-1, 28, 28, 1) / 255

num_train = 3000
num_val = 1000
num_test = 1000

x_train = mnist_data[:num_train]
y_train = mnist_targets[:num_train]

x_val = mnist_data[num_train:num_train+num_val]
y_val = mnist_targets[num_train:num_train+num_val]

x_test = mnist_data[-num_test:]
y_test = mnist_targets[-num_test:]

```

Define the hyperparameters, the model, and the optimiser, where we choose the [Adam optimiser](#) this time.

```
[4]: learning_rate = 0.0001
      batch_size = 64
      num_epochs = 20
      latent_dim = 2

      vae_model = VAE(latent_dim, batch_size)
      optimizer = keras.optimizers.Adam(learning_rate=learning_rate)
```

In the following, we write our own loop for training and validation.

Please note that the training can take up to several minutes!

```
[5]: for epoch in range(1, num_epochs + 1):

      # Training steps
      for i in range(num_train // batch_size):
          train_step(vae_model, x_train[i:i + batch_size], optimizer)

      # Validation steps
      r_val = num_val // batch_size
      loss = keras.metrics.Mean()
      elbo = 0

      for i in range(r_val):
          loss(compute_loss(vae_model, x_val[i:i + batch_size]))
          elbo += -loss.result()

      elbo /= r_val

      print(f'Epoch: {epoch}/{num_epochs}, validation loss (ELBO): {elbo}')
```

```
Epoch: 1/20, validation loss (ELBO): -525.4475708007812
Epoch: 2/20, validation loss (ELBO): -417.3727722167969
Epoch: 3/20, validation loss (ELBO): -336.41778564453125
Epoch: 4/20, validation loss (ELBO): -285.1370849609375
Epoch: 5/20, validation loss (ELBO): -249.80979919433594
Epoch: 6/20, validation loss (ELBO): -233.10906982421875
Epoch: 7/20, validation loss (ELBO): -224.14547729492188
Epoch: 8/20, validation loss (ELBO): -219.64218139648438
Epoch: 9/20, validation loss (ELBO): -216.69764709472656
Epoch: 10/20, validation loss (ELBO): -214.56991577148438
Epoch: 11/20, validation loss (ELBO): -212.3656768798828
Epoch: 12/20, validation loss (ELBO): -210.2912139892578
Epoch: 13/20, validation loss (ELBO): -208.59295654296875
Epoch: 14/20, validation loss (ELBO): -207.23239135742188
Epoch: 15/20, validation loss (ELBO): -206.16909790039062
```

```
Epoch: 16/20, validation loss (ELBO): -204.2316131591797
Epoch: 17/20, validation loss (ELBO): -203.1484832763672
Epoch: 18/20, validation loss (ELBO): -201.591064453125
Epoch: 19/20, validation loss (ELBO): -200.03050231933594
Epoch: 20/20, validation loss (ELBO): -198.95579528808594
```

For our custom model, we need to store the trained weights in the following way:

```
[ ]: vae_model.save_weights('output/MNIST_VAE.h5', save_format='h5')
```

You can load a pretrained model like this:

```
[6]: vae_model = VAE(latent_dim, num_test)
vae_model(x_test) # For input dimension specification

# Our pretrained model (300 epochs):
vae_model.load_weights('output/MNIST_VAE_pretrained.h5')

# ... or load your model:
# vae_model.load_weights('output/MNIST_VAE.h5')
```

Let's check out the results for the test data.

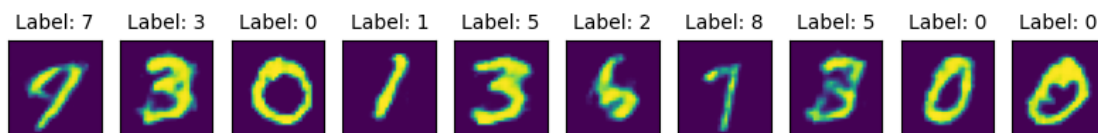
```
[7]: mean_z, logvar_z = vae_model.encode(x_test)
latent_z = vae_model.reparameterize(mean_z, logvar_z)
predictions = vae_model.sample(latent_z)

fig, axs = plt.subplots(1,10, figsize=(10,5))

for i, ax in enumerate(axs.ravel()):
    ax.imshow(predictions[i])

    ax.set_title(f"Label: {int(y_test[i])}", fontsize=10)
    ax.set_xticks([])
    ax.set_yticks([])

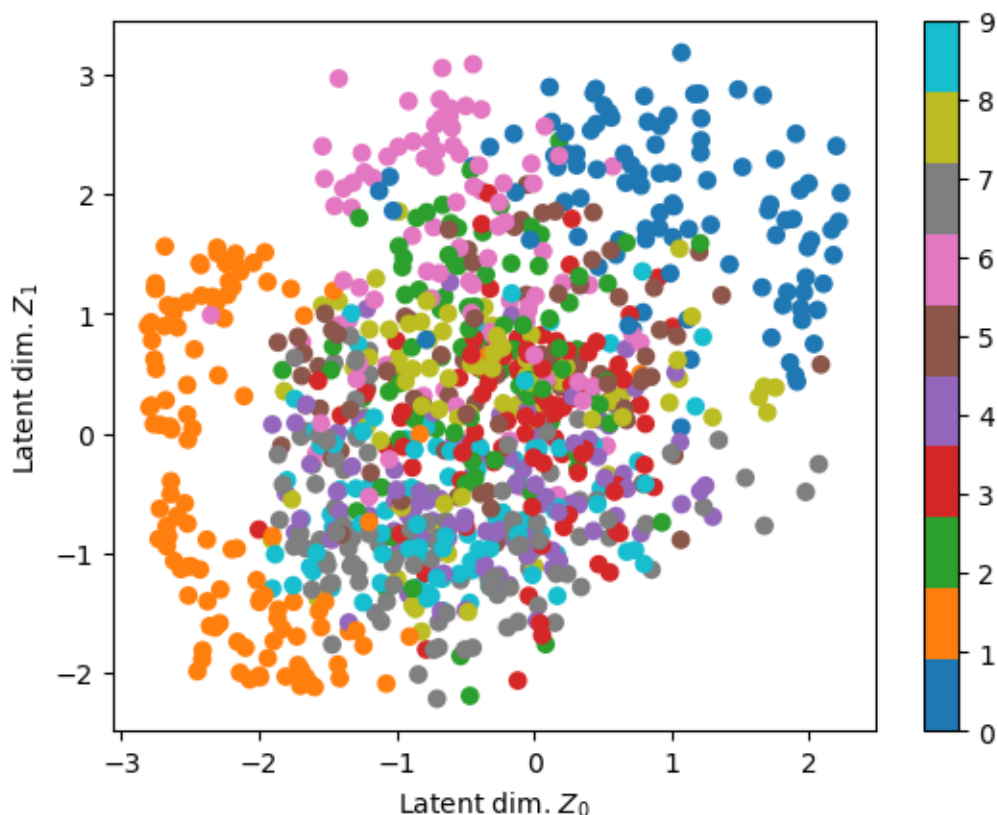
plt.show()
```



We can also investigate where the test datapoints lie in the latent space.

```
[8]: plt.scatter(latent_z[:, 0], latent_z[:, 1], c=y_test, cmap='tab10')
plt.colorbar()
```

```
plt.xlabel(r"Latent dim. $Z_0$")
plt.ylabel(r"Latent dim. $Z_1$")
plt.show()
```



1.2 Variational Autoencoder for Molecules

In the first part, we considered unsupervised learning. In the following we apply the VAE-idea on a molecular dataset and train a **Chemical VAE**. In addition to the standard, unsupervised VAE, we also predict a chemical property Y . This supervised setting allows us to do a **guided search of novel molecules** which match a property of our interest.

As illustrated above, we predict a chemical property with an **additional decoder network** directly from the latent sample. This not only provides additional information about the molecules but also structures the latent representation Z with respect to property Y . This will have the effect that the chemical property will gradually change within the latent space.

So far, we have worked with tabular data and images as inputs. Molecules can be represented in various ways. A common representation is a graph. Here, the nodes of a molecular graph are the atoms and the edges are molecular bonds. A popular and simple depiction of a molecular graph is a so-called **SMILES representation**. It is a text string that encloses all the information required

to convert it to a graph.

For our experiments, we will use a variant called the **Deep-SMILES representation**, which makes generation of valid molecular structures easier.

Let's have a closer look at the dataset and what these SMILES and Deep-SMILES look like.

```
[9]: import pandas as pd
import numpy as np

molecule_data = pd.read_pickle('data/molecule_data.pkl')
molecule_data
```

```
[9]:
```

	id	smiles	deep_smiles	band_gap
0	2	N	N	0.3399
1	3	O	O	0.3615
2	4	C#C	C#C	0.3351
3	7	CC	CC	0.4426
4	8	CO	CO	0.3437
...	...	...	...	...
31667	41529	C1CC2=CCC0CC12	CCC=CCC0CC97	0.2548
31668	41530	C1CC2=CC0CCC12	CCC=CC0CCC97	0.2528
31669	41531	C1CC2=CC0C0C12	CCC=CC0C0C97	0.2504
31670	41533	C10C2=NCCCCC12	C0C=NCCCCC97	0.2701
31671	41535	C10C2=NCCC0C12	C0C=NCCC0C97	0.2648

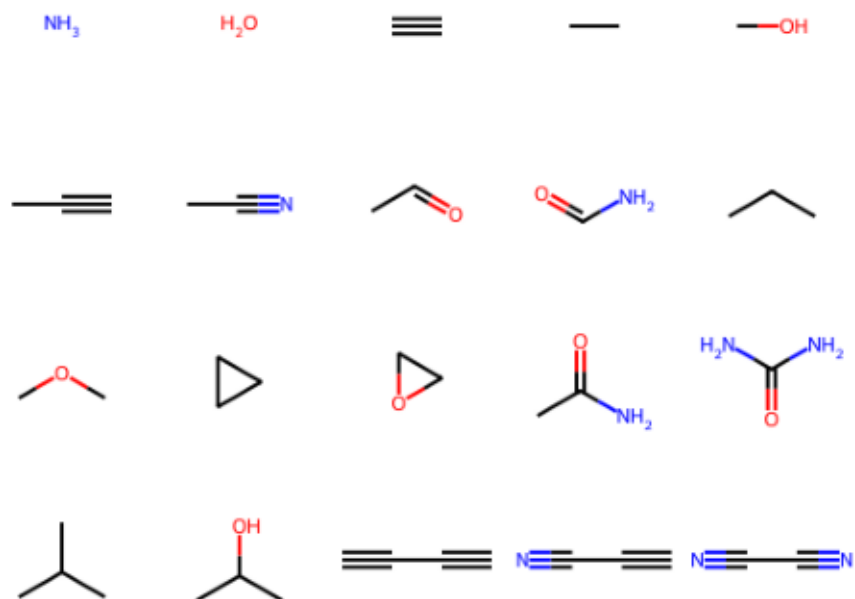
[31672 rows x 4 columns]

We can now plot the first and the last molecules from the dataset as chemical graphs.

```
[10]: from rdkit import Chem
from rdkit.Chem import Draw
import matplotlib.pyplot as plt

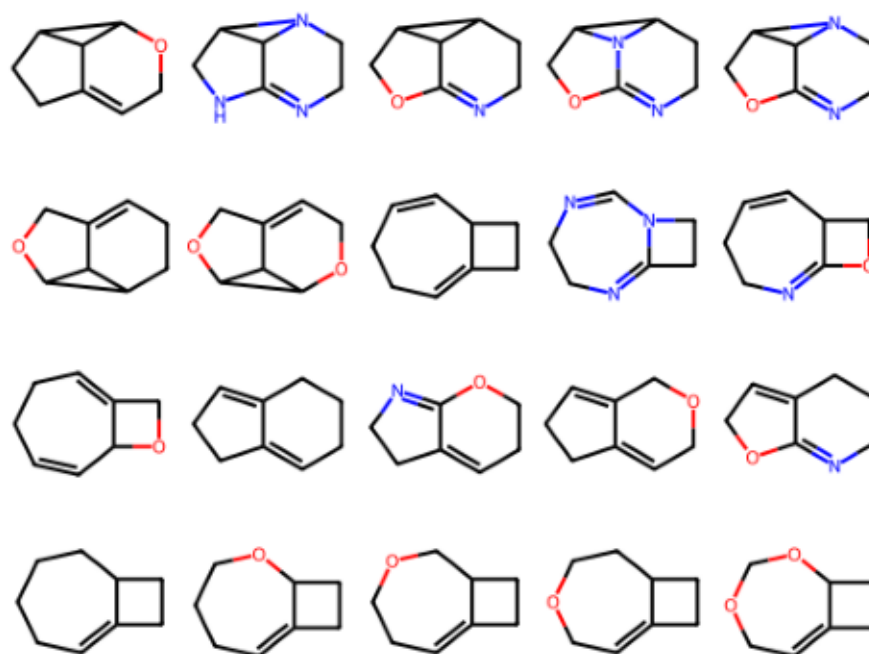
smiles_list = molecule_data['smiles'].values[0:20]
molecule_list = [Chem.MolFromSmiles(smiles) for smiles in smiles_list]
figure = Draw.MolsToGridImage(molecule_list, molsPerRow=5, subImgSize=(100,
↪100), returnPNG=False)

plt.imshow(figure)
plt.axis('off')
plt.show()
```



```
[11]: smiles_list = molecule_data['smiles'].values[31650:31670]
molecule_list = [Chem.MolFromSmiles(smiles) for smiles in smiles_list]
figure = Draw.MolsToGridImage(molecule_list, molsPerRow=5, subImgSize=(100,
↪100), returnPNG=False)

plt.imshow(figure)
plt.axis('off')
plt.show()
```

Let's define some functions which help us to convert deep-smiles into a one-hot representation and normalise the property values.

```
[12]: def smiles_to_onehot(deep_smiles):

    max_len = max([len(x) for x in deep_smiles])
    data = []

    for item in deep_smiles:
        ch_lst = list(item) + (max_len - len(item)) * [stop_character]

        res = []

        for ch in ch_lst:
            r = [0] * len(chars)
            r[chars.index(ch)] = 1
            res.append(r)

        data.append(res)

    return data
```

```

def one_hot_to_smiles(data, chars, stop_character):

    res = []

    for item in data:
        ch_lst = []

        for ch in item:
            ch_lst.append(chars[np.argmax(ch)])

        res.append(''.join(ch_lst).split(stop_character)[0])

    return res

def sparse_to_smiles(data):

    res = []

    for item in data:
        ch_lst = []

        for ch in item:
            ch_lst.append(chars[ch[0]])

        res.append(''.join(ch_lst))

    return res

```

First, we shuffle the data.

```
[13]: molecule_data = molecule_data.sample(frac=1, random_state=123)
```

Then, we use our function to convert deep smiles to a one-hot representation

```

[14]: stop_character = 'x'

chars = sorted(list(set(''.join(molecule_data['deep_smiles'].values))))
chars = chars + [stop_character]

print('Unique characters:', chars)

data = smiles_to_onehot(molecule_data['deep_smiles'].values)
data = np.array(data).reshape((len(data), 17, 15, 1)).astype(np.float32)

```

```
Unique characters: ['#', ')', '3', '4', '5', '6', '7', '8', '9', '=', 'C', 'F',
'N', 'O', 'x']
```

and normalise the property values

```
[15]: band_gap_mean = molecule_data['band_gap'].mean()
band_gap_std = molecule_data['band_gap'].std()
target = (molecule_data['band_gap'] - band_gap_mean) / band_gap_std
target = target.values.reshape((len(data), 1)).astype(np.float32)
```

As usual, we split the dataset into training, validation, and test sets.

```
[16]: num_train = 28000
num_val = 1000
num_test = 1000

x_train = data[:num_train]
y_train = target[:num_train]

x_val = data[num_train:num_train + num_val]
y_val = target[num_train:num_train + num_val]

x_test = data[num_train + num_val:num_train + num_val + num_test]
y_test = target[num_train + num_val:num_train + num_val + num_test]
```

We can now create our new model CVAE which provides property predictions.

```
[17]: import tensorflow as tf
from tensorflow import keras

class CVAE(keras.Model):

    def __init__(self, latent_dim, batch_size):
        super(CVAE, self).__init__()
        self.latent_dim = latent_dim
        self.batch_size = batch_size

        self.encoder = keras.Sequential(
            [
                keras.layers.InputLayer(input_shape=(17, 15, 1)),
                keras.layers.Flatten(),
                keras.layers.Dense(units=1120, activation='relu'),
                keras.layers.Dense(units=1120, activation='relu'),
                keras.layers.Dense(latent_dim + latent_dim),
            ]
        )

        self.decoder = keras.Sequential(
            [
                keras.layers.InputLayer(input_shape=(latent_dim,)),
                keras.layers.Dense(units=1120, activation='relu'),
                keras.layers.Dense(units=1120, activation='relu'),
                keras.layers.Dense(units=255),
```

```

        keras.layers.Reshape(target_shape=(17, 15, 1)),
    ]
)

# Our new decoder:
self.decoder_property = keras.Sequential(
    [
        keras.layers.InputLayer(input_shape=(latent_dim,)),
        keras.layers.Dense(units=1120, activation='relu'),
        keras.layers.Dense(units=1120, activation='relu'),
        keras.layers.Dense(units=1),
    ]
)

def call(self, x):
    mean, logvar = tf.split(self.encoder(x), num_or_size_splits=2, axis=1)
    z = self.reparameterize(mean, logvar)
    return self.decoder(z), mean, logvar

@tf.function
def sample(self, eps=None):
    if eps is None:
        eps = tf.random.normal(shape=(self.batch_size, self.latent_dim))
    return self.decode(eps, apply_sigmoid=True)

def encode(self, x):
    mean, logvar = tf.split(self.encoder(x), num_or_size_splits=2, axis=1)
    return mean, logvar

def reparameterize(self, mean, logvar):
    eps = tf.random.normal(shape=(self.batch_size, self.latent_dim))
    return eps * tf.exp(logvar * .5) + mean

def decode(self, z, apply_sigmoid=False):
    logits = self.decoder(z)
    if apply_sigmoid:
        probs = tf.sigmoid(logits)
        return probs
    return logits

def log_normal_pdf(sample, mean, logvar, raxis=1):
    log2pi = tf.math.log(2.0 * np.pi)

    return tf.reduce_sum(-0.5 * ((sample - mean) ** 2.0 * tf.exp(-logvar) +
    ↪ logvar + log2pi), axis=raxis)

```

```

def compute_loss(model, x, y):
    mean, logvar = model.encode(x)
    z = model.reparameterize(mean, logvar)
    x_logit = model.decode(z)
    cross_ent = tf.nn.sigmoid_cross_entropy_with_logits(logits=x_logit,
↪labels=x)
    logpx_z = -tf.reduce_sum(cross_ent, axis=[1, 2, 3])
    logpz = log_normal_pdf(z, 0., 0.)
    logqz_x = log_normal_pdf(z, mean, logvar)

    # Property
    y_pred = model.decoder_property(z)
    y_mse = -tf.reduce_mean(tf.square(y - y_pred), axis = 0)

    return -tf.reduce_mean(100 * y_mse + logpx_z + logpz - logqz_x)

def compute_y_mae(model, x, y):
    mean, logvar = model.encode(x)
    z = model.reparameterize(mean, logvar)
    y_pred = model.decoder_property(z)
    y_mae = tf.reduce_mean(tf.abs(y - y_pred), axis = 0)
    return y_mae

@tf.function
def train_step(model, x, y, optimizer):
    with tf.GradientTape() as tape:
        loss = compute_loss(model, x, y)
        gradients = tape.gradient(loss, model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))

```

With this, we can now set the hyperparameters and create our model and optimiser.

```

[18]: learning_rate = 0.0001
      batch_size = 100
      num_epochs = 31
      latent_dim = 14

      optimizer = keras.optimizers.Adam(learning_rate)
      model = CVAE(latent_dim, batch_size)

```

You can try to train this model:

```

[19]: for epoch in range(1, num_epochs + 1):

      # Training steps

```

```

    for i in range(num_train // batch_size):
        train_step(model, x_train[i:i + batch_size], y_train[i:i + batch_size],
optimizer)

    # Validation steps
    r_val = num_val // batch_size
    loss = keras.metrics.Mean()
    elbo = 0
    y_loss = 0

    for i in range(r_val):
        loss(compute_loss(model, x_val[i:i + batch_size], y_val[i:i +
batch_size]))
        y_loss += compute_y_mae(model, x_val[i:i + batch_size], y_val[i:i +
batch_size]).numpy()[0]
        elbo += -loss.result()

    elbo /= r_val
    y_loss = y_loss / r_val * band_gap_std * 627

    # Print validation results
    print(f'Epoch: {epoch}/{num_epochs}, validation loss (ELBO): {elbo}, Y MAE:
{y_loss}')

    if epoch % 10:
        path = 'output/Molecules_VAE.h5'
        model.save_weights(path, save_format='h5')

```

Epoch: 1/31, validation loss (ELBO): -69.86585998535156, Y MAE:
13.398013362266083

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
Cell In[19], line 5
      1 for epoch in range(1, num_epochs + 1):
      2
      3     # Training steps
      4     for i in range(num_train // batch_size):
----> 5         train_step(model, x_train[i:i + batch_size], y_train[i:i +
batch_size], optimizer)
      7     # Validation steps
      8     r_val = num_val // batch_size

File ~/anaconda3/APML/lib/python3.11/site-packages/tensorflow/python/util/
traceback_utils.py:150, in filter_traceback.<locals>.error_handler(*args,
**kwargs)
    148 filtered_tb = None

```

```

149 try:
--> 150     return fn(*args, **kwargs)
151 except Exception as e:
152     filtered_tb = _process_traceback_frames(e.__traceback__)

```

File ~/anaconda3/APML/lib/python3.11/site-packages/tensorflow/python/eager/
↳polymorphic_function/polymorphic_function.py:894, in Function.__call__(self,
↳*args, **kwargs)

```

891 compiler = "xla" if self._jit_compile else "nonXla"
893 with OptionalXlaContext(self._jit_compile):
--> 894     result = self._call(*args, **kwargs)
896 new_tracing_count = self.experimental_get_tracing_count()
897 without_tracing = (tracing_count == new_tracing_count)

```

File ~/anaconda3/APML/lib/python3.11/site-packages/tensorflow/python/eager/
↳polymorphic_function/polymorphic_function.py:926, in Function._call(self,
↳*args, **kwargs)

```

923 self._lock.release()
924 # In this case we have created variables on the first call, so we run
↳the
925 # defunned version which is guaranteed to never create variables.
--> 926 return self._no_variable_creation_fn(*args, **kwargs) # pylint:␣
↳disable=not-callable
927 elif self._variable_creation_fn is not None:
928     # Release the lock early so that multiple threads can perform the cal
929     # in parallel.
930 self._lock.release()

```

File ~/anaconda3/APML/lib/python3.11/site-packages/tensorflow/python/eager/
↳polymorphic_function/tracing_compiler.py:143, in TracingCompiler.
↳__call__(self, *args, **kwargs)

```

140 with self._lock:
141     (concrete_function,
142     filtered_flat_args) = self._maybe_define_function(args, kwargs)
--> 143 return concrete_function._call_flat(
144     filtered_flat_args, captured_inputs=concrete_function.
↳captured_inputs)

```

File ~/anaconda3/APML/lib/python3.11/site-packages/tensorflow/python/eager/
↳polymorphic_function/monomorphic_function.py:1757, in ConcreteFunction.
↳_call_flat(self, args, captured_inputs, cancellation_manager)

```

1753 possible_gradient_type = gradients_util.PossibleTapeGradientTypes(args)
1754 if (possible_gradient_type == gradients_util.POSSIBLE_GRADIENT_TYPES_NO_E
1755     and executing_eagerly):
1756     # No tape is watching; skip to running the function.
-> 1757 return self._build_call_outputs(self._inference_function.call(
1758     ctx, args, cancellation_manager=cancellation_manager))
1759 forward_backward = self._select_forward_and_backward_functions(
1760     args,

```

```

1761     possible_gradient_type,
1762     executing_eagerly)
1763 forward_function, args_with_tangents = forward_backward.forward()

File ~/anaconda3/APML/lib/python3.11/site-packages/tensorflow/python/eager/
↳polymorphic_function/monomorphic_function.py:381, in _EagerDefinedFunction.
↳call(self, ctx, args, cancellation_manager)
    379 with _InterpolateFunctionError(self):
    380     if cancellation_manager is None:
--> 381         outputs = execute.execute(
    382             str(self.signature.name),
    383             num_outputs=self._num_outputs,
    384             inputs=args,
    385             attrs=attrs,
    386             ctx=ctx)
    387     else:
    388         outputs = execute.execute_with_cancellation(
    389             str(self.signature.name),
    390             num_outputs=self._num_outputs,
    (...)
    393             ctx=ctx,
    394             cancellation_manager=cancellation_manager)

File ~/anaconda3/APML/lib/python3.11/site-packages/tensorflow/python/eager/
↳execute.py:52, in quick_execute(op_name, num_outputs, inputs, attrs, ctx, name)
    50 try:
    51     ctx.ensure_initialized()
----> 52     tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name
    53                                         inputs, attrs, num_outputs)
    54 except core._NotOkStatusException as e:
    55     if name is not None:

KeyboardInterrupt:

```

Alternatively, you can just load the pretrained model here:

```

[20]: model = CVAE(latent_dim, num_test)
      model(x_test)

# Our pretrained model (31 epochs):
model.load_weights('output/Molecules_VAE_pretrained.h5')

# ... or load your model:
# model.load_weights('output/Molecules_VAE.h5')

```

We can now make predictions with our model and use PCA for dimensionality reduction in order to visualise the latent space.

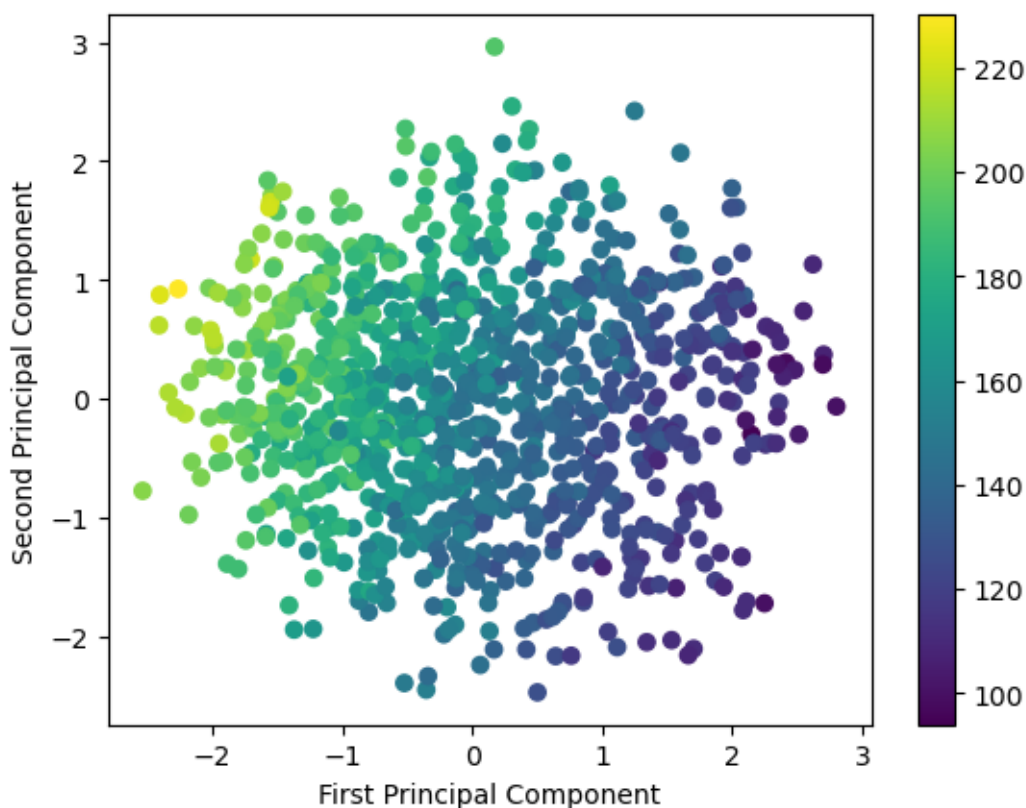

```
[21]: from sklearn.decomposition import PCA

mean_z, logvar_z = model.encode(x_test)
latent_z = model.reparameterize(mean_z, logvar_z)
target_pred = model.decoder_property(latent_z).numpy().flatten()

# Invert the scaling and convert the property unit to Kcal/mol
target_pred = (target_pred * band_gap_std + band_gap_mean) * 627

data_pca = PCA(n_components=2)
data_pca.fit(mean_z.numpy())
data_pca_transform = data_pca.transform(mean_z.numpy())

plt.scatter(data_pca_transform[:, 0], data_pca_transform[:, 1], c=target_pred)
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.colorbar()
plt.show()
```



In this plot we can see a **smooth transition of the property correspondence** making it **easy to sample molecules of our interest**. To this end, we can linearly interpolate between

two molecules.

Let us pick 1000 datapoints in the latent space.

```
[22]: num_inter_data = 1000

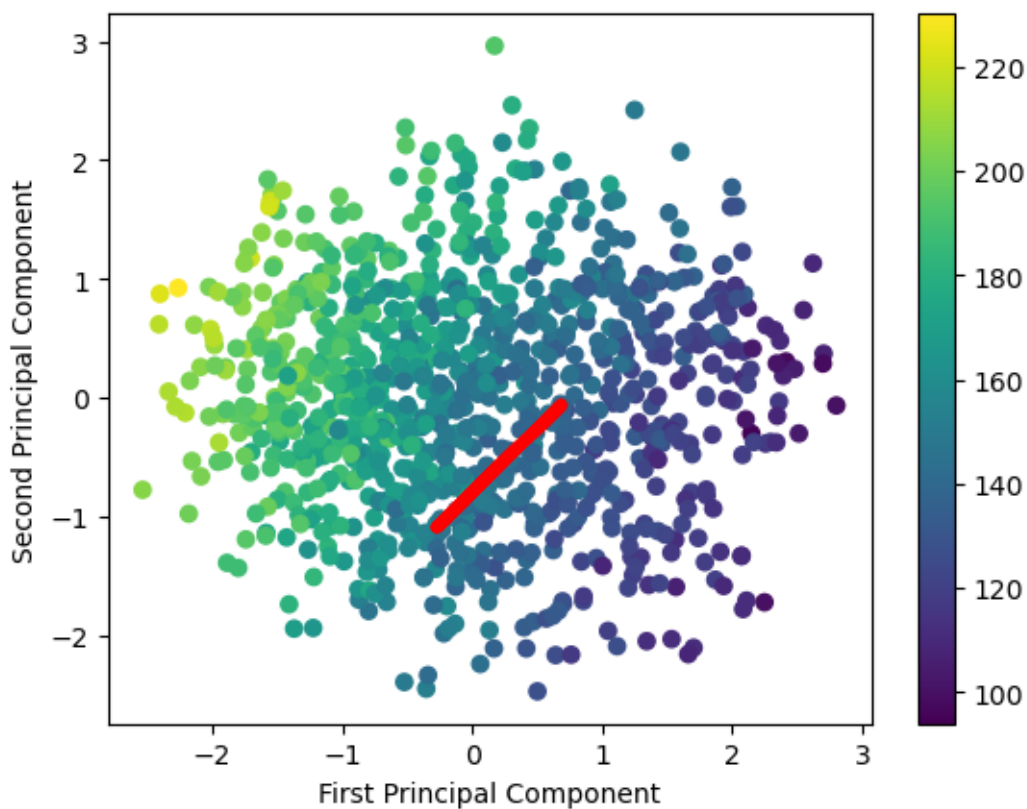
latent_z_inter = np.linspace(mean_z[7:8][0], mean_z[8:9][0], num_inter_data)
latent_z_inter_transform = data_pca.transform(latent_z_inter)

plt.scatter(data_pca_transform[:, 0], data_pca_transform[:, 1], c=target_pred)
plt.colorbar()

plt.scatter(latent_z_inter_transform[:, 0], latent_z_inter_transform[:, 1],
            marker='.', c='red')

plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')

plt.show()
```



We can now take these latent datapoints and decode molecules and their corresponding property value.

```
[23]: data_inter_pred = model.decoder(latent_z_inter)
target_inter_pred = model.decoder_property(latent_z_inter).numpy()

deep_smi_inter_pred = sparse_to_smiles(tf.argmax(data_inter_pred, axis=2).
↳numpy())
deep_smi_inter_pred = [item.replace(stop_character, '') for item in
↳deep_smi_inter_pred]

deep_smi_inter_pred, idx = np.unique(deep_smi_inter_pred, return_index=True)

# Invert the scaling and convert the property unit to Kcal/mol
target_inter_pred = (target_inter_pred[idx] * band_gap_std + band_gap_mean) *
↳627
```

Parse generated molecules and sort out invalid samples.

```
[24]: import deepsmiles

converter = deepsmiles.Converter(rings=True, branches=True)

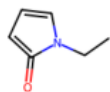
molecule_list = []
labels = []

for i, item in enumerate(deep_smi_inter_pred):
    try:
        smi = converter.decode(item)
        mol = Chem.MolFromSmiles(smi, sanitize = False)

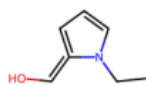
        if mol is not None:
            molecule_list.append(mol)
            p = target_inter_pred[i][0]
            labels.append(f'{smi} - {p:.2f} kcal/mol')
    except:
        pass

figure = Draw.MolsToGridImage(molecule_list, legends=labels, molsPerRow=4,
↳subImgSize=(300, 300), returnPNG=False)

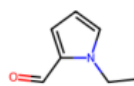
plt.figure(figsize=(10, 10))
plt.imshow(figure)
plt.axis('off')
plt.show()
```



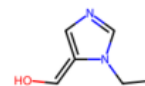
CCN1C=CC=C1=O - 139.47 kcal/mol



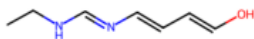
CCN1C=CC=C1=CO - 139.47 kcal/mol



CCN1C=CC=C1C=O - 139.46 kcal/mol



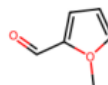
CCN1C=NC=C1=CO - 139.48 kcal/mol



CCNC=NC=CC=CO - 139.48 kcal/mol



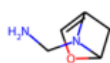
CO1C=CC=C1C - 139.41 kcal/mol



CO1C=CC=C1C=O - 139.41 kcal/mol



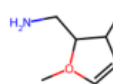
C12OC=C(C1)N2C - 140.50 kcal/mol



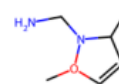
C12OC=C(C1)N2CN - 140.41 kcal/mol



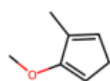
C1OC2=CC1ON2C - 140.84 kcal/mol



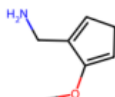
CO1C=CC(C)C1CN - 139.90 kcal/mol



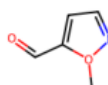
CO1C=CC(C)N1CN - 140.03 kcal/mol



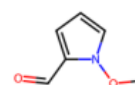
COC1=CCC=C1C - 139.48 kcal/mol



COC1=CCC=C1CN - 139.51 kcal/mol



CO1N=CC=C1C=O - 139.41 kcal/mol



CON1C=CC=C1C=O - 139.40 kcal/mol

Class_8-Transformers

May 20, 2026

1 8. Transformers

Convolutional neural network approaches have long dominated image processing. In this notebook, we introduce the **Transformer architecture**, which has revolutionised both natural language processing and computer vision, and is currently the dominant paradigm across much of deep learning.

In this notebook, we will get an idea of

- the attention mechanism and how it works,
- multi-head attention,
- positional encoding,
- the Vision Transformer (ViT) for image classification,
- and compare transformers with CNNs on MNIST.

The [Illustrated Transformer](#) is a great introduction to the topic and provides many nice illustrations, some of which we borrow in this notebook.

Keywords: Transformers, Attention, Self-Attention, Multi-Head Attention, Vision Transformer (ViT), Patch Embedding, Positional Encoding

1.1 Attention is All You Need

Transformers were introduced in the seminal [Attention is All You Need](#) paper by Vaswani et al. in 2017 and have become the dominant architecture in deep learning.

What makes transformers particularly powerful? For convolutional neural networks (CNNs), we have seen that they - process images with convolutional filters over **local receptive fields**, i.e. e.g. 3×3 windows of neighbouring pixels and - learn long-range dependencies through many convolutional and pooling layers.

The underlying design choice or **inductive bias** for CNNs is hence the idea of grouping information of neighbouring cells / pixels together, allowing for **locality** and **translation equivariance**.

Transformers introduce a more general design choice, overcoming some of the limitations of CNNs and other neural network architectures: - **Self-attention**: Every position can **attend** to / be processed with every other position (think of “position” as “pixel” for the time being) - **Global receptive field** from the first layer: long-range dependencies are immediately learnable - **Learned relationships** rather than hard-coded inductive biases like the local receptive field - **Parallelizable** computation (unlike recurrent neural networks, for example)

1.2 Understanding the Attention Mechanism

To illustrate the idea of self-attention, consider an example where we want to translate the sentence:

The animal didn't cross the street because it was too tired.

In a first step, we separate the sentence into **tokens**, a (pre-defined) basic unit of text / characters that the transformer is supposed to process. The illustrations below shows the different tokens in different lines. Self-attentions learns what other words (tokens) are important to process e.g. the word (token) **it**, with darker shades of orange higher importance / weight like for **animal**.

Image source: The Illustrated Transformer

The **attention mechanism** which allows the model to focus on relevant parts of the input when making predictions. Think of attention like a **database query**: - **Query (Q)**: What am I looking for? In our example, that's **it**. - **Key (K)**: What does each token / position represent? In our example, all other words (tokens), think "semantic content". - **Value (V)**: What information does each token / position contain? In our example, think of the "value" of information for the query.

The attention mechanism computes which values to retrieve based on the similarity between queries and keys.

Let's implement and understand transformers step by step!

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import math
import tensorflow as tf
from tensorflow import keras

seed = 123
```

2026-05-19 10:51:53.809689: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE4.1 SSE4.2, in other operations, rebuild TensorFlow with the appropriate compiler flags.

1.2.1 Step 1: Scaled Dot-Product Attention

The fundamental attention operation is the **scaled dot-product attention**

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

with - $Q \in \mathbb{R}^{n \times d_k}$ are the **queries** (what we're looking for) - $K \in \mathbb{R}^{m \times d_k}$ are the **keys** (what each position represents) - $V \in \mathbb{R}^{m \times d_v}$ are the **values** (actual content at each position) - d_k is the dimension of queries/keys - n is the number of query positions - m is the number of key/value positions - and the softmax operation we encountered previously $\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^C \exp(z_j)}$

How this works

1. **Compute similarity:** QK^T gives similarity scores between queries and keys
2. **Scaling:** Divide by $\sqrt{d_k}$ to prevent too small/big values, particularly **vanishing gradients**
3. **Normalise:** Applying the softmax provides attention weights between 0 and 1
4. **Aggregate:** Weighted sum of values using attention weights

1.2.2 Step 2: Self-Attention

In **self-attention**, Q , K , and V all come from the same input, like our sentence example above:
 - Each token / position attends to all positions (including itself) - Learns relationships between different parts of the input

We will implement attention as a custom Keras layer by subclassing `keras.layers.Layer` to see this in action. As seen many times in this course, there's also a convenient built-in `keras.layers.MultiHeadAttention` layer, too.

```
[2]: class ScaledDotProductAttention(keras.layers.Layer):
    def __init__(self, temperature, **kwargs):
        super().__init__(**kwargs)
        self.temperature = temperature # sqrt(d_k)

    def call(self, q, k, v, mask=None):
        """
        Args:
            q: Queries (batch_size, n_heads, len_q, d_k)
            k: Keys (batch_size, n_heads, len_k, d_k)
            v: Values (batch_size, n_heads, len_v, d_v)
            mask: Optional mask

        Returns:
            output: Attention output (batch_size, n_heads, len_q, d_v)
            attn: Attention weights (batch_size, n_heads, len_q, len_k)
        """
        # Compute attention scores: Q * K^T / sqrt(d_k)
        attn = tf.matmul(q, k, transpose_b=True) / self.temperature

        # Apply mask if provided (for padding or causality)
        if mask is not None:
            attn = tf.where(mask == 0, tf.constant(-1e9, dtype=attn.dtype),
                ↪attn)

        # Apply softmax to get attention weights
        attn = tf.nn.softmax(attn, axis=-1)

        # Apply attention weights to values
        output = tf.matmul(attn, v)

    return output, attn
```

1.2.3 Step 3: Multi-Head Attention

How about using several attention layers? **Multi-head attention** runs multiple attention operations in parallel:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where each head is

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

In the illustration below, the two heads are shown with attention values in orange and green for processing it. The first head *attends* to / puts large weights on **The animal**, while the second focuses more on **tired**.

Image source: The Illustrated Transformer

Similar to using several convolutional filters, multiple heads offer the possibility that - different heads can attend to different patterns, - one head might focus on local patterns, another on global structure, or - model capacity and expressiveness are increased.

```
[3]: class MultiHeadAttention(keras.layers.Layer):
    def __init__(self, n_heads, d_model, dropout=0.1, **kwargs):
        super().__init__(**kwargs)

        assert d_model % n_heads == 0, "d_model must be divisible by n_heads"

        self.n_heads = n_heads
        self.d_model = d_model
        self.d_k = d_model // n_heads  # Dimension per head

        # Linear projections for Q, K, V
        self.w_q = keras.layers.Dense(d_model)
        self.w_k = keras.layers.Dense(d_model)
        self.w_v = keras.layers.Dense(d_model)

        # Output projection
        self.w_o = keras.layers.Dense(d_model)

        # Attention mechanism
        self.attention = ScaledDotProductAttention(temperature=self.d_k ** 0.5)

        self.dropout = keras.layers.Dropout(dropout)
        self.layer_norm = keras.layers.LayerNormalization()

    def call(self, q, k, v, mask=None, training=None):
        batch_size = tf.shape(q)[0]
        len_q = tf.shape(q)[1]
```



```

len_k = tf.shape(k)[1]
len_v = tf.shape(v)[1]

# Save residual for later
residual = q

# Linear projections and split into heads
# (batch, len, d_model) -> (batch, len, n_heads, d_k) -> (batch,
↪ n_heads, len, d_k)
q = tf.reshape(self.w_q(q), [batch_size, len_q, self.n_heads, self.d_k])
q = tf.transpose(q, [0, 2, 1, 3])
k = tf.reshape(self.w_k(k), [batch_size, len_k, self.n_heads, self.d_k])
k = tf.transpose(k, [0, 2, 1, 3])
v = tf.reshape(self.w_v(v), [batch_size, len_v, self.n_heads, self.d_k])
v = tf.transpose(v, [0, 2, 1, 3])

# Apply attention
output, attn = self.attention(q, k, v, mask=mask)

# Concatenate heads
# (batch, n_heads, len_q, d_k) -> (batch, len_q, n_heads, d_k) ->
↪ (batch, len_q, d_model)
output = tf.transpose(output, [0, 2, 1, 3])
output = tf.reshape(output, [batch_size, len_q, self.d_model])

# Output projection
output = self.dropout(self.w_o(output), training=training)

# Add residual and normalize
output = self.layer_norm(output + residual)

return output, attn

```

1.2.4 Step 4: Positional Encoding

So far, our attention layers have no notion of order or position! While in CNNs the position is implicit, because neighboring pixels are related, all positions are treated equally in attention. To address this, **positional encodings** are used to provide transformers with information about position, i.e. how far two words are apart from each other.

The original transformer uses sinusoidal positional encoding

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

where pos is the position, i is the dimension and different frequencies are used for different dimensions.

```
[4]: class PositionalEncoding(keras.layers.Layer):
    def __init__(self, d_model, max_len=5000, **kwargs):
        super().__init__(**kwargs)

        # Create positional encoding matrix
        pe = np.zeros((max_len, d_model), dtype=np.float32)
        position = np.arange(0, max_len, dtype=np.float32)[: , np.newaxis]
        div_term = np.exp(
            np.arange(0, d_model, 2, dtype=np.float32)
            * (-math.log(10000.0) / d_model)
        )

        pe[:, 0::2] = np.sin(position * div_term)
        pe[:, 1::2] = np.cos(position * div_term)

        # Add batch dimension and store as a non-trainable constant
        self.pe = tf.constant(pe[np.newaxis, ...])

    def call(self, x):
        # x: (batch_size, seq_len, d_model)
        seq_len = tf.shape(x)[1]
        return x + self.pe[:, :seq_len, :]
```

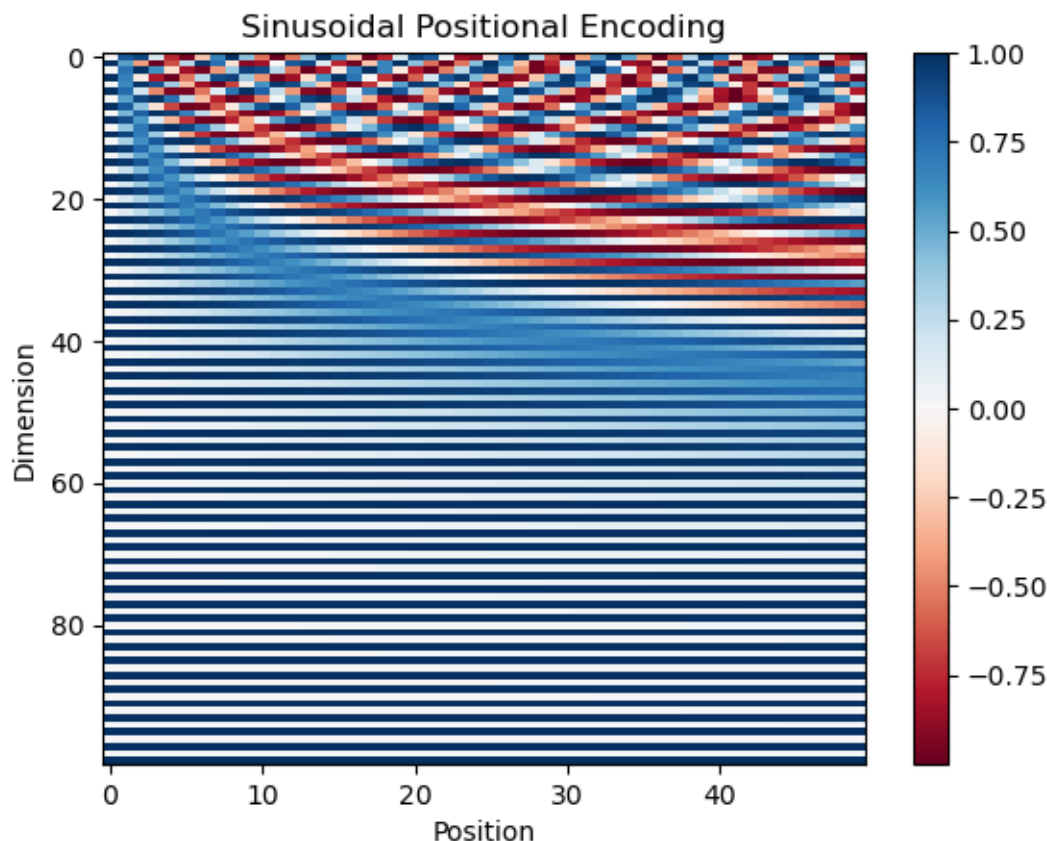
```
[5]: pe_layer = PositionalEncoding(d_model=100, max_len=50)
encoding = pe_layer.pe[0].numpy()

plt.imshow(encoding.T, aspect='auto', cmap='RdBu')

plt.xlabel('Position')
plt.ylabel('Dimension')
plt.title('Sinusoidal Positional Encoding')

plt.colorbar()

plt.show()
```



Note that the first dimensions have faster oscillations and thus capture short-range / detailed information, and the later dimensions have fewer oscillations and capture long-range / broader information.

1.3 The Vision Transformer

So far, our discussion has been very much focused on NLP examples. The [An Image Is Worth 16x16 Words: Transformers For Image Recognition At Scale](#) paper by Dosovitskiy et al. from 2021 showed how to apply transformers to image classification! The **Vision Transformer (ViT)** treats an image as a sequence of patches, as illustrated below.

Image source: Vision Transformer paper

In the following, we will adapt the ViT for our MNIST classification example. The architecture will look something like this:

Image (28x28)

↓

Split into patches (7x7 patches → 16 patches of 7x7)

↓

```

Flatten patches (16 patches of 49 pixels)
↓
Linear projection (16 patches → 16 embeddings)
↓
Add [CLS] token + positional embeddings
↓
Transformer Encoder (L layers)
↓
Extract [CLS] token
↓
Classification head (MLP)
↓
Output (10 classes for MNIST)

```

The key components here will be

1. **Patch Embedding:** Divide an image into patches and linearly project them
2. **[CLS] Token:** Special learnable token for classification
3. **Positional Embedding:** Learnable position information
4. **Transformer Encoder:** Stack of multi-head attention + FFNN layers
5. **Classification Head:** MLP on [CLS] token output

```

[6]: class PatchEmbedding(keras.layers.Layer):
    """Split image into patches and embed them"""

    def __init__(self, img_size=28, patch_size=7, embed_dim=64, **kwargs):
        super().__init__(**kwargs)
        self.img_size = img_size
        self.patch_size = patch_size
        self.embed_dim = embed_dim
        self.n_patches = (img_size // patch_size) ** 2 # Number of patches

        # Conv2D with kernel=patch_size and stride=patch_size
        # extracts non-overlapping patches and projects them to embed_dim
        self.proj = keras.layers.Conv2D(
            filters=embed_dim,
            kernel_size=patch_size,
            strides=patch_size,
        )

    def call(self, x):
        # x: (batch_size, height, width, channels) -- channel-last (Keras
        ↪ default)
        x = self.proj(x) # (batch_size, n_patches_h, n_patches_w, embed_dim)

        # Flatten the spatial dims into a sequence of patches
        batch_size = tf.shape(x)[0]
        x = tf.reshape(x, [batch_size, self.n_patches, self.embed_dim])

```

```

        return x # (batch_size, n_patches, embed_dim)

class TransformerEncoderBlock(keras.layers.Layer):
    def __init__(self, d_model, n_heads, mlp_ratio=4.0, dropout=0.1, **kwargs):
        super().__init__(**kwargs)

        # Multi-head self-attention
        self.attn = MultiHeadAttention(n_heads, d_model, dropout)

        # Feed-forward network
        mlp_hidden_dim = int(d_model * mlp_ratio)
        self.mlp = keras.Sequential([
            keras.layers.Dense(mlp_hidden_dim, activation='gelu'), # GELU is a
            ↪ smoother than ReLU
            keras.layers.Dropout(dropout),
            keras.layers.Dense(d_model),
            keras.layers.Dropout(dropout),
        ])

        self.norm1 = keras.layers.LayerNormalization()
        self.norm2 = keras.layers.LayerNormalization()

    def call(self, x, training=None):
        # Self-attention with residual connection (pre-norm)
        x_norm = self.norm1(x)
        attn_out, attn_weights = self.attn(x_norm, x_norm, x_norm,
            ↪ training=training)
        x = x + attn_out

        # Feed-forward with residual connection
        x = x + self.mlp(self.norm2(x), training=training)

        return x, attn_weights

class VisionTransformer(keras.Model):
    def __init__(
        self,
        img_size=28,
        patch_size=7,
        num_classes=10,
        embed_dim=64,
        depth=4,
        n_heads=4,
        mlp_ratio=4.0,
        dropout=0.1,
        **kwargs,
    ):

```

```

):
    super().__init__(**kwargs)

    # Patch embedding
    self.patch_embed = PatchEmbedding(img_size, patch_size, embed_dim)
    n_patches = self.patch_embed.n_patches

    # [CLS] token - learnable classification token
    self.cls_token = self.add_weight(
        name='cls_token',
        shape=(1, 1, embed_dim),
        initializer=keras.initializers.TruncatedNormal(stddev=0.02),
        trainable=True,
    )

    # Positional embeddings (learnable)
    self.pos_embed = self.add_weight(
        name='pos_embed',
        shape=(1, n_patches + 1, embed_dim),
        initializer=keras.initializers.TruncatedNormal(stddev=0.02),
        trainable=True,
    )

    self.pos_drop = keras.layers.Dropout(dropout)

    # Transformer encoder blocks
    self.blocks = [
        TransformerEncoderBlock(embed_dim, n_heads, mlp_ratio, dropout,
        ↪name=f'block_{i}')
        for i in range(depth)
    ]

    self.norm = keras.layers.LayerNormalization()

    # Classification head (softmax probabilities, matching the rest of the
    ↪course)
    self.head = keras.layers.Dense(num_classes, activation='softmax')

    def call(self, x, training=None, return_attention=False):
        batch_size = tf.shape(x)[0]

        # Patch embedding
        x = self.patch_embed(x) # (batch_size, n_patches, embed_dim)

        # Prepend [CLS] token
        cls_tokens = tf.tile(self.cls_token, [batch_size, 1, 1])
        x = tf.concat([cls_tokens, x], axis=1) # (batch_size, n_patches+1,
        ↪embed_dim)

```

```

    # Add positional encoding
    x = x + self.pos_embed
    x = self.pos_drop(x, training=training)

    # Apply transformer blocks
    attention_maps = []
    for block in self.blocks:
        x, attn = block(x, training=training)
        if return_attention:
            attention_maps.append(attn)

    x = self.norm(x)

    # Extract [CLS] token for classification
    cls_output = x[:, 0]

    # Classification probabilities
    probs = self.head(cls_output)

    if return_attention:
        return probs, attention_maps
    return probs

```

Let's load the MNIST data as we've done before

```

[7]: mnist_data = np.load('data/mnist_data_5k.npy', allow_pickle=True).
    ↪astype('float32')
    mnist_targets = np.load('data/mnist_labels_5k.npy', allow_pickle=True).
    ↪astype('int64')

    mnist_data = mnist_data.reshape(-1, 28, 28, 1) / 255.0

    num_train = 3000
    num_val = 1000
    num_test = 1000

    x_train = mnist_data[:num_train]
    y_train = mnist_targets[:num_train]

    x_val = mnist_data[num_train:num_train + num_val]
    y_val = mnist_targets[num_train:num_train + num_val]

    x_test = mnist_data[-num_test:]
    y_test = mnist_targets[-num_test:]

    print(f"Training set: {x_train.shape}")

```

```
print(f"Validation set: {x_val.shape}")
print(f"Test set:      {x_test.shape}")
```

```
Training set: (3000, 28, 28, 1)
Validation set: (1000, 28, 28, 1)
Test set:      (1000, 28, 28, 1)
```

And train our ViT model on MNIST

```
[9]: batch_size = 64
num_epochs = 10
learning_rate = 0.001

img_size = 28
patch_size = 7 # 28/7 = 4x4 = 16 patches
embed_dim = 64
depth = 10 # Number of transformer blocks
n_heads = 8 # Number of attention heads
num_classes = 10

keras.utils.set_random_seed(seed)

# Create the model
vit = VisionTransformer(
    img_size=img_size,
    patch_size=patch_size,
    num_classes=num_classes,
    embed_dim=embed_dim,
    depth=depth,
    n_heads=n_heads,
    name='ViT',
)

# Build the model so we can call .summary() before training
vit.build(input_shape=(None, img_size, img_size, 1))

# Cosine-decay learning rate schedule (matches the spirit of CosineAnnealingLR)
steps_per_epoch = num_train // batch_size
schedule = keras.optimizers.schedules.CosineDecay(
    initial_learning_rate=learning_rate,
    decay_steps=num_epochs * steps_per_epoch,
    alpha=1e-6 / learning_rate, # eta_min / initial_lr
)

# AdamW with weight decay (typical choice for transformers)
optimizer = keras.optimizers.AdamW(
    learning_rate=schedule,
    weight_decay=0.01,
```



```

)

vit.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)

vit_history = vit.fit(
    x=x_train, y=y_train,
    batch_size=batch_size,
    validation_data=(x_val, y_val),
    epochs=num_epochs,
)

test_loss, test_acc = vit.evaluate(x_test, y_test, verbose=0)
print(f"\nTest accuracy: {test_acc:.4f}")

vit.summary()

```

```

Epoch 1/10
47/47 [=====] - 19s 145ms/step - loss: 1.7865 -
accuracy: 0.3810 - val_loss: 1.0894 - val_accuracy: 0.6590
Epoch 2/10
47/47 [=====] - 5s 107ms/step - loss: 0.8534 -
accuracy: 0.7163 - val_loss: 0.5797 - val_accuracy: 0.8110
Epoch 3/10
47/47 [=====] - 5s 103ms/step - loss: 0.5323 -
accuracy: 0.8363 - val_loss: 0.3838 - val_accuracy: 0.8830
Epoch 4/10
47/47 [=====] - 5s 108ms/step - loss: 0.3598 -
accuracy: 0.8893 - val_loss: 0.3788 - val_accuracy: 0.8770
Epoch 5/10
47/47 [=====] - 5s 105ms/step - loss: 0.2652 -
accuracy: 0.9247 - val_loss: 0.2886 - val_accuracy: 0.9090
Epoch 6/10
47/47 [=====] - 5s 105ms/step - loss: 0.2230 -
accuracy: 0.9300 - val_loss: 0.2612 - val_accuracy: 0.9210
Epoch 7/10
47/47 [=====] - 5s 105ms/step - loss: 0.1868 -
accuracy: 0.9400 - val_loss: 0.2530 - val_accuracy: 0.9270
Epoch 8/10
47/47 [=====] - 5s 107ms/step - loss: 0.1685 -
accuracy: 0.9467 - val_loss: 0.2489 - val_accuracy: 0.9230
Epoch 9/10
47/47 [=====] - 5s 103ms/step - loss: 0.1503 -
accuracy: 0.9540 - val_loss: 0.2457 - val_accuracy: 0.9230
Epoch 10/10

```

47/47 [=====] - 5s 101ms/step - loss: 0.1424 - accuracy: 0.9587 - val_loss: 0.2458 - val_accuracy: 0.9250

Test accuracy: 0.9160

Model: "ViT"

Layer (type)	Output Shape	Param #
patch_embedding_1 (PatchEmbedding)	multiple	3200
dropout_31 (Dropout)	multiple	0
block_0 (TransformerEncoder Block)	multiple	50112
block_1 (TransformerEncoder Block)	multiple	50112
block_2 (TransformerEncoder Block)	multiple	50112
block_3 (TransformerEncoder Block)	multiple	50112
block_4 (TransformerEncoder Block)	multiple	50112
block_5 (TransformerEncoder Block)	multiple	50112
block_6 (TransformerEncoder Block)	multiple	50112
block_7 (TransformerEncoder Block)	multiple	50112
block_8 (TransformerEncoder Block)	multiple	50112
block_9 (TransformerEncoder Block)	multiple	50112
layer_normalization_61 (LayerNormalization)	multiple	128
dense_121 (Dense)	multiple	650

```
=====
Total params: 506,250
Trainable params: 506,250
Non-trainable params: 0
-----
```

1.3.1 Training Visualisation

```
[10]: fig, axs = plt.subplots(1, 2, figsize=(12, 4))

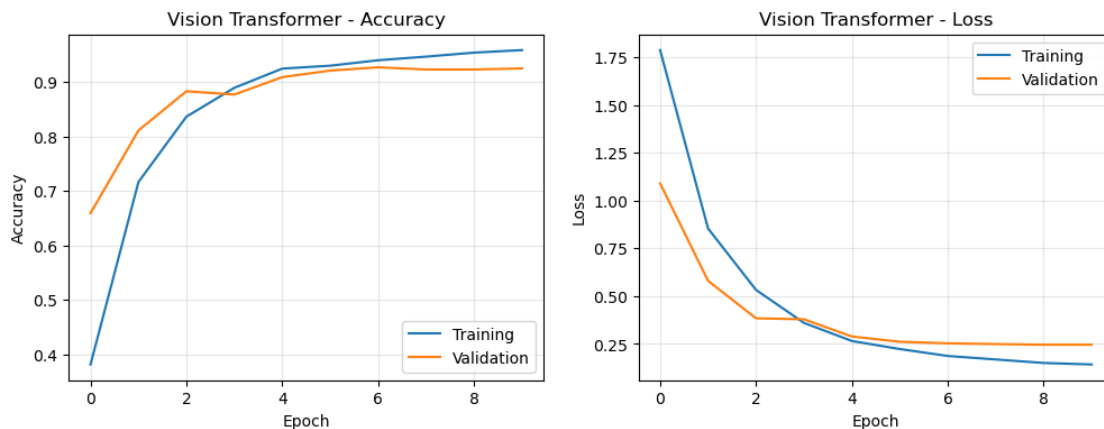
axs[0].plot(vit_history.history['accuracy'], label='Training')
axs[0].plot(vit_history.history['val_accuracy'], label='Validation')

axs[0].set_xlabel('Epoch')
axs[0].set_ylabel('Accuracy')
axs[0].set_title('Vision Transformer - Accuracy')
axs[0].legend()
axs[0].grid(True, alpha=0.3)

axs[1].plot(vit_history.history['loss'], label='Training')
axs[1].plot(vit_history.history['val_loss'], label='Validation')

axs[1].set_xlabel('Epoch')
axs[1].set_ylabel('Loss')
axs[1].set_title('Vision Transformer - Loss')
axs[1].legend()
axs[1].grid(True, alpha=0.3)

plt.show()
```



1.3.2 Visualize Attention Maps

One of the most interesting aspects of transformers is that we can **visualise what the model is attending to!**

Because our VisionTransformer accepts a `return_attention=True` flag, we can call it directly outside of `fit` to retrieve the per-head attention weights.

```
[11]: def visualise_attention(model, image, target, n_heads=4):
    """Visualise attention maps for a single image."""
    # Add a batch dimension; image has shape (28, 28, 1)
    image_tensor = image[np.newaxis, ...]
    probs, attention_maps = model(image_tensor, training=False,
    ↪return_attention=True)
    pred = int(tf.argmax(probs, axis=1).numpy()[0])

    # Get attention from the last transformer block
    # Shape: (batch=1, n_heads, n_tokens, n_tokens)
    last_attn = attention_maps[-1][0].numpy() # (n_heads, n_tokens, n_tokens)

    # Attention from the [CLS] token (index 0) to each patch token.
    # This shows which patches matter for the classification decision.
    cls_attn = last_attn[:, 0, 1:] # (n_heads, n_patches)

    # Reshape to image grid (4x4 for patch_size=7 on 28x28)
    n_patches_per_side = int(cls_attn.shape[1] ** 0.5)

    # fig, axes = plt.subplots(3, n_heads // 3 + 1, figsize=(6, 6))
    fig, axes = plt.subplots(1, n_heads + 1, figsize=(12, 8))
    axes = axes.flatten()

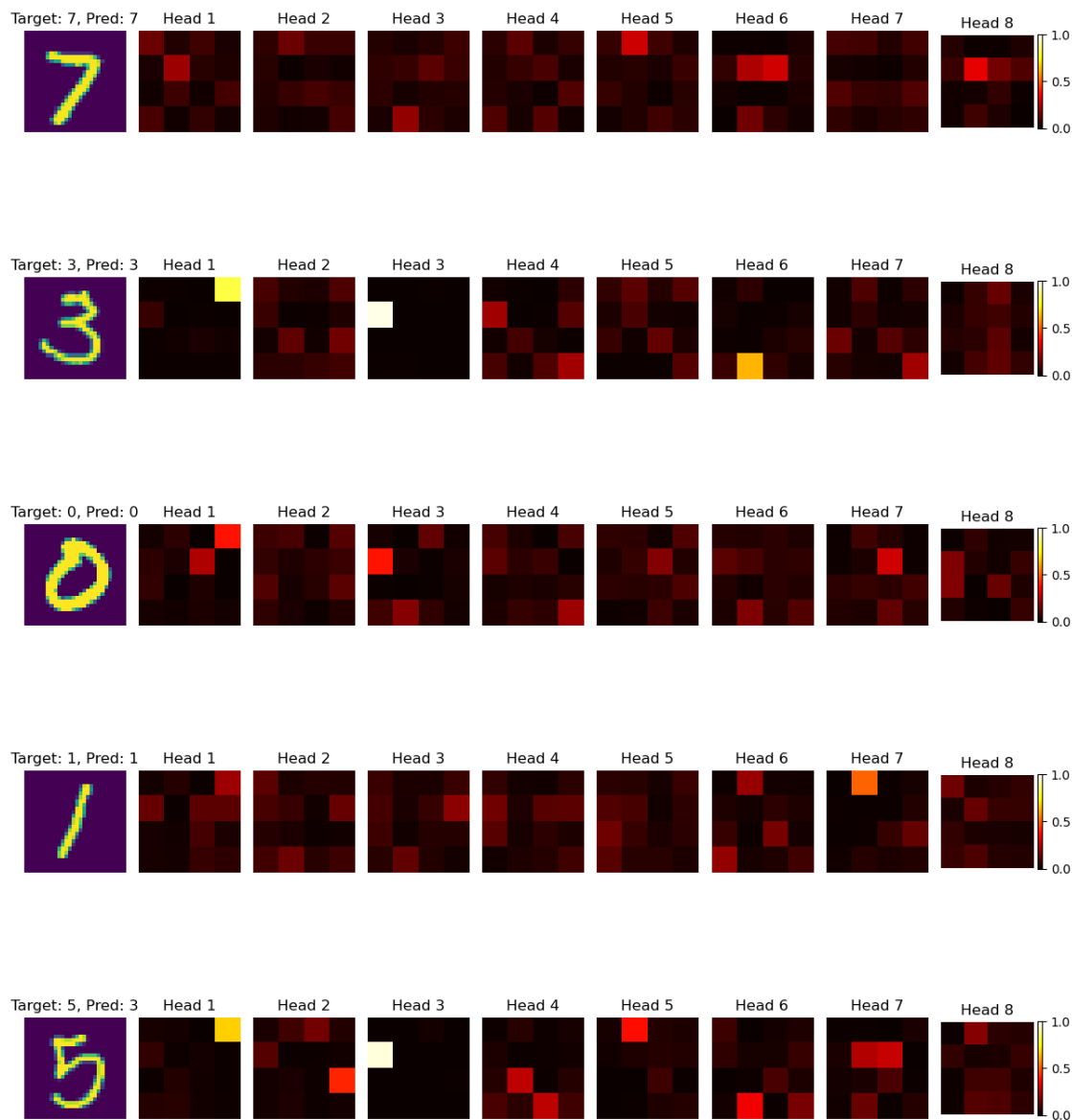
    # Show original image
    axes[0].imshow(image.squeeze())
    axes[0].set_title(f"Target: {target}, Pred: {pred}")
    axes[0].axis('off')

    # Show attention for each head
    for i in range(n_heads):
        attn_map = cls_attn[i].reshape(n_patches_per_side, n_patches_per_side)
        im = axes[i + 1].imshow(attn_map, cmap='hot', vmin=0, vmax=1)
        axes[i + 1].set_title(f'Head {i + 1}')
        axes[i + 1].axis('off')

    fig.colorbar(im, ax=axes[i + 1], fraction=0.046, pad=0.04)

    plt.tight_layout()
    plt.show()
```

```
[12]: for i in range(5):
      visualise_attention(vit, x_test[i], y_test[i], n_heads=8)
```



1.3.3 Compare with a CNN

Let's compare our Vision Transformer with a CNN, similar to the one we have seen before.

```
[13]: keras.utils.set_random_seed(seed)

cnn = keras.Sequential([
    keras.layers.InputLayer(input_shape=(28, 28, 1)),
```

```

    keras.layers.Conv2D(filters=32, kernel_size=(3, 3), padding='same',
↪activation='relu'),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    keras.layers.Conv2D(filters=64, kernel_size=(3, 3), padding='same',
↪activation='relu'),
    keras.layers.MaxPooling2D(pool_size=(2, 2)),
    keras.layers.Flatten(),
    keras.layers.Dense(units=128, activation='relu'),
    keras.layers.Dropout(0.5),
    keras.layers.Dense(units=10, activation='softmax'),
], name='CNN')

cnn.compile(
    optimizer=keras.optimizers.Adam(learning_rate=learning_rate),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)

cnn_history = cnn.fit(
    x=x_train, y=y_train,
    batch_size=batch_size,
    validation_data=(x_val, y_val),
    epochs=num_epochs,
)

cnn_test_loss, cnn_test_acc = cnn.evaluate(x_test, y_test, verbose=0)

```

Epoch 1/10

47/47 [=====] - 2s 22ms/step - loss: 1.2694 - accuracy:
0.5923 - val_loss: 0.4094 - val_accuracy: 0.8810

Epoch 2/10

47/47 [=====] - 1s 19ms/step - loss: 0.4740 - accuracy:
0.8580 - val_loss: 0.2397 - val_accuracy: 0.9370

Epoch 3/10

47/47 [=====] - 1s 18ms/step - loss: 0.3210 - accuracy:
0.9100 - val_loss: 0.1720 - val_accuracy: 0.9400

Epoch 4/10

47/47 [=====] - 1s 17ms/step - loss: 0.2494 - accuracy:
0.9257 - val_loss: 0.1580 - val_accuracy: 0.9560

Epoch 5/10

47/47 [=====] - 1s 19ms/step - loss: 0.2104 - accuracy:
0.9377 - val_loss: 0.1322 - val_accuracy: 0.9610

Epoch 6/10

47/47 [=====] - 1s 19ms/step - loss: 0.1741 - accuracy:
0.9467 - val_loss: 0.1154 - val_accuracy: 0.9640

Epoch 7/10

47/47 [=====] - 1s 18ms/step - loss: 0.1398 - accuracy:

```

0.9557 - val_loss: 0.1130 - val_accuracy: 0.9680
Epoch 8/10
47/47 [=====] - 1s 19ms/step - loss: 0.1425 - accuracy:
0.9563 - val_loss: 0.1106 - val_accuracy: 0.9640
Epoch 9/10
47/47 [=====] - 1s 18ms/step - loss: 0.1130 - accuracy:
0.9653 - val_loss: 0.0945 - val_accuracy: 0.9730
Epoch 10/10
47/47 [=====] - 1s 18ms/step - loss: 0.1050 - accuracy:
0.9700 - val_loss: 0.0898 - val_accuracy: 0.9710

```

1.3.4 Comparison

```

[14]: fig, axs = plt.subplots(3, 1, figsize=(6, 10))

# Accuracy comparison
axs[0].plot(cnn_history.history['accuracy'], label='CNN Train', c='blue',
            linestyle='-', alpha=0.7)
axs[0].plot(cnn_history.history['val_accuracy'], label='CNN Val', c='blue',
            linestyle='--', alpha=0.7)
axs[0].plot(vit_history.history['accuracy'], label='ViT Train', c='red',
            linestyle='-', alpha=0.7)
axs[0].plot(vit_history.history['val_accuracy'], label='ViT Val', c='red',
            linestyle='--', alpha=0.7)
# axs[0].set_xlabel('Epoch')
axs[0].set_ylabel('Accuracy')
axs[0].set_title('Training Accuracy: CNN vs ViT')
axs[0].legend()
axs[0].grid(True, alpha=0.3)

# Loss comparison
axs[1].plot(cnn_history.history['loss'], label='CNN Train', c='blue',
            linestyle='-', alpha=0.7)
axs[1].plot(cnn_history.history['val_loss'], label='CNN Val', c='blue',
            linestyle='--', alpha=0.7)
axs[1].plot(vit_history.history['loss'], label='ViT Train', c='red',
            linestyle='-', alpha=0.7)
axs[1].plot(vit_history.history['val_loss'], label='ViT Val', c='red',
            linestyle='--', alpha=0.7)
# axs[1].set_xlabel('Epoch')
axs[1].set_ylabel('Loss')
axs[1].set_title('Training Loss: CNN vs ViT')
axs[1].legend()
axs[1].grid(True, alpha=0.3)

# Model comparison table
axs[2].axis('off')

```

```

comparison_data = [
    ['Metric', 'CNN', 'ViT'],
    ['Test Accuracy', f"{cnn_test_acc:.4f}", f"{test_acc:.4f}"],
    ['Parameters', f"{cnn.count_params():,}", f"{vit.count_params():,}"],
]

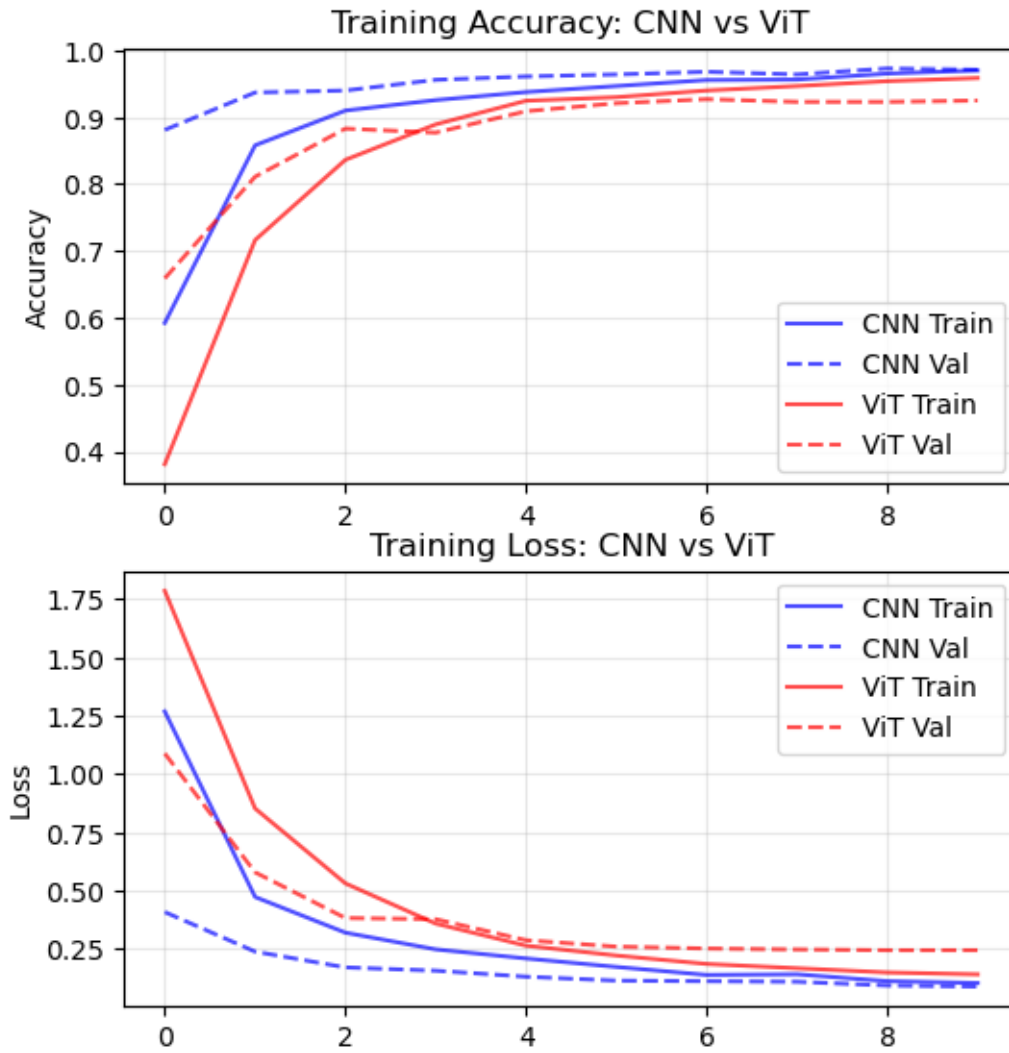
table = axs[2].table(cellText=comparison_data, loc='center', cellLoc='left')
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1, 2)

# Style header row
for i in range(3):
    table[(0, i)].set_facecolor('#40466e')
    table[(0, i)].set_text_props(weight='bold', color='white')

# axs[2].set_title('Model Comparison', fontsize=12, weight='bold')

plt.show()

```

Metric	CNN	ViT
Test Accuracy	0.9600	0.9160
Parameters	421,642	506,250

1.4 Summary

Aspect	CNN	Transformer (ViT)
Receptive Field	Local $\rightarrow$ Global	Global from start
Inductive Bias	Strong (locality)	Weak (learned)
Data Efficiency	High	Low (needs more data)
Interpretability	Limited	High (attention maps)
Scalability	Good	Excellent
Parameters	Fewer	More

So, when could it be a good idea to use transformers?

Good for: - Large datasets - Transfer learning scenarios - When interpretability matters - Long-range dependencies - Multi-modal tasks

But **not ideal for:** - Very small datasets - Limited computational resources - When strong locality bias helps